UNIVERSITÀ DEGLI STUDI DI UDINE
HIC SUNT FUTURA

DIPARTIMENTO DI SCIENZE MATEMATICHE, INFORMATICHE E FISICHE

TESI DI LAUREA IN
INFORMATICA

Generazione agentica di moduli per la cattura dei comandi delle botnet

CANDIDATO
Omar Paviotti

RELATORE
Prof. Marino Miculan

CORRELATORI
Dr. Matteo Paier
Dott. Elia Cal

Anno accademico 2025-2026

Contatti dell'Istituto
Dipartimento di Scienze Matematiche, Informatiche e Fisiche
Università degli Studi di Udine
Via delle Scienze, 206
33100 Udine — Italia
+39 0432 558400
https://www.dmif.uniud.it/

© 2026 Omar Paviotti
This work is shared under the Creative Commons 4.0 License Attribution-NonCommercial-ShareAlike.

Ringraziamenti

Desidero cominciare ringraziando il Professor Marino Miculan, supervisore del mio tirocinio e relatore di questa tesi. La fiducia dimostrata sin dall'inizio e la sua guida preziosa sono state fondamentali per questa esperienza, trasformandola in una significativa opportunità di crescita, sia dal punto di vista accademico che personale. Un sentito ringraziamento va anche a Matteo, per il contributo offerto nella fase di ricerca e nella stesura dell'elaborato.

Ringrazio Elia per il supporto fornito in qualità di esperto di dominio, a partire dalla definizione dell'idea alla base di questo lavoro fino allo sviluppo e alla valutazione del sistema. Un grazie al gruppo del Laboratorio di Cybersecurity, che ha reso possibili le sperimentazioni, e in particolare a Roberto per la sua disponibilità.

Ci tengo a ringraziare Samuele, prima come amico e poi come compagno di corso, per aver condiviso con me questo percorso ed esserci avvicinati insieme, esame dopo esame, alla Laurea.

Voglio quindi ringraziare tutta la mia famiglia, specialmente i miei nonni, che sono sempre stati curiosi di sapere come procedeva il mio cammino universitario. Grazie per avermi costantemente incoraggiato e avermi trasmesso l'importanza dello studio.

Infine, ringrazio Mamma e Papà per avermi dato la possibilità di vivere questi anni con serenità e per il sostegno che mi avete dato ogni giorno. Non dimenticate mai che ci siete anche voi dietro a questo traguardo. Grazie e vi voglio bene.

Indice

1 Introduzione **1**
1.1 Struttura della tesi . . . 2

2 Background **3**
2.1 Malware e botnet . . . 3
2.1.1 Architettura delle botnet . . . 3
2.1.2 Ciclo di vita delle botnet . . . 5
2.1.3 IoT botnet e Mirai . . . 5
2.2 Malware analysis . . . 8
2.2.1 Analisi statica . . . 8
2.2.2 Analisi dinamica . . . 9
2.3 Large Language Model . . . 10
2.3.1 Prompt engineering . . . 11
2.3.2 Agenti AI . . . 11
2.3.3 Model Context Protocol . . . 12
2.4 Software Reverse Engineering assistito . . . 13
2.4.1 LLM nella binary analysis . . . 13
2.4.2 LLM nella malware analysis . . . 15
2.4.3 Interazione Persona-LLM nel SRE . . . 16

3 BotScope: analisi dei requisiti **19**
3.1 Studio manuale e infiltrazione delle botnet . . . 19
3.2 BotScope . . . 20
3.2.1 Requisiti del sistema . . . 21
3.2.2 Proprietà del Capture Module . . . 22

4 BotScope: progettazione **25**
4.1 Architettura del sistema . . . 25
4.2 Componenti del sistema . . . 25
4.2.1 Artefatti . . . 25
4.2.2 Componenti decisionali . . . 27
4.2.3 Integrazione MCP . . . 28
4.2.4 Ambienti di analisi . . . 29
4.3 Flusso di esecuzione . . . 29
4.4 Interazioni di tool calling . . . 31

5 BotScope: implementazione **33**
5.1 Ambiente di sviluppo . . . 33
5.2 Moduli principali . . . 33
5.2.1 Classifier . . . 33
5.2.2 SRE Agent . . . 34
5.2.3 Code Agent . . . 36
5.3 SRE Framework . . . 36

5.3.1 Ghidra MCP Plugin . . . 37
5.3.2 SRE MCP Server . . . 37
5.4 Sandbox Environment . . . 37
5.4.1 Configurazione . . . 37
5.4.2 Sandbox MCP Server . . . 38
5.5 MCP Client . . . 38
5.6 LLM . . . 39
5.6.1 LLM Client . . . 39
5.7 Message Schema . . . 40
5.7.1 Classificazione dei messaggi . . . 40
5.7.2 JSON Schema . . . 40
5.8 Code Templates . . . 41
5.8.1 Modulo Botnet . . . 41
5.8.2 Modulo Message Parser . . . 42
5.9 Prompt Archive . . . 43
5.9.1 Renaming Task . . . 43
5.9.2 Reversing Task . . . 43
5.9.3 Code Agent . . . 44

6 BotScope: valutazione **45**
6.1 Obiettivi e criteri . . . 45
6.2 Configurazione sperimentale . . . 46
6.2.1 Ambiente di esecuzione e LLM . . . 46
6.2.2 Protocollo sperimentale . . . 46
6.3 Definizione dei criteri di valutazione . . . 46
6.3.1 Complessità del campione . . . 46
6.3.2 Efficacia della Renaming Task . . . 47
6.3.3 Accuratezza della Reversing Task . . . 48
6.3.4 Qualità del Capture Module . . . 49
6.4 Campagna sperimentale . . . 50
6.4.1 Dataset dei campioni . . . 50
6.4.2 *Complexity Score* dei campioni . . . 51
6.5 Risultati sperimentali . . . 51
6.5.1 Statistiche di esecuzione . . . 52
6.5.2 Raccolta dei risultati . . . 52
6.6 Discussione dei risultati . . . 53

7 Conclusioni **59**
7.1 Sintesi del lavoro . . . 59
7.2 Limiti e attività rimanenti . . . 60
7.3 Sviluppi futuri . . . 60

A LLM e inferenza **63**
A.1 LLM e configurazione dell'inferenza . . . 63
A.2 Parametri di campionamento . . . 64

B Risorse, artefatti e risultati **65**
B.1 Contenuto e struttura del repository . . . 65
B.2 Accesso al repository . . . 66

Elenco delle tabelle

6.1 Criteri di valutazione della complessità dei campioni malevoli 48
6.2 Criteri di valutazione della Reversing Task 49
6.3 Criteri di valutazione del Capture Module 50
6.4 Caratteristiche principali dei campioni selezionati 51
6.5 *Complexity Score* dei campioni selezionati 52
6.6 Statistiche di esecuzione del sistema 53
6.7 Risultati della valutazione del Classifier 53
6.8 Risultati della valutazione della Renaming Task 54
6.9 Risultati della valutazione della Reversing Task 54
6.10 Risultati della valutazione dei Capture Module 55
6.11 Correlazione tra *Complexity Score* e *Reversing Score* 57

A.1 Parametri di campionamento definiti 64

Elenco delle figure

2.1 Modelli e topologie delle botnet 4
2.2 Ciclo di vita di una botnet 5
2.3 Schema di funzionamento della botnet *Mirai* 7
2.4 Architettura del Model Context Protocol 13
2.5 Framework *End2end-Decompile* e *Refined-Decompile* 14
2.6 Panoramica dell'architettura di *MASDroid* 16

3.1 Creazione e utilizzo del Capture Module 20
3.2 Diagramma di contesto di BotScope 21
3.3 Definizione iniziale dell'interfaccia astratta Botnet Interface 23
3.4 Esempio di uso di un generico Capture Module 23

4.1 Diagramma dei componenti del sistema 26
4.2 Diagramma delle attività del sistema 30
4.3 Diagramma di sequenza delle interazioni del tool calling 32

6.1 Confronto tra i punteggi *Renaming Score*, *Reversing Score* e *Module Score* 55
6.2 Relazione tra i punteggi *Complexity Score* e *Reversing Score* 57

A.1 Configurazioni vLLM utilizzate 64

B.1 QR code per l'accesso al repository 66

1

Introduzione

Le botnet costituiscono da anni una minaccia rilevante nel panorama della sicurezza informatica [1]. L'evoluzione delle superfici di attacco ha favorito la diffusione di botnet composte da dispositivi appartenenti all'Internet of Things (IoT), suscitando un crescente interesse verso il loro studio [2], anche in ragione del loro impiego in attacchi di elevata intensità [3].

Parallelamente, in tempi più recenti i Large Language Model (LLM) hanno assunto un ruolo sempre più centrale in numerosi contesti, grazie alle loro capacità avanzate e trasversali a molteplici domini [4]. Oltre alle applicazioni in ambiti quali la medicina, l'ingegneria e lo sviluppo software, un settore di particolare interesse è rappresentato dalla cybersecurity, dove i LLM e gli agenti LLM-based possono essere impiegati per svolgere attività di scoperta autonoma delle vulnerabilità (red teaming), di difesa dalle minacce (blue teaming) e per affrontare problematiche di sicurezza domain-specific [5].

La presente tesi si colloca proprio all'intersezione tra l'esigenza di analizzare e monitorare le attività delle botnet e le nuove opportunità offerte dai LLM. L'idea alla base del lavoro nasce dall'esperienza maturata dal Dott. Elia Cal, responsabile della gestione di un honeypot proprietario del gruppo di ricerca in cybersecurity dell'Università degli Studi di Udine. Nell'ambito di tale attività, il Dott. Cal si occupa della cattura e dell'analisi di campioni di malware associati a botnet, in particolare rivolte a dispositivi IoT, così da sviluppare moduli destinati all'infiltrazione [6] nell'infrastruttura Command-and-Control. Tali moduli consentono di acquisire le comunicazioni ricevute e i dati scambiati sulla rete, che possono essere successivamente esaminati per individuare i comandi di attacco, i relativi bersagli e, più in generale, i trend evolutivi delle botnet.

Nonostante la maggior parte dei campioni osservati sia riconducibile a mutazioni di poche famiglie di botnet dominanti [7], l'intero processo, quando viene svolto manualmente, risulta particolarmente oneroso. Alla luce di questi aspetti, emerge la domanda di ricerca su cui si fonda la presente tesi: in che misura un sistema autonomo basato su LLM può automatizzare l'analisi del malware e la generazione di moduli per catturare le comunicazioni delle botnet, riducendo l'intervento umano e mantenendo un livello di affidabilità sufficiente per l'utilizzo operativo?

Per rispondere a tale domanda, questa tesi introduce BotScope, un sistema autonomo progettato per effettuare il Software Reverse Engineering (SRE) di campioni di malware e creare automaticamente i corrispondenti moduli di cattura. Il sistema è in grado di operare su campioni di malware potenzialmente mai osservati in precedenza, senza richiedere informazioni specifiche su di essi e senza un intervento

umano diretto, affidandosi principalmente alle capacità e alle conoscenze dei LLM.

La progettazione e l'implementazione di BotScope affrontano in particolare il requisito di garantire la sua autonomia e la possibilità di integrarlo con sistemi esterni. A questi si aggiunge la necessità di assicurare trasparenza e tracciabilità delle operazioni eseguite e dei risultati prodotti, elementi essenziali per orientare i possibili sviluppi futuri del sistema.

BotScope è stato infine sottoposto a una valutazione sperimentale basata su campioni rappresentativi dello scenario operativo considerato, con l'obiettivo di misurarne l'accuratezza nello svolgimento delle attività e verificarne l'effettiva applicabilità. I risultati ottenuti hanno evidenziato l'utilità del sistema, suggerendo al contempo che, allo stato attuale, il suo impiego risulti più appropriato se accompagnato da una successiva revisione dell'output da parte dell'utente. Nonostante i limiti riscontrati, gli esiti della valutazione hanno confermato la validità dell'approccio al problema basato su LLM. Il lavoro ha messo in luce il loro potenziale per l'automazione, parziale o totale, di attività di questa natura, sebbene permangano margini di miglioramento in termini di affidabilità e accuratezza complessiva.

1.1 Struttura della tesi

Il Capitolo 2 ha lo scopo di presentare i concetti teorici necessari alla comprensione della tesi e dei capitoli successivi. Vengono introdotti gli aspetti essenziali su cui si fonda il lavoro descritto, fornendo il contesto per le scelte progettuali e metodologiche adottate.

Il Capitolo 3 esamina nel dettaglio l'attività di studio delle botnet considerata, individuando le principali criticità legate all'analisi manuale e introducendo BotScope come possibile soluzione. Si passa quindi alla caratterizzazione degli input e degli output previsti e, in seguito, vengono identificati i requisiti e i vincoli che guideranno la realizzazione del sistema.

Il Capitolo 4 presenta l'architettura proposta per BotScope, analizzandola da diversi punti di vista. In particolare, viene dapprima introdotta l'architettura generale attraverso il diagramma dei componenti, per poi approfondire le caratteristiche e le responsabilità di ognuno di essi. Successivamente, viene descritto il flusso di esecuzione del sistema, concludendo con una sezione dedicata alle interazioni basate sul Model Context Protocol.

Il Capitolo 5 è dedicato alla descrizione dello sviluppo del sistema. Per ciascun componente vengono presentate le scelte implementative adottate, con un livello di dettaglio variabile in funzione della loro complessità e della loro rilevanza.

Il Capitolo 6 descrive la campagna sperimentale a cui è stato sottoposto BotScope. Inizialmente, vengono definiti gli obiettivi della valutazione, l'ambiente e il protocollo sperimentale. Sono quindi illustrati i criteri oggettivi adottati per valutare le diverse attività svolte dal sistema e misurarne l'accuratezza. La valutazione, effettuata su un insieme selezionato di campioni di malware, porta infine alla presentazione e alla discussione dei risultati ottenuti e delle relative implicazioni.

Il Capitolo 7 è dedicato alle conclusioni del lavoro svolto, con particolare attenzione ai principali limiti e agli aspetti che richiedono ulteriori approfondimenti. Infine, vengono delineate alcune possibili direzioni per il futuro sviluppo di BotScope.

2

Background

2.1 Malware e botnet

In generale, qualsiasi software progettato per compromettere la riservatezza, l'integrità o la disponibilità di un sistema informatico, di una rete o dei dati di un utente può essere classificato come *malware* [8], abbreviazione di *malicious software*. Il malware può assumere diverse forme e, in base al proprio obiettivo, può appartenere a una o più categorie, tra cui, ad esempio, *backdoor*, *information stealer*, *launcher*, *worm*, *virus* e altre.

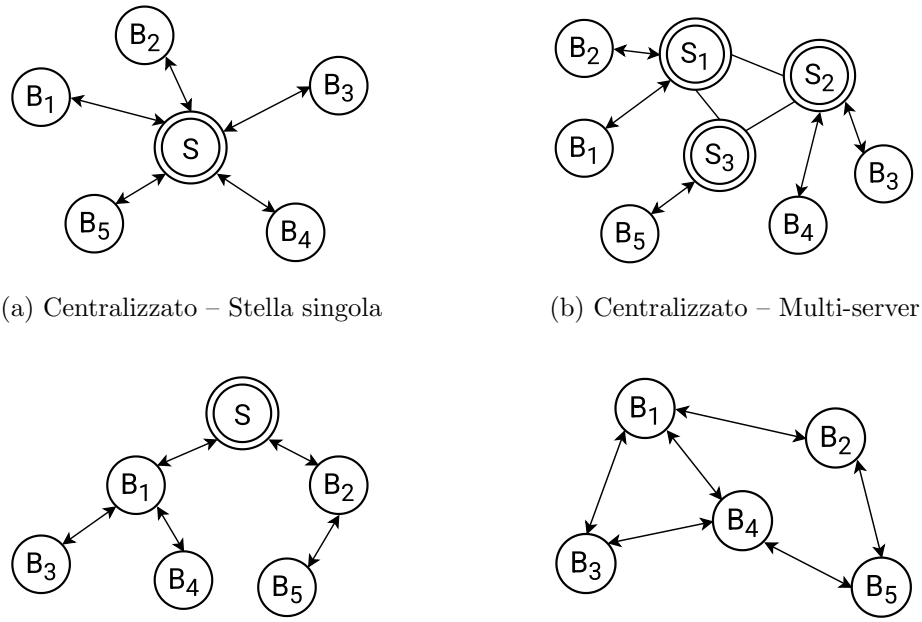
Nel corso degli anni, l'evoluzione delle infrastrutture di rete e il cambiamento delle abitudini tecnologiche hanno portato a un aumento del numero di dispositivi connessi e del tempo durante il quale essi rimangono online. Questo fenomeno, unito alla presenza di vulnerabilità di sicurezza nei sistemi, ha favorito lo sviluppo e la diffusione di nuovi vettori di attacco, tra cui le botnet, oggi considerate una delle principali minacce nel panorama della sicurezza informatica.

Una *botnet* [1] è una rete di host compromessi, detti *bot*, controllati da remoto da uno o più attaccanti, chiamati *botmaster*. Il malware installato sul dispositivo consente a questi ultimi di impartire comandi e di eseguire azioni malevole.

Dal punto di vista degli attaccanti, le botnet rappresentano uno strumento estremamente redditizio, poiché consentono di ottenere un ritorno economico attraverso lo sfruttamento dei dispositivi compromessi. Una volta infettati, tali dispositivi possono essere impiegati simultaneamente per eseguire attività malevole su larga scala, generando danni economici per utenti, aziende e Internet Service Provider (ISP). Le botnet possono fungere da infrastruttura per numerose attività criminali online, tra cui invio di spam, distribuzione di malware, furto di informazioni e attacchi *Distributed Denial of Service* (DDoS). Tra queste, gli attacchi DDoS rappresentano uno degli utilizzi più diffusi, poiché consentono di sfruttare la capacità trasmissiva complessiva dei dispositivi compromessi per sovraccaricare il bersaglio e rendere i suoi servizi indisponibili.

2.1.1 Architettura delle botnet

Il componente fondamentale di una botnet è l'infrastruttura *Command-and-Control* (C&C) [1], che permette la comunicazione tra il botmaster e i bot compromessi, coordinandone le attività. Attraverso il canale C&C vengono trasmessi i comandi che i bot interpretano ed eseguono sul dispositivo infetto.



(a) Centralizzato – Stella singola

(b) Centralizzato – Multi-server

(c) Centralizzato – Gerarchica

(d) Distribuito – Casuale

Figura 2.1: Modelli e topologie delle botnet. Adattata da [9].

Per realizzare tale comunicazione possono essere impiegati diversi protocolli di rete, che, così come le funzionalità implementate, variano in funzione degli obiettivi della botnet. Oltre alla trasmissione dei comandi, il canale C&C consente di mantenere il controllo dei bot, distribuire aggiornamenti del malware e raccogliere informazioni dai dispositivi compromessi.

L'architettura dell'infrastruttura C&C influenza direttamente la robustezza e la capacità di sopravvivenza della botnet. Le architetture possono essere ricondotte principalmente a tre modelli [1, 9]: centralizzato, distribuito e ibrido.

Nel *centralized C&C model*, il botmaster individua uno o più host che fungono da server C&C e costituiscono il punto di contatto per i bot compromessi, secondo tre principali configurazioni topologiche. La topologia a *stella singola* prevede un unico server C&C al quale si collegano tutti i bot, secondo un'organizzazione analoga al modello *client-server* (Figura 2.1a). La topologia a *stella multi-server* utilizza invece più server C&C interconnessi, tra i quali vengono distribuiti i bot, favorendo una maggiore ridondanza e scalabilità dell'infrastruttura (Figura 2.1b). Infine, la topologia *gerarchica* prevede che alcuni bot assumano la funzione di *proxy server*, fungendo da intermediari tra gli altri bot e i server C&C, riducendo il numero di bot che conoscono direttamente l'indirizzo del server C&C (Figura 2.1c).

Il *distributed C&C model* adotta invece un'architettura decentralizzata, tipicamente basata su reti *peer-to-peer* (P2P), dove non esiste un server C&C centrale, i bot comunicano tra loro e il botmaster impartisce i comandi collegandosi a un nodo qualsiasi della rete (Figura 2.1d). Sebbene questo modello sia generalmente più complesso da realizzare, l'infrastruttura è più resistente alle attività di rilevamento e smantellamento.

Infine, l'*hybrid C&C model* combina, secondo diversi approcci, elementi dei modelli centralizzato e distribuito, spesso assegnando ruoli differenti ai nodi.

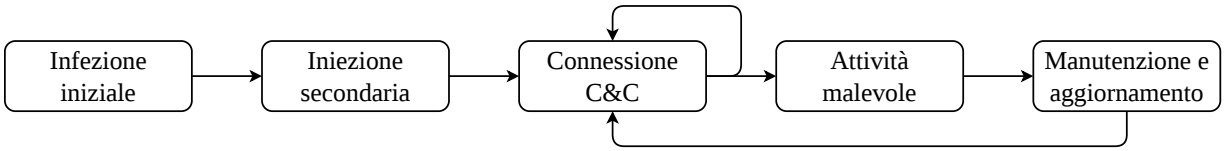


Figura 2.2: Ciclo di vita di una botnet. Adattata da [1].

2.1.2 Ciclo di vita delle botnet

Secondo il modello proposto da Silva et al. [1] e rappresentato in Figura 2.2, il ciclo di vita di una botnet può essere generalmente suddiviso in cinque fasi, realizzate secondo modalità differenti in funzione dell'architettura C&C adottata:

Infezione iniziale: un host viene compromesso attraverso dei vettori di attacco comuni e diventa un potenziale bot.

Iniezione secondaria: il malware iniziale, che può essere un *dropper* o un *downloader*, scarica o attiva il componente principale della botnet, trasformando il dispositivo in un bot controllabile dal botmaster.

Connessione C&C: il nuovo bot stabilisce una connessione con l'infrastruttura C&C per registrare la propria presenza e per mettersi in attesa di istruzioni. L'individuazione dei nodi o degli *endpoint* dell'infrastruttura C&C può avvenire tramite metodi statici (indirizzi IP o nomi di dominio *hardcoded*) oppure con tecniche più sofisticate, come i *Domain Generation Algorithm* (DGA).

Attività malevole: il bot si trova nella fase operativa, è pronto per ricevere i *payload* di comando ed eseguire le attività malevole ordinate dal botmaster.

Manutenzione e aggiornamento: alla fine la botnet entra nella fase di manutenzione, che ha l'obiettivo di garantire la sua sopravvivenza nel lungo periodo. Il botmaster può inviare aggiornamenti del malware, con lo scopo di correggere vulnerabilità o aggiungere nuove funzionalità.

Il modello C&C, in particolare nelle architetture centralizzate, introduce però una vulnerabilità intrinseca [1]: la necessità per i bot di comunicare con l'infrastruttura di comando espone l'attività malevola all'osservazione, costituendo un punto debole sfruttato dalle metodologie di rilevamento. Il traffico delle botnet analizzato include l'*handshake* iniziale, con lo scambio di identificativi e l'eventuale *profiling* dell'host compromesso, il *beaconing* periodico verso un *C&C endpoint* per segnalare la disponibilità del bot (*keepalive*), oltre ai payload di comando e attacco.

2.1.3 IoT botnet e Mirai

La progressiva diffusione delle tecnologie e dei dispositivi appartenenti all'*Internet of Things* ha contribuito ad ampliare la superficie di attacco e, di conseguenza, le opportunità per la proliferazione delle botnet [2]. L'adozione dell'IoT ha infatti interessato un numero crescente di settori, dalle applicazioni domestiche a quelle industriali, e i dispositivi IoT possono raccogliere, elaborare e trasmettere attraverso Internet ingenti quantità di dati, spesso di grande valore e caratterizzati da requisiti di sicurezza e riservatezza particolarmente stringenti.

Le caratteristiche dell'ecosistema IoT rappresentano uno dei principali fattori che possono esporre questi dispositivi vulnerabili alla compromissione da parte degli attaccanti [2]. Molti di essi presentano risorse computazionali e capacità di memoria limitate, rendendo più complessa l'adozione di meccanismi di sicurezza avanzati senza incidere sulle loro funzionalità e prestazioni. A ciò si aggiungono configurazioni di sicurezza talvolta insufficienti, che possono trasformare i dispositivi in punti deboli dell'ecosistema, facilitandone l'arruolamento all'interno di una botnet.

Questi elementi hanno favorito la diffusione delle *IoT botnet* e degli attacchi condotti con esse, principalmente di tipo DDoS. Le IoT botnet rappresentano una sfida rilevante per la sicurezza informatica anche dal punto di vista del rilevamento, poiché, a differenza di alcuni attacchi condotti attraverso botnet tradizionali, possono generare traffico proveniente da indirizzi di rete legittimi e con caratteristiche difficili da distinguere dal traffico normale.

L'impatto potenziale delle IoT botnet è stato evidenziato in modo significativo dalla diffusione a partire dall'agosto 2016 della botnet Mirai. In pochi mesi, Mirai ha compromesso centinaia di migliaia di dispositivi ed è stata impiegata per alcuni dei più grandi attacchi DDoS osservati fino ad allora, colpendo servizi e organizzazioni di rilievo in Europa e negli Stati Uniti [2, 10]. La minaccia rappresentata dalle botnet, in particolare nell'ambito IoT, rimane tuttavia attuale [3]: nel 2025 sono state osservate nuove generazioni di varianti della famiglia Mirai, con oltre trentamila dispositivi IoT compromessi e una capacità di attacco stimata fino a 30 $Tbps$.

Mirai [10] è una famiglia di malware di tipo *worm-like* e rappresenta un'evoluzione della preesistente botnet *BASHLITE*, conosciuta anche come *Gafgyt*, che infettava dispositivi Linux attraverso il *brute force* di credenziali predefinite. La sua rapida espansione è stata favorita dalla semplicità del malware, dall'efficienza della scansione della rete e, soprattutto, dallo sfruttamento di credenziali predefinite deboli o mai modificate sui dispositivi IoT. La sua architettura è centralizzata e prevede tre componenti principali: il C&C, il *report server* e il *loader*, che costituiscono l'infrastruttura interposta tra i bot e il botmaster. Il flusso operativo della botnet è illustrato nella Figura 2.3.

La propagazione di Mirai avviene attraverso un rapido meccanismo di scansione che invia in modo asincrono e *stateless* pacchetti TCP-SYN verso indirizzi IPv4 pseudocasuali, sulle porte 23 e 2323, utilizzate dal protocollo Telnet. Quando viene identificata una potenziale vittima, Mirai tenta di stabilire una connessione mediante un attacco di tipo brute force, provando diverse coppie di username e password selezionate da un pool di 62 credenziali. Rispetto a BASHLITE, il set di credenziali viene ampliato con combinazioni specifiche per router consumer e dispositivi IoT.

Una volta completato con successo il login, Mirai invia al report server l'indirizzo IP della vittima e le relative credenziali. Questo evento attiva il loader, che procede all'infezione del dispositivo vulnerabile eseguendo il login, identificando l'architettura e l'ambiente di esecuzione e scaricando il binario specifico per il sistema. Al termine dell'infezione, il binario scaricato viene eliminato e il malware procede all'offuscamento del nome del processo e all'interruzione dei processi associati a esecuzioni o infezioni concorrenti, comprese altre varianti di Mirai. A questo punto, il bot effettua la scansione di nuove potenziali vittime e, simultaneamente, rimane in ascolto dei comandi provenienti dal C&C, che possono essere utilizzati per condurre attacchi DDoS.

Un evento significativo nella storia di Mirai è stato il rilascio pubblico del codice sorgente nell'ottobre 2016 [10], che ha favorito la proliferazione di nuove varianti e la rapida introduzione di nuove funzionalità.

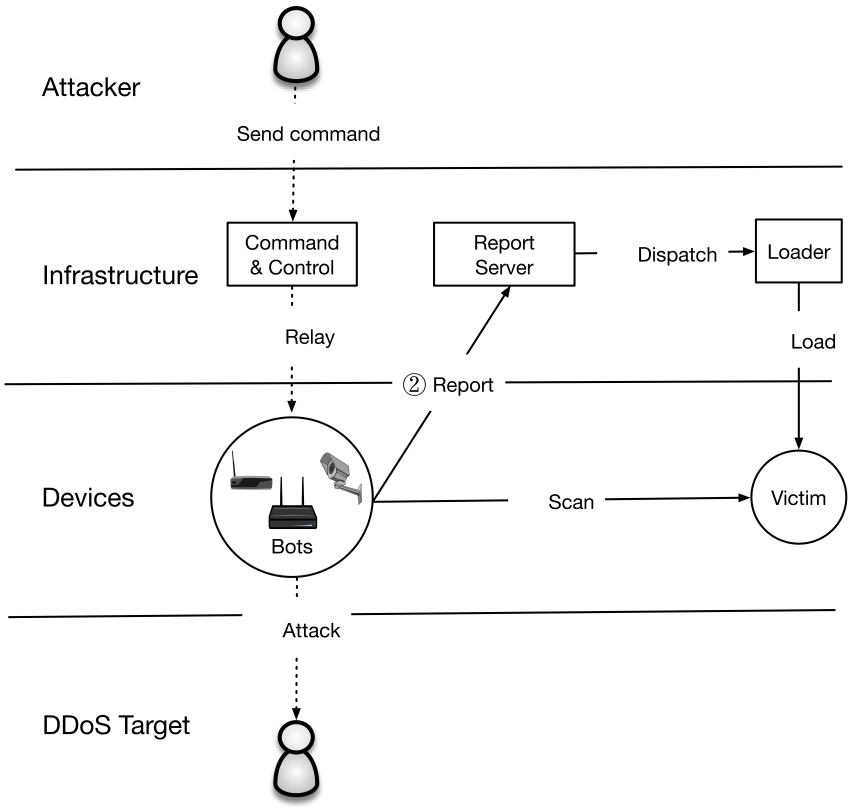


Figura 2.3: Schema di funzionamento della botnet *Mirai*. Adattata da [10].

Queste spaziano dal miglioramento delle capacità di infezione, attraverso l'ampliamento delle credenziali utilizzate e il potenziamento dei meccanismi di protezione dalle infezioni concorrenti, fino all'impiego di binari progettati per complicare l'attività di SRE, ad esempio mediante tecniche di packing.

Sebbene Mirai continui a essere impiegata, oggi il principale rischio, come già osservato in precedenza, è rappresentato dalle sue varianti emergenti. Queste possono derivare sia da famiglie di malware preesistenti, come Gafgyt, che hanno incorporato moduli o caratteristiche di Mirai, sia dallo sviluppo di nuove famiglie basate sul suo codice sorgente. Tra le varianti successive più rilevanti si annoverano [7]:

Hajime (2016): utilizza una blacklist per escludere determinati indirizzi IP dagli attacchi, supporta l'aggiornamento di funzionalità e adotta una comunicazione peer-to-peer, basata su BitTorrent.

Satori (2017): per la sua diffusione sfrutta vulnerabilità del protocollo Universal Plug and Play (UPnP) presenti in router Huawei e dispositivi basati sul Realtek SDK, senza ricorrere allo scanning. Da Satori è derivata a sua volta *Coin Robber*, che sfrutta vulnerabilità di Claymore Miner per il furto di criptovalute.

IoT Reaper (2018): sfrutta vulnerabilità note nei servizi HTTP dei dispositivi IoT e, a differenza della connessione C&C persistente di Mirai, effettua periodicamente richieste HTTP non cifrate.

Mozi (2019): combina caratteristiche di Gafgyt, Mirai e IoT Reaper, adottando un'architettura peer-to-peer, gestita con un protocollo basato su Distributed Hash Table (DHT). Questo viene utilizzato per la comunicazione tra i nodi e per veicolare payload malevoli, rendendo più difficile il loro rilevamento.

2.2 Malware analysis

La *malware analysis* [8] è la disciplina che si occupa di analizzare i malware al fine di identificarne le funzionalità, comprenderne il comportamento e sviluppare adeguate contromisure per il rilevamento e la mitigazione delle minacce a essi legate.

Nel contesto della risposta a un incidente di sicurezza, la malware analysis consente di raccogliere le informazioni necessarie per ricostruire quanto accaduto, determinando gli obiettivi, le capacità e il comportamento del malware. Quando l'analista ha a disposizione un campione del malware, generalmente si tratta del file eseguibile, mentre non ha accesso al relativo codice sorgente. La comprensione del suo funzionamento richiede l'impiego di tecniche di *Software Reverse Engineering* (SRE), che consentono di ricostruire la logica del programma a partire dal codice compilato. Le tecniche di SRE si dividono principalmente in due categorie, analisi statica e analisi dinamica, basate su approcci complementari.

2.2.1 Analisi statica

La *static analysis* [8] consiste nell'esaminare un malware senza eseguirlo, analizzandone la struttura e il contenuto al fine di ricavarne informazioni sul funzionamento.

Le tecniche di analisi statica di base comprendono l'utilizzo di strumenti antivirus per una prima classificazione del campione, il calcolo di hash per identificarlo univocamente e l'ispezione delle informazioni contenute nell'eseguibile, quali stringhe, intestazioni del formato binario, librerie importate o esportate. Sebbene queste tecniche consentano di ottenere rapidamente informazioni preliminari, da sole raramente permettono di comprendere completamente il comportamento del malware.

Analisi statica del codice

L'analisi statica avanzata si basa sull'ispezione delle istruzioni contenute nel file eseguibile del malware. Poiché tali istruzioni sono espresse in linguaggio macchina e non sono direttamente leggibili dal reverse engineer, viene utilizzato un *disassembler* [8], uno strumento che traduce il codice macchina nel corrispondente linguaggio assembly. Quest'ultimo fornisce una rappresentazione testuale delle istruzioni della CPU, permettendo di analizzare le operazioni eseguite dal programma a basso livello.

Un ulteriore strumento utilizzato nell'analisi dei binari è il *decompiler* [8], che tenta di tradurre il codice macchina in una rappresentazione equivalente espressa in un linguaggio di alto livello. Sebbene il risultato non corrisponda necessariamente al codice sorgente originale, questa rappresentazione introduce un maggiore livello di astrazione e può facilitare la comprensione della logica del programma.

Due dei più noti *framework* di SRE sono IDA Pro e Ghidra [8]. Questi software offrono un ambiente completo per l'analisi dei binari, affiancando al disassembler funzionalità di decompilazione, debugging e strumenti di supporto come la navigazione del codice, l'analisi delle funzioni e la ricostruzione dei tipi. Tuttavia, le esigenze degli sviluppatori e dei reverse engineer possono richiedere l'introduzione di funzionalità aggiuntive non direttamente disponibili. Per questo motivo Ghidra, così come altri framework di SRE, mette a disposizione meccanismi di scripting basati sull'*Application Programming Interface* (API) della piattaforma, che consente di interagire con le funzionalità interne del software [11]. Attraverso l'API è possibile creare ed eseguire script in Java e Python per automatizzare operazioni, sviluppare strumenti di analisi ed estendere le funzionalità della piattaforma mediante plugin dedicati.

Limitazioni

L'analisi statica del codice consente di ottenere una comprensione molto più approfondita del malware rispetto alle tecniche di analisi di base. Tuttavia, presenta alcune limitazioni significative.

Una prima limitazione è rappresentata dall'elevato livello di competenza richiesto per svolgere attività di SRE. Il reverse engineer deve possedere una solida conoscenza dell'architettura del processore, del linguaggio assembly e dei meccanismi con cui i compilatori traducono i costrutti ad alto livello in codice macchina, oltre ad avere una buona esperienza nell'utilizzo degli strumenti di analisi.

Una seconda limitazione è intrinseca all'analisi statica stessa. Alcuni comportamenti del malware dipendono infatti da informazioni disponibili esclusivamente a runtime, come parametri di esecuzione, configurazioni del sistema, dati ricevuti dalla rete o indirizzi risolti dinamicamente, che non possono essere determinati osservando unicamente il codice.

A tali limitazioni si aggiungono numerose tecniche impiegate dagli autori dei malware con l'obiettivo di ostacolare le attività di SRE. Tra le più comuni rientra la *code obfuscation* [12], che modifica la rappresentazione e la struttura del programma preservandone il comportamento. L'offuscamento può riguardare il flusso di controllo (*control-flow obfuscation*), attraverso trasformazioni che rendono più complessa la ricostruzione delle relazioni tra le istruzioni; la posizione dei dati (*data-location obfuscation*), rendendo più difficile determinare dove sono memorizzate variabili e altre informazioni utilizzate dal programma; oppure l'utilizzo dei dati (*data-usage obfuscation*), ostacolando la ricostruzione delle relazioni tra definizione e uso dei valori.

Un'ulteriore tecnica è quella del *self-modifying code*, nella quale il malware genera, decifra o modifica porzioni del proprio codice durante l'esecuzione, alterandone dinamicamente la struttura. Molto diffuso è il *packing* [13], che trasforma il programma originale mediante compressione, cifratura o altre tecniche, sostituendolo con un eseguibile contenente uno *stub* incaricato di ripristinare il codice originale.

Infine, molti malware vengono distribuiti solo dopo essere stati sottoposti a un processo di *stripping* [11], attraverso il quale vengono rimossi simboli, nomi delle funzioni, informazioni di debug e altri metadati dal file eseguibile. La conseguente perdita di informazioni semantiche rende più difficile la ricostruzione della logica del programma durante il processo di SRE.

Poiché tali tecniche sono ampiamente note e consolidate, è comune che i malware moderni ne adottino una o più contemporaneamente, con l'obiettivo di aumentare la complessità e il tempo richiesto per l'analisi del campione.

2.2.2 Analisi dinamica

La *dynamic analysis* [8] consiste nell'analizzare il malware durante la sua esecuzione, osservandone direttamente il comportamento.

L'esecuzione deve ovviamente avvenire all'interno di un ambiente controllato e progettato appositamente per l'analisi, in modo da evitare che il malware possa compromettere il sistema del reverse engineer o la rete circostante. L'ambiente di esecuzione viene generalmente configurato in funzione degli obiettivi dell'analisi e, in alcuni casi, come nello studio delle botnet, è necessario abilitare l'accesso a Internet. Questa scelta espone il reverse engineer ai rischi connessi all'interazione del malware con

l'esterno, come la possibilità di allertare gli autori riguardo all'attività di analisi o, nel caso peggiore, di consentire inconsapevolmente l'esecuzione delle azioni malevole per cui il malware è stato progettato.

L'esecuzione del malware all'interno di una *sandbox*, ovvero un ambiente protetto, consente di monitorare numerosi aspetti del suo comportamento, tra cui i processi creati, le operazioni sul file system, le modifiche al registro di sistema, le connessioni di rete e il traffico generato.

Un livello di analisi ancora più approfondito è rappresentato dal debugging. Un *debugger* [8] è uno strumento software utilizzato per osservare e controllare l'esecuzione di un programma, fornendo piena visibilità sul suo stato interno. Esso permette ai reverse engineer di ispezionare il contenuto delle locazioni di memoria, i registri del processore e gli argomenti delle funzioni, oltre a seguire il flusso di esecuzione del programma. Inoltre, i debugger consentono di modificare dinamicamente lo stato del programma, permettendo di analizzarne specifici comportamenti o di verificarne il funzionamento in particolari condizioni.

Rispetto alla sola analisi statica, il debugging permette quindi di osservare informazioni disponibili esclusivamente a runtime. Tuttavia, anche questo approccio presenta alcune limitazioni. Oltre ai vincoli imposti dall'ambiente di esecuzione, molti malware implementano tecniche di *anti-debugging* e *anti-virtualization* [8], mediante le quali rilevano di essere eseguiti rispettivamente sotto il controllo di un debugger o all'interno di una macchina virtuale. In tali casi, il malware può modificare il proprio comportamento o interrompere completamente l'esecuzione, con l'obiettivo di ostacolare l'analisi.

2.3 Large Language Model

I *Large Language Model* appartengono alla famiglia della *Generative Artificial Intelligence* (GenAI), un campo dell'intelligenza artificiale che si concentra sulla generazione di contenuti, quali testo, immagini, audio e dati sintetici, a partire dai pattern appresi durante l'addestramento su grandi quantità di dati. I LLM rappresentano la specializzazione della GenAI nel dominio del linguaggio naturale.

Un LLM è un modello di *deep learning*, basato su reti neurali artificiali di grandi dimensioni e addestrato su vaste collezioni di testo mediante tecniche di apprendimento auto-supervisionato e semi-supervisionato. Durante il *training*, il modello apprende le strutture e i pattern linguistici, acquisendo la capacità di comprendere e generare testo in linguaggio naturale.

L'architettura alla base dei moderni LLM è il *Transformer*, introdotto nel 2017 da Vaswani et al. [14], ricercatori di Google. Il Transformer si basa sul meccanismo di *self-attention*, che consente al modello di individuare le relazioni tra i diversi *token*, le unità in cui viene suddiviso il testo. Questi modelli adottano prevalentemente un'architettura *decoder-only*, in cui il testo viene generato in modo autoregressivo, prevedendo un token alla volta sulla base di quelli precedentemente elaborati.

Questa architettura, combinata con la natura *general-purpose* dei LLM, li rende utilizzabili per un ampio insieme di compiti [4, 15]. Tra le attività più comuni rientrano la conversazione in linguaggio naturale, la risposta a domande, la classificazione del testo, la traduzione automatica e il riassunto. I LLM vengono inoltre impiegati per attività più complesse, come la generazione di codice, la produzione di contenuti testuali e l'esecuzione di compiti di ragionamento basati sulle informazioni fornite.

Nonostante i notevoli progressi degli ultimi anni, i LLM continuano a presentare importanti questioni tecniche, etiche e sociali. Tra i principali aspetti rientrano la disponibilità e la qualità dei dati di

addestramento, con problematiche legate alla privacy, alla presenza di contenuti protetti da copyright e alla rappresentatività dei dati. A questi si aggiungono i *bias* derivanti dai dati utilizzati durante il training e la possibilità di produrre informazioni errate o inventate, con la generazione di risposte plausibili ma fattualmente scorrette, fenomeno noto come *hallucination* [15].

2.3.1 Prompt engineering

Inizialmente, l'adattamento di un LLM a compiti specifici avveniva generalmente tramite il *fine-tuning* del modello, ovvero un ulteriore addestramento su dati appositamente selezionati. Oggi, il prompt engineering consente di orientarne il comportamento attraverso la composizione di *prompt*, ossia gli input forniti al modello. Anche piccole variazioni nella loro formulazione possono produrre differenze significative nella qualità e nella tipologia delle risposte generate.

Il *prompt engineering* [15] è la pratica di progettare e formulare dei prompt che consentano ai LLM di raggiungere specifici obiettivi. Il suo scopo è quello di tradurre l'intento dell'utente in istruzioni chiare e strutturate, che il modello possa interpretare correttamente durante l'inferenza, la fase in cui genera la risposta. Oltre alle istruzioni esplicite, hanno un ruolo importante anche il contesto fornito al modello e la cronologia della conversazione. Le principali tecniche di prompt engineering sono [15]:

Zero-shot prompting: il modello riceve esclusivamente le istruzioni necessarie per svolgere il compito, senza alcun esempio. Per la generazione della risposta, il LLM si affida alla conoscenza acquisita durante l'addestramento.

Few-shot prompting: il modello, oltre al compito da svolgere, riceve un insieme di esempi, con l'obiettivo di mostrargli il formato o il pattern da seguire durante la generazione della risposta.

Multi-turn prompting: il modello riceve anche la cronologia dei turni conversazionali precedenti, mantenendo il contesto dell'interazione, così da avere conversazioni più naturali e coerenti. Tuttavia, conversazioni molto lunghe possono introdurre difficoltà legate ai limiti della finestra di contesto e alla difficoltà del modello nell'utilizzare efficacemente le informazioni rilevanti, soprattutto quando queste si trovano nella parte centrale del contesto [16].

2.3.2 Agenti AI

L'evoluzione dei LLM è stata originariamente guidata principalmente dallo *scaling* dei modelli, aumentando il numero di parametri, la quantità di dati di addestramento e le risorse computazionali impiegate durante il training. Più recentemente la ricerca si è concentrata anche sul miglioramento delle capacità di ragionamento durante l'inferenza, mostrando come dedicare più tempo al processo di *reasoning* prima della generazione della risposta possa migliorare le prestazioni del modello nei compiti più complessi.

Questo approccio ha favorito lo sviluppo di modelli addestrati a produrre esplicitamente catene di ragionamento, *Chain of Thought* (CoT), ossia sequenze di passaggi logici che guidano il modello nella risoluzione di un problema prima di formulare la risposta finale. Sebbene tutti i più recenti LLM dispongano di avanzate capacità di reasoning, affidarsi esclusivamente al ragionamento interno presenta un'importante limitazione, poiché il modello può costruire catene logiche coerenti senza poter necessariamente verificarne la correttezza attraverso fonti di informazione o strumenti esterni, aumentando il

rischio di produrre allucinazioni. Allo stesso tempo, consentire a un modello di interagire direttamente con un ambiente esterno senza un adeguato processo di ragionamento può portare a decisioni meno accurate, errori di pianificazione e un utilizzo inefficace degli strumenti disponibili.

Al fine di superare queste limitazioni complementari è stato proposto il framework *ReAct* (*Reasoning and Acting*) [17], con l'obiettivo di risolvere compiti complessi combinando il ragionamento del modello con l'esecuzione di azioni finalizzate alla loro risoluzione. Il funzionamento si basa su un ciclo iterativo composto da tre fasi:

Thought: il modello genera un ragionamento testuale sullo stato corrente del problema, analizzando le informazioni disponibili e pianificando l'azione successiva.

Action: il modello produce un'azione strutturata, tipicamente sotto forma di invocazione di uno strumento o di una funzione disponibile nell'ambiente esterno.

Observation: l'ambiente esterno esegue l'azione richiesta e restituisce il risultato, che viene quindi fornito come input al modello per l'iterazione successiva.

Il paradigma ReAct costituisce la base concettuale della maggior parte dei moderni AI agent. Un *AI agent* è un agente intelligente progettato per perseguire uno o più obiettivi attraverso un ciclo continuo di ragionamento e azione, con diversi livelli di autonomia, interagendo con strumenti e ambienti esterni. In questo contesto, il LLM costituisce il motore decisionale dell'agente, che può essere integrato con componenti aggiuntivi, quali memoria persistente, logiche di pianificazione e un software di orchestrazione, detto *agent harness*, responsabile del coordinamento tra le diverse componenti.

2.3.3 Model Context Protocol

Il *Model Context Protocol* (MCP) [18] è uno standard *open-source*, introdotto da Anthropic nel 2024, che permette di connettere le *AI application* con sistemi esterni.

Prima della sua adozione, gli sviluppatori dovevano implementare connettori specifici per ciascuna sorgente di dati o strumento, spesso *vendor-specific*, con conseguenti problemi di interoperabilità e compatibilità. Il MCP nasce per risolvere questo problema, fornendo un'interfaccia standardizzata che collega le AI application a fonti di informazioni, strumenti e altri flussi di lavoro, e consente ai LLM di accedere a dati esterni e svolgere operazioni attraverso sistemi terzi.

Dal punto di vista architetturale, il protocollo distingue tre ruoli principali [18], come mostrato in Figura 2.4:

MCP Host: l'AI application, come un ambiente di sviluppo o un chatbot avanzato, in cui risiede il LLM e che coordina il flusso di lavoro, creando uno o più Client per interfacciarsi con i Server.

MCP Client: un modulo integrato nell'Host che agisce come intermediario, mantenendo una connessione con un singolo Server, instradando le richieste dell'Host e restituendo i risultati ottenuti.

MCP Server: un programma che espone funzionalità specifiche a uno o più Client. Ogni Server può mettere a disposizione una o più *capability*, classificate in tools e resources.

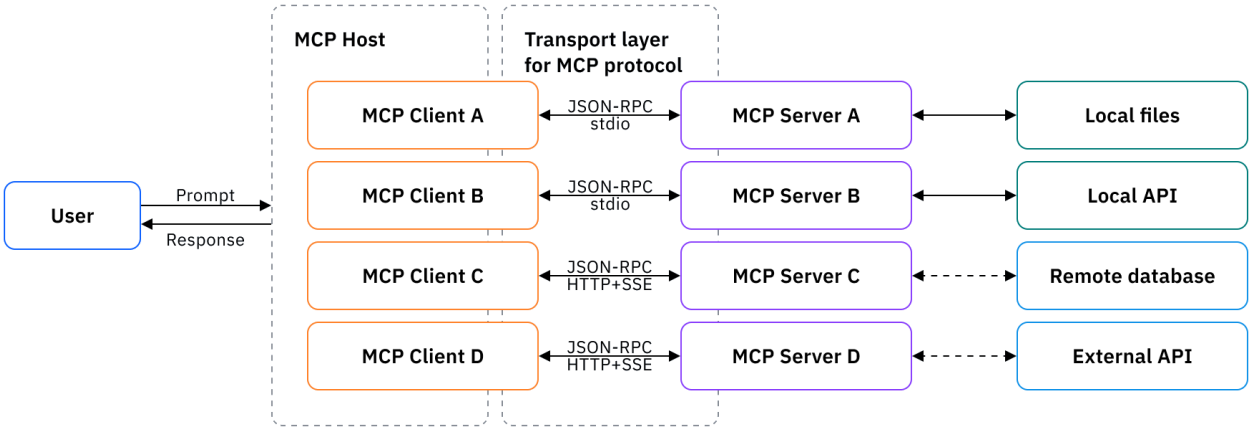


Figura 2.4: Architettura del Model Context Protocol. Adattata da [19].

I *tools* rappresentano operazioni invocabili dal LLM e sono descritti tramite interfacce basate su JSON Schema che definiscono in modo esplicito gli input e gli output previsti. Le *resources*, invece, forniscono accesso strutturato a informazioni esterne e possono essere fornite al LLM come contesto aggiuntivo durante l'esecuzione.

Con riferimento alla comunicazione, il *data layer* definisce il formato e la semantica dei messaggi scambiati tra Client e Server, sulla base di JSON-RPC 2.0. Il *transport layer*, invece, definisce il meccanismo di trasporto e il canale attraverso cui avviene l'invio e la ricezione dei messaggi, occupandosi della gestione della connessione e del *message framing*. Attualmente il protocollo supporta due meccanismi di trasporto: lo *stdio transport*, che utilizza gli stream di input e output standard per la comunicazione tra processi locali sulla stessa macchina e lo *Streamable HTTP transport*, basato su richieste HTTP, che consente la comunicazione con Server remoti.

2.4 Software Reverse Engineering assistito

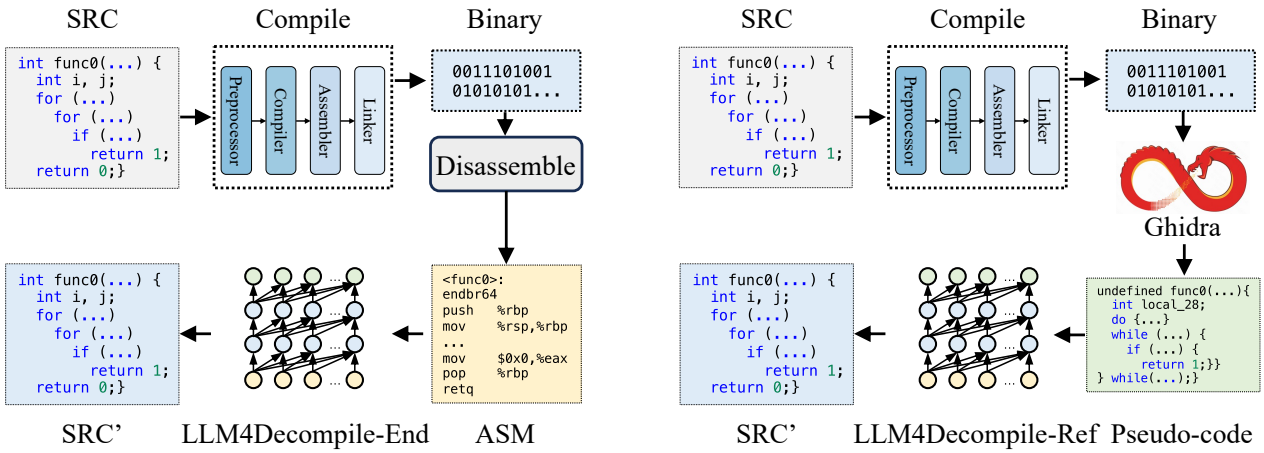
2.4.1 LLM nella binary analysis

I rapidi progressi nel campo dell'AI hanno portato all'impiego dei LLM anche nel SRE, dove le loro capacità di comprensione e generazione del codice possono essere sfruttate dai reverse engineer per supportare il processo di analisi binaria dei software.

In questo contesto si inseriscono i due lavori descritti di seguito.

LLM4Decompile [20]: propone una famiglia di LLM open-source, con dimensioni comprese tra 1.3 e 33 miliardi di parametri, *fine-tuned* per la decompilazione di codice binario in linguaggio C. L'obiettivo è quello di produrre codice sorgente semanticamente corretto e più leggibile rispetto ai decompilatori tradizionali. Lo studio introduce due classi di modelli, che possono essere impiegati in due framework basati su approcci complementari:

- *End2end-Decompile* (Figura 2.5a): dopo il disassemblaggio del codice binario in istruzioni assembly, il LLM genera direttamente il corrispondente codice C, ottenendo risultati superiori sia al decompilatore di Ghidra sia a modelli general-purpose come GPT-4o.



(a) Framework *End2end-Decompile* basato sul LLM *LLM4Decompile-End*

(b) Framework *Refined-Decompile* basato sul LLM *LLM4Decompile-Ref*

Figura 2.5: Framework *End2end-Decompile* e *Refined-Decompile* per la decompilazione del codice binario. Adattata da [20].

- *Refined-Decompile* (Figura 2.5b): dopo il disassemblaggio, il codice viene inizialmente decompilato mediante Ghidra; successivamente, il LLM utilizza il codice decompilato come punto di partenza per migliorarne leggibilità e correttezza semantica. L'intervento non si limita alla semplice revisione dei nomi e dei tipi delle variabili, ma mira a una rielaborazione più generale del codice, aumentando la potenziale eseguibilità rispetto alla sola decompilazione.

I risultati sperimentali ottenuti con LLM4Decompile mostrano come LLM specializzati possano superare gli strumenti tradizionali di decompilazione, evidenziando il potenziale dei LLM come supporto alle attività di SRE.

ReCopilot [21]: presenta un LLM specializzato nelle principali attività dell'analisi statica, progettato per operare come un copilota a supporto del reverse engineer durante il processo di analisi. ReCopilot è stato addestrato attraverso un processo articolato in più fasi:

- *Continued Pre-Training* (CPT): un LLM di base è stato ulteriormente pre-addestrato su un ampio corpus di dati relativi al SRE, al fine di acquisire una conoscenza specialistica del dominio e delle relazioni tra codice binario, assembly, codice sorgente e linguaggio naturale.
- *Supervised Fine-Tuning* (SFT): il modello è stato quindi specializzato su un insieme di task tipiche dell'analisi statica, imparando a seguire le istruzioni dell'utente e a produrre output in formati strutturati. Tra le attività considerate rientrano la decompilazione, l'analisi delle funzioni e dei relativi nomi, parametri e variabili, a cui si aggiungono task come l'identificazione di algoritmi, la classificazione e la generazione di riassunti relativi a codice e funzioni. Il training include inoltre esempi basati su Chain of Thought, con l'obiettivo di guidare il modello nel processo di ragionamento prima della generazione della risposta.
- *Direct Preference Optimization* (DPO): il modello è stato infine ottimizzato rispetto alle preferenze umane, migliorandone la capacità di seguire il formato richiesto e la qualità, la consistenza e la pertinenza delle risposte generate.

Dal punto di vista sperimentale, ReCopilot supera sia gli strumenti tradizionali di SRE, come IDA Pro e Ghidra, sia i LLM general-purpose, pur mantenendo dimensioni contenute. Inoltre, la natura specialistica e strutturata del modello lo rende particolarmente adatto all'integrazione all'interno di sistemi agentici. In questo contesto, ReCopilot può essere impiegato come modulo dedicato all'analisi binaria da parte di un agente di SRE autonomo, al quale può fornire informazioni dettagliate a partire da pseudocodice o istruzioni assembly.

2.4.2 LLM nella malware analysis

Dopo le prime applicazioni dei LLM al SRE in contesti generici, il loro impiego è stato progressivamente esteso anche ad ambiti più specifici della cybersecurity, tra cui la malware analysis.

A questo scopo, sono emerse soluzioni specializzate, come quelle presentate di seguito, progettate per sfruttare le capacità dei LLM a supporto dell'analisi dei campioni malevoli e delle attività di SRE.

Project IRE [22]: sviluppato da Microsoft, rappresenta uno dei primi esempi industriali di applicazione dei LLM alla malware analysis. Il sistema è progettato per classificare automaticamente un file eseguibile come benigno o maligno disponendo esclusivamente del campione binario, senza fare affidamento su metadati, telemetria o altre informazioni relative alla sua provenienza. L'obiettivo è assistere gli analisti nella gestione dell'elevato numero di file da analizzare, riducendo i tempi e il carico operativo.

Project IRE adotta un'architettura agentica in cui un LLM coordina una suite di strumenti specializzati per il SRE, consentendo di combinare diversi livelli di analisi, da quella binaria a basso livello fino all'interpretazione ad alto livello del comportamento del codice. Gli strumenti, accessibili tramite API, comprendono sandbox per l'analisi dei campioni, soluzioni Microsoft e componenti open source, tra cui decompilatori e sistemi per la ricerca e il recupero della documentazione.

Durante l'analisi, il sistema costruisce una *chain of evidence*, ovvero una sequenza di evidenze che documenta il percorso seguito per giungere alla classificazione finale. Questa traccia risulta particolarmente utile durante la revisione da parte degli analisti, poiché consente di verificare le evidenze alla base della classificazione e di individuare i passaggi che possono aver contribuito a un'eventuale decisione errata.

Le valutazioni preliminari del sistema riportano che circa il 90 % dei campioni analizzati è stato classificato correttamente, con una percentuale di falsi positivi prossima al 2 %. Questi risultati suggeriscono la possibilità di impiegare il sistema in scenari operativi, mantenendo, tuttavia, la revisione da parte di analisti esperti come componente del processo di analisi.

MASDroid [23]: propone un framework multi-agente dedicato all'analisi di malware Android. Il sistema combina diversi agenti specializzati con tecniche di analisi comportamentale, producendo una classificazione del malware accompagnata da spiegazioni interpretabili dall'utente.

L'obiettivo del framework è superare alcune delle principali limitazioni degli approcci esistenti, tra cui la possibilità di elusione dell'analisi statica, i costi e i limiti di scalabilità dell'analisi dinamica e, soprattutto, la scarsa trasparenza dei processi decisionali dei sistemi tradizionali. L'architettura di MASDroid, rappresentata anche nella Figura 2.6, può essere suddivisa in tre sezioni principali:

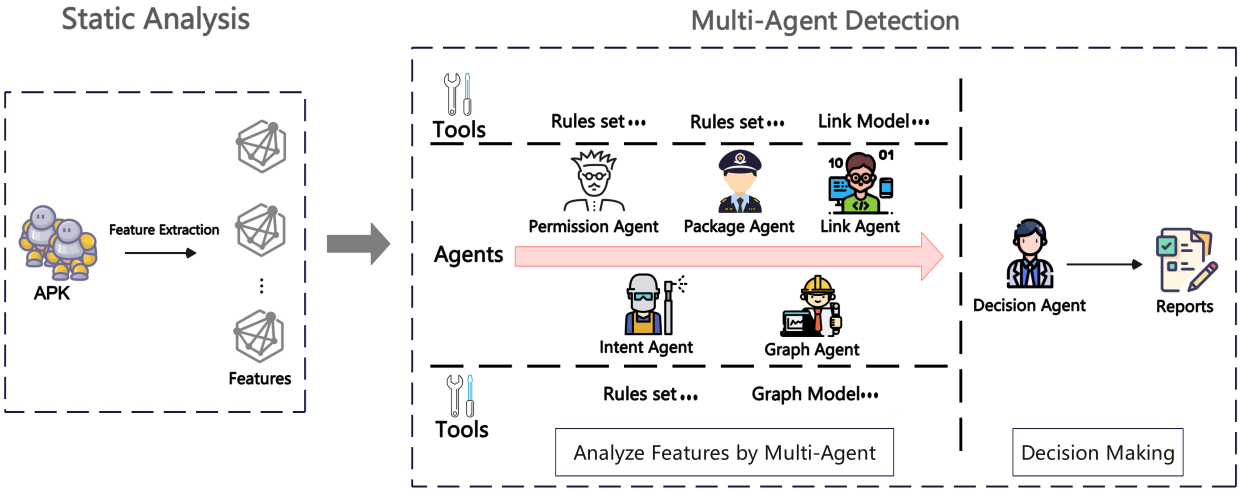


Figura 2.6: Panoramica dell'architettura di *MASDroid*. Adattata da [23], © 2026 IEEE.

- *Static Analysis*: il file APK viene preliminarmente decompilato e analizzato, estraendo un insieme di caratteristiche che viene successivamente fornito ai diversi agenti come base comune per l'analisi.
- *Multi-Agent Collaboration*: i diversi agenti operano in parallelo, occupandosi ciascuno di una specifica attività di analisi e utilizzando, quando necessario, i tool disponibili. Gli output prodotti vengono organizzati secondo un formato strutturato, che combina un *maliciousness score* con una *evidence list*, e vengono successivamente forniti all'agente responsabile della decisione finale.
- *Decision Agent*: l'agente di decisione integra le informazioni provenienti dai diversi agenti ed esegue la classificazione finale. Esso genera inoltre un report strutturato contenente lo score finale, i riferimenti alle evidenze e alle attività dei singoli agenti, rendendo il processo decisionale maggiormente interpretabile da un esperto di cybersecurity.

Le valutazioni sperimentali del sistema mostrano prestazioni superiori al 97 % sia nella rilevazione del malware sia nella classificazione della relativa famiglia. Oltre ai risultati quantitativi, gli autori evidenziano come l'approccio multi-agente migliori l'interpretabilità delle decisioni rispetto ai tradizionali modelli *black-box*, fornendo classificazioni supportate dalle evidenze raccolte durante il processo di analisi.

2.4.3 Interazione Persona-LLM nel SRE

Il crescente coinvolgimento dei LLM nelle attività di SRE evidenzia come lo stato dell'arte della ricerca si stia progressivamente orientando verso l'integrazione di tali modelli all'interno del flusso di lavoro dei reverse engineer.

In questo contesto, tra il 2024 e il 2025, Basque et al. [24] hanno condotto il primo studio sistematico volto a comprendere come i LLM possano collaborare con i reverse engineer durante le attività di SRE. Lo studio ha esaminato le modalità con cui i LLM vengono integrati nel processo di analisi, il loro impatto sul flusso di lavoro dei reverse engineer e la percezione del loro utilizzo.

Per lo svolgimento dello studio, i ricercatori hanno selezionato 48 partecipanti, equamente suddivisi tra reverse engineer con esperienza limitata ed esperti. I partecipanti sono stati quindi coinvolti in un esperimento controllato, nel quale dovevano analizzare due programmi binari progettati in stile *Capture The Flag* (CTF), contenenti sfide rappresentative di comuni attività di SRE. L'esperimento prevedeva due condizioni, rispettivamente con e senza il supporto di un LLM. Nel secondo caso, ai partecipanti veniva messo a disposizione un plugin per IDA Pro con funzionalità specifiche di analisi basate su LLM, oltre a una *chatbox* per interagire liberamente con un LLM.

L'osservazione delle attività svolte dai partecipanti ha portato all'individuazione di 18 risultati sperimentali, successivamente sintetizzati dagli autori in cinque principali conclusioni [24]:

Supporto nelle fasi iniziali: l'uso dei LLM è particolarmente utile nelle fasi iniziali dell'analisi, grazie al fatto che fornisce una visione d'insieme a fronte di un basso costo cognitivo. Questo approccio avvantaggia soprattutto i reverse engineer meno esperti, mentre il beneficio si riduce nel caso di analisi più approfondite.

Impatto limitato sugli esperti: l'apporto dei LLM migliora solo marginalmente le competenze degli esperti, che preferiscono delegargli le attività di routine, concentrandosi sulla comprensione di logiche complesse e ridistribuendo lo sforzo.

Rischio delle allucinazioni: anche delle allucinazioni rare possono compromettere significativamente il flusso di analisi, poiché la formulazione e l'esplorazione di ipotesi errate possono prolungare i tempi complessivi di lavoro.

Recupero degli artefatti vs. comprensione semantica: la comprensione semantica del programma da parte del reverse engineer dipende fortemente dal recupero degli artefatti. Nonostante i LLM aiutino nella ricostruzione di tipi, nomi di funzioni e variabili, questo non si traduce automaticamente in una comprensione profonda del funzionamento del codice.

Limiti nella delegazione delle attività: ai LLM possono essere demandate solo le attività più semplici, come ad esempio la descrizione e la caratterizzazione di funzioni *single-purpose* e di algoritmi standard. Nel caso di funzioni complesse, con più ruoli o dotate di logiche intricate è necessaria un'analisi umana iterativa, eventualmente supportata dai LLM.

Lo studio evidenzia dunque che, in assenza di un avanzamento tecnologico nell'ambito della GenAI, di un miglioramento delle conoscenze specifiche dei LLM e, soprattutto, di un'evoluzione delle modalità di interazione, questi modelli rimarranno per la maggior parte strumenti di supporto per principianti, piuttosto che veri collaboratori alla pari per gli esperti di SRE [24].

Tuttavia, è fondamentale sottolineare che tale conclusione sia stata formulata considerando una modalità di interazione basata su assistenti conversazionali e sullo stato dell'arte della GenAI precedente alle più recenti evoluzioni. La diffusione, tra il 2025 e il 2026, dei sistemi agentici basati su LLM ha contribuito ad affermare un paradigma di interazione differente, nel quale il modello può pianificare autonomamente le attività da svolgere, utilizzare strumenti e adeguare le proprie azioni in base ai risultati ottenuti. Questo approccio può modificare il ruolo dei LLM nel SRE, spostandolo da una funzione prevalentemente assistiva a forme di collaborazione caratterizzate da una maggiore autonomia.

Proprio in questa direzione si sviluppa il lavoro di Radey et al. [25], che analizza le capacità tecniche dei sistemi agentici applicati al SRE e propone una classificazione basata sulle modalità di analisi adottate. Gli autori distinguono tra *static analysis agents*, che si concentrano prevalentemente sull'analisi statica del codice decompilato, e *hybrid agents*, che combinano attività di analisi statica e dinamica. Un'ulteriore distinzione riguarda il grado di autonomia con cui i sistemi operano. Nei sistemi *human-in-the-loop*, il reverse engineer mantiene un ruolo attivo nel processo, verificando le azioni dell'agente e garantendo che l'analisi venga svolta in sicurezza. Nei sistemi autonomi, invece, gli agenti possono condurre autonomamente il processo di analisi, coordinandosi e comunicando tra loro senza un intervento umano diretto.

Nonostante le potenzialità del paradigma agentico, lo studio evidenzia diverse limitazioni dei sistemi attuali negli scenari realistici e complessi di SRE [25]. Le capacità di analisi possono essere compromesse dall'offuscamento del codice, complicandone la ricostruzione semantica, in modo analogo a quanto già avviene nell'analisi condotta da un reverse engineer umano. A ciò si aggiungono i vincoli di contesto dei LLM, la difficoltà di monitorare il processo di ragionamento dell'agente e le problematiche legate alla gestione in sicurezza dell'ambiente di esecuzione. Infine, è necessario considerare il compromesso tra autonomia e supervisione: un ragionamento non supervisionato può portare l'agente a seguire direzioni poco produttive, riducendo l'efficacia dell'analisi, mentre il coinvolgimento umano assicura un maggiore controllo, a scapito dell'autonomia del sistema.

Il paradigma agentico rappresenta dunque una naturale evoluzione dell'impiego dei LLM nel SRE, ma è ancora in una fase iniziale e in rapido sviluppo, come testimonia la crescente attenzione della ricerca verso questi sistemi. La loro efficacia deve ancora essere verificata in maniera estesa in scenari realistici, anche attraverso benchmark capaci di catturare adeguatamente le caratteristiche delle attività tipiche di reverse engineering. Le sfide future non riguarderanno quindi soltanto il miglioramento dei LLM, ma anche la progettazione di agenti AI robusti in grado di sfruttarne pienamente il potenziale, integrandoli efficacemente con gli strumenti e gli ambienti di analisi, così da garantire autonomia e affidabilità al processo di reverse engineering.

3

BotScope: analisi dei requisiti

3.1 Studio manuale e infiltrazione delle botnet

All'interno dell'ambito della sicurezza informatica, la *threat research* si occupa di raccogliere, analizzare e condividere informazioni (*cyber threat information*) riguardanti minacce esistenti o emergenti. Tali evidenze possono essere ulteriormente elaborate fino alla produzione di *threat intelligence reports* [26], documenti che descrivono le *tactics, techniques, and procedures* (TTPs) adottate dai *threat actors*, insieme alle tipologie di sistemi e di informazioni presi di mira.

Una possibile applicazione della *threat research* nel contesto delle botnet consiste nella caratterizzazione e nel reverse engineering del protocollo C&C utilizzato. La ricostruzione del protocollo può consentire di interpretare le comunicazioni tra i bot e l'infrastruttura C&C, permettendo di identificare e analizzare i comandi trasmessi e, più in generale, di studiare il comportamento della botnet.

Per svolgere tale attività, un esperto di cybersecurity, una volta entrato in possesso di un campione di malware appartenente a una botnet, può eseguirne la malware analysis, ricorrendo a tecniche di analisi statica o dinamica, combinandole tra loro e scegliendo quelle più adatte al campione. A tale scopo, l'esperto si avvale delle proprie conoscenze teoriche e della sua esperienza, integrandole, quando necessario, con le informazioni ricavate dalla consultazione di fonti e documentazione disponibili online.

Quando l'esperto ha raccolto tutte le informazioni rilevanti sul protocollo C&C, può realizzare un modulo per la cattura dei messaggi, da qui in avanti indicato come Capture Module. Il Capture Module implementa esclusivamente le funzionalità necessarie per comunicare con l'infrastruttura C&C e simulare il comportamento di un bot, con l'obiettivo di condurre un'attività di *botnet infiltration* [6]. A tal fine, il modulo si connette all'infrastruttura, presentandosi come una potenziale vittima, interagisce con essa e cattura i messaggi ricevuti. Questi possono essere successivamente salvati e utilizzati per analisi più approfondite e su scala più ampia, considerando contemporaneamente più botnet.

Il processo descritto è riassunto nella Figura 3.1, che illustra sia la fase di creazione del modulo, attraverso l'analisi del campione di malware, sia quella del suo utilizzo operativo, durante la quale vengono catturati i messaggi.

Il vantaggio di questo approccio risiede soprattutto nel mantenimento del controllo diretto dell'intero processo, che permette all'esperto di distribuire tempo e risorse tra i diversi aspetti in relazione alla loro

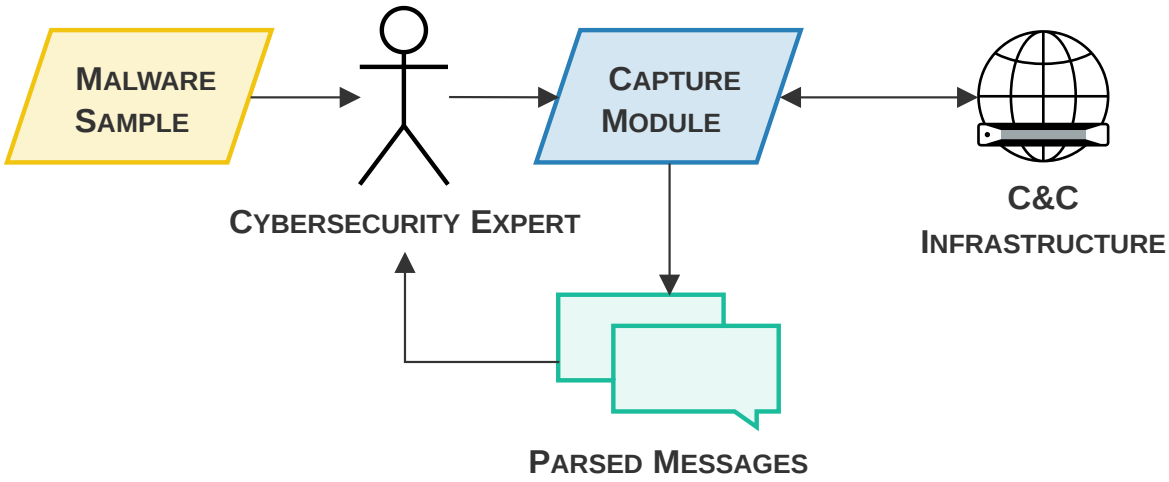


Figura 3.1: Creazione del Capture Module da parte dell'umano e successivo utilizzo per l'infiltrazione e la cattura dei messaggi C&C.

rilevanza e alla difficoltà riscontrata. Tuttavia, il lavoro manuale richiesto rimane oneroso, anche quando l'esperienza acquisita e l'adozione di pratiche volte a velocizzarlo ne migliorano l'efficienza.

Considerando che l'esperto probabilmente analizzerà botnet differenti, nella realizzazione dei Capture Module cercherà, quando possibile, di mantenere un'architettura e un comportamento uniformi, ad esempio adottando un'interfaccia comune e un formato strutturato per i messaggi catturati. Questo evidenzia ulteriormente i limiti dell'approccio manuale, che richiede un processo lungo e spesso ripetitivo, aumentando il carico cognitivo richiesto. Infine, non bisogna sottovalutare il fatto che tale processo possa richiedere competenze specialistiche di malware analysis, che non tutti i threat researcher necessariamente possiedono.

3.2 BotScope

Un'opportunità per ridurre il carico sull'esperto di cybersecurity consiste nell'automatizzare, interamente o parzialmente, le attività di analisi del campione e la successiva creazione del Capture Module. La presente tesi si concentra proprio su questo aspetto, proponendo il sistema BotScope come soluzione al problema descritto.

Il sistema dovrà essere progettato per ricevere in input un file binario arbitrario, di seguito indicato come Binary File, che nella pratica operativa corrisponderà generalmente a un campione di malware in formato eseguibile appartenente a una botnet. Un ulteriore input, di natura opzionale, è rappresentato da una Knowledge Base, il cui utilizzo non deve costituire un prerequisito per il funzionamento di BotScope. Essa può fornire al sistema un insieme di conoscenze utili a supportare e migliorare l'attività di analisi, quali documentazione tecnica, istruzioni operative specifiche, informazioni relative a una determinata famiglia di botnet o altri contenuti ritenuti rilevanti. La natura dell'output restituito dal sistema dipenderà dall'esito della valutazione del Binary File: qualora il file sia riconosciuto come un campione malevolo riconducibile a una botnet e sia ritenuto meritevole di analisi, il sistema dovrà svolgere tutte le attività necessarie alla costruzione del Capture Module; altrimenti, qualora vi sia certezza che non sia di interesse, il processo potrà essere interrotto e il file sarà scartato.

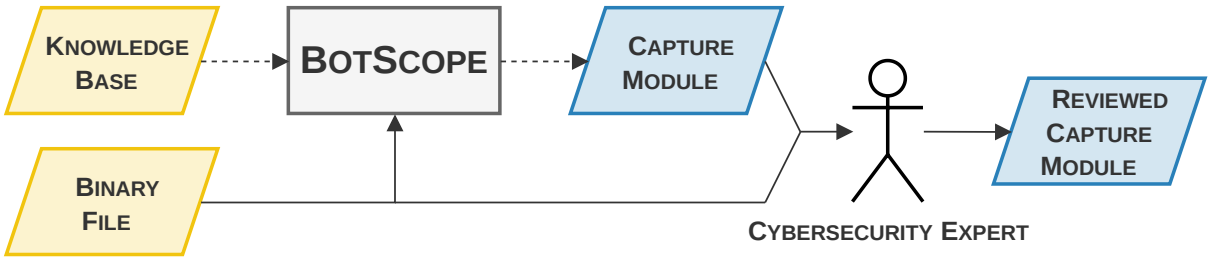


Figura 3.2: Diagramma di contesto del sistema BotScope.

L'obiettivo principale di BotScope consiste quindi nell'acquisire le informazioni necessarie al reverse engineering del protocollo C&C utilizzato, al fine di sviluppare un Capture Module basato sulle proprietà specifiche della botnet in esame. Non è invece richiesta una comprensione completa delle caratteristiche e del funzionamento interno del malware analizzato, anche se ciò potrebbe essere una conseguenza collaterale dell'analisi. BotScope adotta infine un approccio *best-effort*, volto a ottenere il risultato atteso attraverso le attività di analisi previste, senza tuttavia poter garantire, per ogni campione, la piena funzionalità del Capture Module prodotto, né la completezza o l'accuratezza della sua implementazione.

Il diagramma di contesto riportato in Figura 3.2 fornisce una rappresentazione ad alto livello del sistema proposto, considerato come una *black-box*, e dei relativi input e output. Esso evidenzia come BotScope possa essere integrato direttamente all'interno del processo descritto nella Figura 3.1, andando a sostituire la fase relativa alle attività di analisi e produzione del codice precedentemente affidate all'esperto di cybersecurity. L'introduzione di BotScope modifica quindi il ruolo di quest'ultimo, il cui compito non consiste più prevalentemente nello studio manuale del campione, ma si orienta maggiormente verso la revisione dei risultati prodotti dal sistema. L'esperto provvede quindi alla correzione del Capture Module generato da BotScope, potendo eventualmente anche modificarlo o estenderlo in funzione delle proprie valutazioni. Il modulo così revisionato, denominato Reviewed Capture Module, può dunque essere utilizzato dall'esperto per le medesime finalità previste nel processo originale.

3.2.1 Requisiti del sistema

Dopo aver definito gli obiettivi del sistema e le sue caratteristiche principali, la progettazione di BotScope non è soggetta a vincoli specifici in termini di approcci architetturali e componenti da utilizzare. Tale libertà progettuale dovrà essere esercitata nel rispetto delle metodologie e delle pratiche del *software engineering*, tenendo inoltre conto dei requisiti definiti in questa fase e di quelli che emergeranno e verranno progressivamente formalizzati nel corso dello sviluppo del sistema. I quattro requisiti definiti inizialmente costituiscono i principi fondanti di BotScope, ponendone le basi architetturali, e possono essere distinti tra requisiti non funzionali e requisiti di sicurezza.

Requisiti non funzionali:

- **Indipendenza dalla famiglia di botnet**: il sistema dovrà essere progettato in modo da poter essere applicato a campioni appartenenti a differenti famiglie di botnet, senza introdurre dipendenze specifiche da una particolare tipologia o dalle caratteristiche peculiari di una singola famiglia.

La soluzione proposta dovrà essere in grado di gestire differenti modalità di interazione tra il malware e l'infrastruttura C&C, indipendentemente dal protocollo di comunicazione, dalle tecnologie di trasporto o dai formati dei messaggi utilizzati.

- **Autonomia del processo di analisi**: il sistema dovrà operare in maniera completamente autonoma, senza richiedere interventi da parte dell'utente durante la sua esecuzione e senza disporre di alcuna conoscenza preliminare sul campione analizzato. Un utente del sistema dovrà fornire il Binary File in ingresso e si limiterà a ricevere il risultato prodotto dal sistema al termine dell'analisi.

  L'utente potrà intervenire solo a processo concluso, effettuando una revisione del lavoro svolto oppure utilizzando e modificando il Capture Module. A tal fine, è fondamentale garantire un adeguato livello di trasparenza sulle operazioni eseguite dal sistema durante l'analisi, così da facilitare le attività successive svolte dall'utente.

Requisiti di sicurezza:

- **Natura difensiva del modulo generato**: il Capture Module dovrà avere esclusivamente finalità difensive e dovrà limitarsi a catturare passivamente le comunicazioni C&C. Le funzionalità implementate dovranno essere circoscritte alla gestione della connessione e alle fasi di ricezione, decodifica, classificazione e *parsing* dei messaggi ricevuti. Non dovranno essere inclusi comportamenti offensivi, né meccanismi volti a eseguire attacchi o altre azioni malevole. Le uniche comunicazioni attive consentite saranno quelle strettamente necessarie all'instaurazione e al mantenimento della sessione con l'infrastruttura C&C.
- **Sicurezza del processo di analisi**: il processo di analisi del Binary File dovrà essere progettato in modo tale da garantire la protezione dell'ambiente di esecuzione. L'elaborazione dei campioni malevoli non dovrà in alcun modo esporre il sistema di analisi, né eventuali sistemi esterni coinvolti, a rischi di compromissione o alterazione del loro funzionamento.

3.2.2 Proprietà del Capture Module

Al fine di favorire una semplice integrazione del sistema all'interno di *workflow* esterni già esistenti e pipeline di automazione più ampie, è necessario che ogni Capture Module sia conforme a un insieme di proprietà prestabilite. Tali vincoli sono stati pertanto identificati e definiti durante l'analisi dei requisiti, anticipandone la formalizzazione rispetto alle successive fasi di progettazione e implementazione.

Si è deciso quindi di fissare un linguaggio di programmazione per i Capture Module, al fine di garantire l'uniformità dei moduli, scegliendo a questo proposito Python, in virtù di due motivi principali. Il primo è l'ampio utilizzo del linguaggio nell'ambito della cybersecurity e della *Network Forensics* [27], grazie alla disponibilità di un vasto ecosistema di librerie e strumenti dedicati all'analisi e all'elaborazione dei dati di rete. Il secondo riguarda la disponibilità di costrutti avanzati, come i generatori [28], che consentono di elaborare progressivamente i messaggi C&C secondo un modello *lazy*, permettendo così di gestirne il flusso in modo efficiente anche durante la cattura del traffico di rete in tempo reale.

Ciascun Capture Module dovrà inoltre essere *self-contained*, ovvero completamente autoconsistente, includendo al suo interno tutte le definizioni, le classi e le dipendenze necessarie a implementare in

```python
from abc import ABC,
    abstractmethod
class BotnetInterface(ABC):
  @abstractmethod
  def connect(self):
    pass
  @abstractmethod
  def sniff(self):
    pass
```

Figura 3.3: Definizione iniziale dell'interfaccia astratta Botnet Interface.

```python
b = Botnet()

while True:
  try:
    b.connect()
    for message in b.sniff():
      print(message)
  except:
    pass
```

Figura 3.4: Esempio di uso di un generico Capture Module, con classe principale Botnet che implementa Botnet Interface.

autonomia le funzionalità previste dal modulo, tra cui l'infiltrazione, la cattura del traffico, l'elaborazione e il parsing dei comandi. Infine, per favorire l'integrazione dei Capture Module all'interno di sistemi esterni e consentire la gestione scalabile di più moduli, sono stati definiti un'interfaccia standardizzata per il modulo e uno schema comune per i messaggi.

L'interfaccia dei moduli, Botnet Interface, definisce il contratto comune che ogni Capture Module dovrà implementare. La standardizzazione dell'interfaccia consente a qualsiasi componente che conosca tale contratto di caricare e utilizzare direttamente il modulo, senza doverne conoscere la struttura e i dettagli interni. La Botnet Interface stabilisce in particolare la presenza di due metodi fondamentali, `connect` e `sniff`.

Il metodo `connect` ha lo scopo di inizializzare o, eventualmente, di ripristinare la connessione con l'infrastruttura C&C. Il metodo è progettato per essere eseguito all'avvio del Capture Module, prima dell'inizio dell'attività di cattura, e successivamente ogni volta che il metodo `sniff` terminerà la propria esecuzione a causa della chiusura della connessione o di un errore non recuperabile. Il metodo `sniff` ha lo scopo di mantenere attiva la connessione con l'infrastruttura C&C ed eseguire la cattura passiva dei messaggi. Il metodo è progettato per rimanere in esecuzione indefinitamente e dovrà essere implementato come un generatore Python, restituendo mediante l'istruzione `yield` gli oggetti corrispondenti ai messaggi catturati, dopo averli ricevuti e decodificati.

La definizione completa della Botnet Interface è riportata in Figura 3.3, mentre la Figura 3.4 mostra un possibile utilizzo pratico di un Capture Module, la cui classe principale, Botnet, implementa la Botnet Interface definita.

Lo schema dei messaggi, Message Schema, definisce il formato comune a cui dovranno essere conformi gli oggetti restituiti dal metodo `sniff` del Capture Module. Considerata la natura eterogenea delle botnet analizzate, caratterizzate da protocolli di comunicazione e formati dei messaggi differenti, è essenziale avere una rappresentazione uniforme dei messaggi che consenta ai componenti successivi di elaborarli indipendentemente dalla botnet di provenienza. A differenza della Botnet Interface, per la quale sono già stati specificati i metodi e il comportamento atteso, in questa fase non è necessario imporre vincoli sulla struttura interna del Message Schema. L'unico requisito è l'esistenza di uno schema fisso e condiviso, che potrà essere definito direttamente durante l'implementazione del sistema.

4

BotScope: progettazione

4.1 Architettura del sistema

Per descrivere l'architettura proposta per BotScope e fornire una visione complessiva della sua organizzazione, si è scelto di ricorrere a un diagramma dei componenti. Nella notazione UML, il diagramma dei componenti [29] consente di rappresentare le principali parti di un sistema software e le relazioni tra esse, mettendo in evidenza le modalità in cui collaborano alla realizzazione delle funzionalità complessive.

Il diagramma dei componenti creato, ispirato alla corrispondente notazione UML, è riportato in Figura 4.1. Il diagramma permette di mettere in risalto le aree funzionali in cui è organizzato BotScope, che è progettato come una *pipeline* di analisi automatizzata, suddivisa in moduli. All'interno dell'architettura si distinguono, da un lato, la logica decisionale e gli agenti basati su LLM e, dall'altro, gli ambienti di analisi esterni con cui questi interagiscono attraverso il Model Context Protocol. A questi componenti si affiancano inoltre gli artefatti di input e di output, che comprendono i contenuti forniti alle diverse fasi della pipeline e i risultati prodotti nel corso dell'elaborazione, tra cui rientrano Binary File, Knowledge Base e Capture Module.

4.2 Componenti del sistema

In questa sezione vengono descritti nel dettaglio i singoli componenti che costituiscono l'architettura proposta. Per ciascuno di essi vengono individuati i principali requisiti, le responsabilità e le funzionalità messe a disposizione degli altri componenti del sistema. La descrizione segue l'organizzazione mostrata nel diagramma di Figura 4.1, in ordine di appartenenza alle diverse aree funzionali individuate.

4.2.1 Artefatti

Il Binary File, la Knowledge Base e il Capture Module rappresentano rispettivamente gli input e l'output principali del sistema, mentre la Botnet Interface e il Message Schema sono gli artefatti che formalizzano l'interfaccia e lo schema dei messaggi a cui il Capture Module deve attenersi. Tali elementi sono già stati introdotti e descritti diffusamente nel Capitolo 3; per questo motivo, non è necessario approfondire ulteriormente la loro definizione nella fase di progettazione.

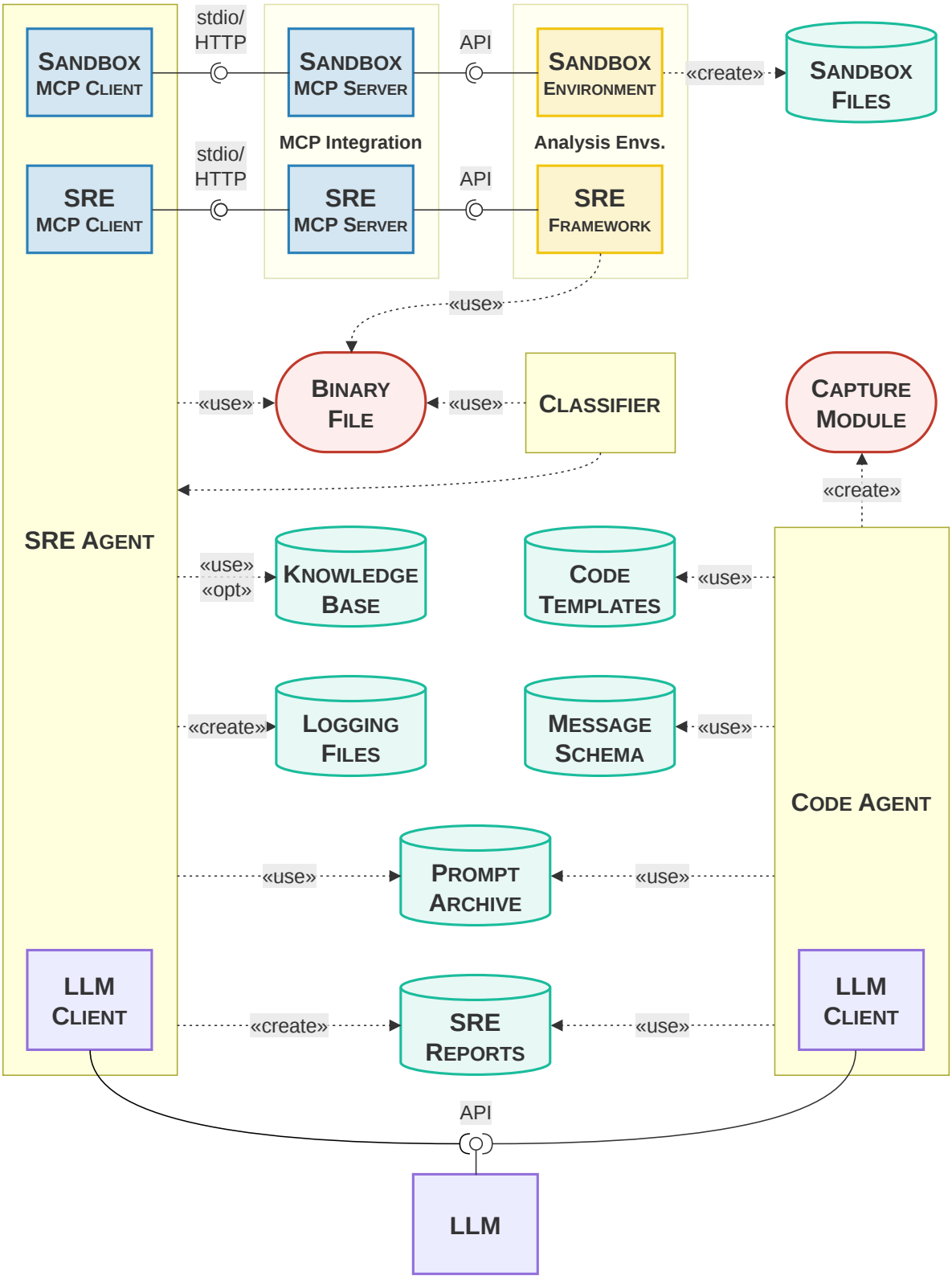


Figura 4.1: Diagramma dei componenti dell'architettura proposta per BotScope.

I Sandbox Files sono i file prodotti dal SRE Agent durante le attività svolte nel Sandbox Environment. Dal momento che sono generati da un LLM che interagisce con un malware, devono essere considerati *untrusted* e, pertanto, non possono essere eseguiti automaticamente nel resto della pipeline. L'unica operazione consentita è la lettura dei file in forma testuale, mentre qualsiasi altra operazione su di essi può essere effettuata esclusivamente dall'utente.

I Logging Files permettono di tracciare in dettaglio le azioni svolte dal LLM all'interno del SRE Agent, dando la possibilità all'utente di svolgere una revisione delle operazioni eseguite durante l'analisi e supportando le attività di debugging. Possono essere costituiti da più tipologie di file, che riportano informazioni differenti e presentano livelli di strutturazione variabili.

I Code Templates rappresentano l'insieme dei template di codice che il Code Agent può utilizzare come riferimento per la creazione del Capture Module, secondo un approccio di few-shot prompting. Tra questi deve essere presente almeno un file principale contenente una classe che implementi la Botnet Interface, eventualmente affiancato da ulteriori file per modularizzare il codice e separare le responsabilità. Analogamente alla Knowledge Base, anche i Code Templates potrebbero essere declinati in versioni specifiche per ciascuna famiglia di botnet. L'introduzione dei Code Templates mira a standardizzare e strutturare, almeno in parte, l'output del sistema, limitando la variabilità del codice prodotto dal LLM anche a fronte di campioni simili, pur mantenendo una certa libertà implementativa. Inoltre, essi contribuiscono a ridurre il rischio che il codice generato contenga funzionalità malevole, fornendo al LLM un esempio privo di tali funzionalità.

Il Prompt Archive raccoglie le istruzioni, system prompt e user prompt, che vengono fornite agli agenti AI di SRE Agent e Code Agent. La funzione dei *system prompt* [15] è quella di definire il ruolo dell'agente, il comportamento da assumere, il tono da utilizzare, i vincoli e le capacità previste per l'applicazione specifica. Gli *user prompt* [15] descrivono in dettaglio le attività che l'utente richiede al LLM di eseguire nel ruolo definito, insieme all'indicazione dell'output atteso e a eventuali informazioni aggiuntive. A differenza dei prompt, che tendono a rimanere invariati nel tempo, come anticipato la Knowledge Base è pensata per avere una natura maggiormente dinamica, potendo essere aggiornata o modificata in funzione delle esigenze del reverse engineer o dell'evoluzione delle conoscenze disponibili.

I SRE Reports rappresentano l'artefatto chiave dell'intera pipeline, poiché si collocano tra SRE Agent e Code Agent. I report devono quindi sintetizzare con un elevato livello di dettaglio l'attività di SRE svolta dal SRE Agent e, allo stesso tempo, fungere da *single source of truth* (SSOT) per la successiva produzione del Capture Module da parte del Code Agent. Inoltre, i SRE Reports devono fornire una documentazione verificabile dall'utente, con evidenze e riferimenti a supporto delle conclusioni raggiunte, permettendogli di validare autonomamente i risultati ottenuti e correggere eventuali errori nel Capture Module. A questo scopo, è importante che ciascun documento sia organizzato in modo chiaro, eventualmente adottando un formato strutturato, per facilitarne la lettura da parte dell'utente e la comprensione da parte del LLM.

4.2.2 Componenti decisionali

Il LLM funge da base per l'implementazione delle funzionalità previste dal SRE Agent e dal Code Agent, che interagiscono con il modello attraverso il LLM Client. Il LLM assume dunque il ruolo di motore cognitivo, fornendo le capacità necessarie alla comprensione, al ragionamento, alla pianifica-

zione e, ove previsto, all'utilizzo dei tool. Dal punto di vista tecnico, il LLM utilizzato deve garantire prestazioni adeguate alle attività previste, con buone capacità di reasoning, il supporto nativo a tool call strutturate e una finestra di contesto sufficientemente ampia.

Il Classifier agisce come *gatekeeper* della pipeline di analisi e ha il compito di esaminare preliminarmente il Binary File ricevuto, decidendo se instradarlo verso il SRE Agent oppure interrompere prematuramente il flusso di analisi, per evitare l'impiego di risorse nello studio di campioni che non lo richiedono o che non risultano interessanti. Inoltre, pur non essendo necessario per il funzionamento di base, questo modulo può essere esteso per estrarre ulteriori informazioni che possono essere utilizzate per fornire al SRE Agent un contesto più specifico, sfruttando anche la Knowledge Base.

Il SRE Agent si occupa di orchestrare l'attività di SRE sul Binary File, in accordo con le istruzioni recuperate dal Prompt Archive e, quando disponibili, con le conoscenze aggiuntive contenute nella Knowledge Base. A tal fine pianifica la strategia di analisi, la esegue attraverso i tool messi a disposizione dal SRE Framework e dalla Sandbox Environment, e infine riassume le informazioni raccolte all'interno dei SRE Reports. Il modulo è inoltre responsabile della gestione del collegamento tra il LLM Client e gli ambienti di analisi mediante il MCP, inoltrando le tool call generate dal LLM ai rispettivi client MCP e restituendo i risultati delle operazioni eseguite. Infine, il SRE Agent deve implementare le funzionalità di *logging* richieste e deve adattare il proprio comportamento in funzione del fatto che il campione analizzato sia stripped o not-stripped.

Il Code Agent coordina le attività di generazione del Capture Module, sulla base delle istruzioni definite nei relativi prompt. Come input riceve i SRE Reports, il Message Schema e i Code Templates, che includono anche la Botnet Interface. A partire dall'insieme di questi elementi, il Code Agent deve estrarre e combinare le informazioni necessarie per costruire il Capture Module, in conformità ai requisiti strutturali e di sicurezza previsti.

4.2.3 Integrazione MCP

Il SRE MCP Server e il Sandbox MCP Server sono i server MCP dedicati ai due ambienti di analisi del sistema ed espongono i tools che potranno essere utilizzati dal SRE Agent. Ogni tool [18] deve avere un `name` univoco che permette di identificare chiaramente il suo scopo, un `title` opzionale, una *description* delle modalità di utilizzo e infine un `inputSchema`, un JSON Schema che specifica, per ciascun parametro di input, il tipo, la relativa descrizione e se è obbligatorio. I server devono inoltre predisporre il meccanismo di trasporto attraverso cui accettare le connessioni dei client, utilizzando il protocollo *stdio* nel caso di client locali oppure *Streamable HTTP* nel caso di client remoti. A fronte delle richieste di esecuzione dei tool ricevute dai client, i server devono effettuare il *dispatch* verso la relativa implementazione e, dopo l'esecuzione, devono restituire il risultato in un formato prestabilito.

Il SRE MCP Client e il Sandbox MCP Client sono i client MCP associati ai rispettivi server MCP. Ciascun client, utilizzando gli *endpoint* `tools/list` e `tools/call` del server, deve esporre le funzionalità che permettano a un agente AI di scoprire i tool disponibili e di invocare un tool, specificandone il nome e gli eventuali argomenti.

4.2.4 Ambienti di analisi

Il SRE Framework deve disporre di funzionalità proprie dei sistemi di SRE, tra cui il disassemblaggio, la decompilazione e, qualora disponibile, il debugging, così da fornire al SRE Agent le capacità necessarie per svolgere le attività di analisi statica del codice e, eventualmente, di analisi dinamica avanzata. Tra gli strumenti per l'analisi statica da esporre rientrano, a titolo esemplificativo, l'analisi e la gestione delle funzioni e delle relative *cross-reference*, degli indirizzi e della memoria, del *type system*, nonché la gestione e la rinomina dei simboli.

Il Sandbox Environment ha lo scopo di mettere a disposizione funzionalità dinamiche aggiuntive per l'analisi, potenzialmente diverse da quelle offerte dalle classiche attività di esecuzione o debugging, così da poter essere affiancato e combinato al SRE Framework. All'interno dell'architettura proposta, il Sandbox Environment rappresenta un ambiente generale nel quale, durante il processo di SRE, è possibile costruire ed eseguire strumenti personalizzati per trasformare, analizzare e interpretare le informazioni estratte durante l'analisi, oppure per automatizzare determinate attività. Uno degli aspetti più critici dell'implementazione di questo componente consiste nel garantire la sicurezza dell'ambiente. Il Sandbox Environment deve garantire l'isolamento dall'host, operare con privilegi minimi e prevedere limiti alle risorse computazionali disponibili. L'accesso alla rete dovrebbe essere negato di default e consentito solamente quando l'implementazione prevede adeguate condizioni di sicurezza.

Per entrambi gli ambienti di analisi, ai fini dell'integrazione con il LLM all'interno del SRE Agent, è fondamentale che le funzionalità offerte siano esposte attraverso un'interfaccia che ne consenta l'accesso tramite il MCP.

4.3 Flusso di esecuzione

Dopo aver descritto la struttura complessiva dell'architettura di BotScope e le responsabilità dei singoli componenti, questa sezione illustra il flusso di esecuzione completo, ripercorrendo le attività che si susseguono a partire dal Binary File di input e mostrando il percorso seguito attraverso i diversi componenti del sistema, fino ad arrivare alla produzione del Capture Module.

A questo scopo viene utilizzato un diagramma delle attività, ispirato alla corrispondente notazione UML [29], che permette di descrivere lo svolgimento di un processo attraverso il flusso delle attività, le decisioni, i percorsi alternativi, le attività eseguite in parallelo e, ove necessario, il flusso degli oggetti.

Il diagramma delle attività realizzato è riportato in Figura 4.2. A differenza del diagramma dei componenti presentato nella sezione precedente, questo diagramma ha una finalità esclusivamente descrittiva: rappresenta infatti un flusso semplificato utilizzato solamente per illustrare il comportamento del sistema e non costituisce una specifica vincolante per l'implementazione.

Il flusso di esecuzione ha inizio quando l'utente fornisce in ingresso al sistema il Binary File, oltre all'eventuale Knowledge Base. Quindi, il Classifier ne esegue un'analisi preliminare, finalizzata almeno a determinare se il campione sia malevolo oppure no. Quando è possibile stabilire con certezza che il campione non è malevolo oppure che, pur essendo malevolo, non è di interesse per il sistema, allora la pipeline di analisi viene interrotta. Negli altri casi il campione viene trasferito dal Classifier al SRE Agent.

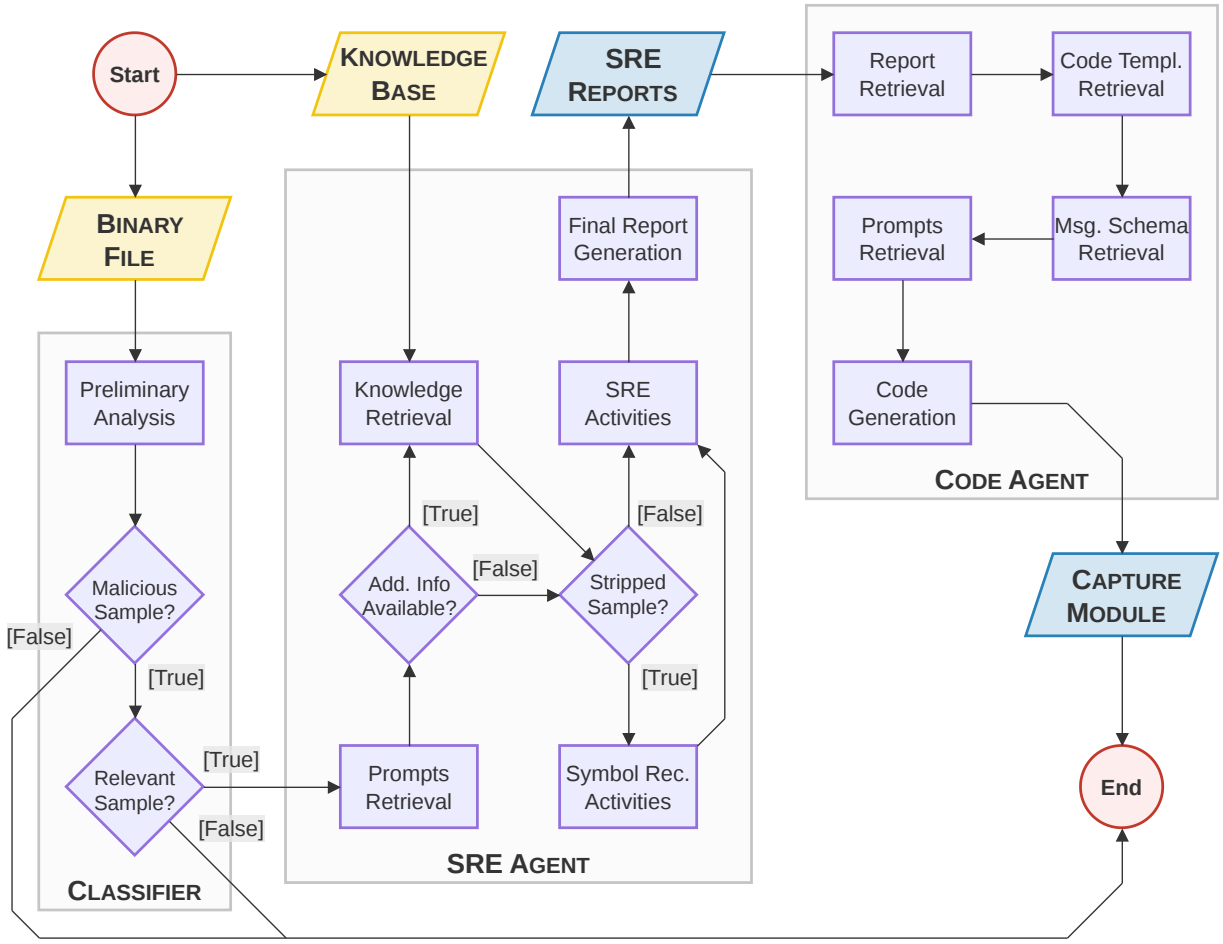


Figura 4.2: Diagramma delle attività e del flusso operativo del sistema.

Come prima cosa, il modulo carica le istruzioni dal PROMPT ARCHIVE e, quando possibile, utilizza le informazioni aggiuntive rese disponibili dal CLASSIFIER per selezionare dalla KNOWLEDGE BASE il contesto supplementare pertinente al campione. A questo punto, il SRE AGENT, basandosi sulla presenza dei simboli, determina se il campione è stripped o not-stripped. Nel primo caso, l'agente potrebbe intraprendere immediatamente delle azioni finalizzate al recupero dei simboli, come illustrato della Figura 4.2, oppure potrà adattare il proprio comportamento successivamente.

La fase centrale è dunque costituita dalle attività di SRE svolte dal SRE AGENT attraverso il LLM, secondo un ciclo iterativo di ragionamento, azione e osservazione. Per fare ciò, il modello utilizza i tool messi a disposizione dagli ambienti SRE FRAMEWORK e SANDBOX ENVIRONMENT, interagendoci attraverso il MCP. Nel caso del SANDBOX ENVIRONMENT, il LLM potrà anche produrre e salvare dei SANDBOX FILES. Quando l'agente dispone di informazioni sufficienti, conclude l'analisi, producendo i SRE REPORTS, mentre le attività svolte vengono registrate nei LOGGING FILES. Per semplicità, nella Figura 4.2 l'insieme di queste attività e interazioni è rappresentato da un *activity node* singolo, *SRE Activities*, seguito dal nodo *Final Report Generation*.

Infine, i SRE REPORTS vengono passati al CODE AGENT, che carica i CODE TEMPLATES, lo schema dei messaggi e le istruzioni operative dal PROMPT ARCHIVE. Il LLM dell'agente utilizza questa serie di informazioni per generare gli elementi necessari a costruire il CAPTURE MODULE di cattura delle comunicazioni, che corrisponde all'output finale del sistema.

4.4 Interazioni di tool calling

Considerata la rilevanza del meccanismo di *tool calling* nell'architettura proposta e il suo frequente impiego durante l'attività di SRE, risulta utile fornire una rappresentazione più dettagliata delle interazioni che avvengono tra il SRE Agent e gli ambienti di analisi mediante il MCP. I diagrammi dei componenti e delle attività precedenti non consentono di cogliere pienamente la natura di tali interazioni, per cui si ricorre a un diagramma di sequenza, ispirato alla corrispondente notazione UML [29], che permette di descrivere come diversi componenti interagiscono tra loro nel corso del tempo, rappresentando lo scambio di messaggi e l'ordine temporale degli eventi.

Il diagramma di sequenza realizzato è riportato in Figura 4.3. Esso rappresenta una sequenza esemplificativa, corrispondente solamente a un sottoinsieme delle interazioni che si verificano durante le attività di analisi del SRE Agent.

Il diagramma illustra, nella fase iniziale della sequenza, le principali operazioni preliminari necessarie alla creazione e alla configurazione degli oggetti che prenderanno parte alle successive interazioni. In particolare, tale fase comprende l'istanziazione, da parte del SRE Agent, dei client MCP, i quali instaurano la connessione con i rispettivi server MCP.

La seconda parte riguarda la scoperta dei tool disponibili per il Sandbox Environment da parte dell'agente. Il SRE Agent richiede al Sandbox MCP Client la lista dei tool tramite la funzione `list_tools()` e il client invoca l'endpoint `tools/list` del Sandbox MCP Server, che restituisce la lista dei tool al client. Tale lista viene successivamente inoltrata al SRE Agent, che può quindi esporre al LLM, per ciascuna richiesta, i tool disponibili, eventualmente limitandoli a un sottoinsieme.

La terza parte riguarda invece l'effettiva invocazione di un generico tool del SRE Framework. Il SRE Agent, dopo aver inviato una richiesta al LLM tramite il LLM Client, riceve una risposta strutturata contenente una o più tool call, le estrae e determina a quale ambiente di analisi appartengano. Nel caso considerato, si suppone sia presente una chiamata che riguarda il SRE Framework, per cui l'agente richiede al SRE MCP Client l'esecuzione del tool tramite la funzione `call_tool(name, arguments)`. Il client invoca quindi l'endpoint `tools/call` del SRE MCP Server, specificando il `name` del tool e gli eventuali `arguments` come oggetto JSON. Ricevuta la richiesta, il SRE MCP Server provvede a gestirla, avviando l'esecuzione dell'operazione corrispondente sul SRE Framework. Al termine dell'operazione, il SRE Framework restituisce il relativo risultato al server, che lo inoltra al SRE MCP Client. Quest'ultimo lo ritorna infine al SRE Agent, che può inserire il risultato del tool nel contesto del LLM, così da proseguire con l'attività di analisi.

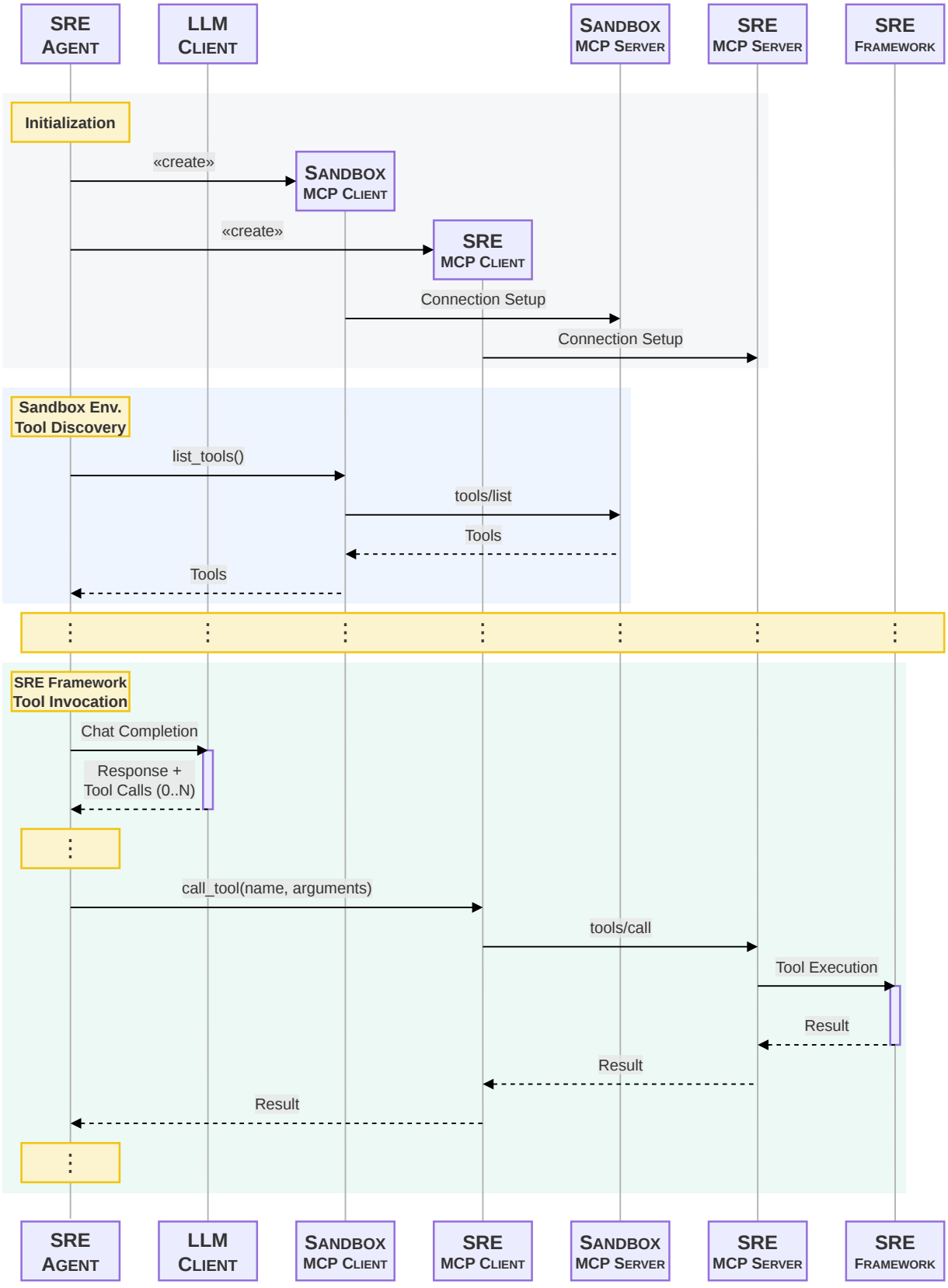


Figura 4.3: Diagramma di sequenza delle interazioni che caratterizzano il processo di tool calling.

5

BotScope: implementazione

5.1 Ambiente di sviluppo

L'implementazione del sistema è stata realizzata in Python 3.13 [30], mentre per la gestione dell'ambiente di sviluppo e delle dipendenze del progetto è stato utilizzato Conda [31].

La scelta del linguaggio di implementazione è ricaduta su Python prevalentemente per l'ampio ecosistema di librerie esistenti dedicate all'analisi del malware, all'integrazione con il LLM, con il MCP e con gli ambienti di analisi. La disponibilità di strumenti già consolidati ha consentito di concentrare lo sviluppo sulle funzionalità proprie del sistema, riducendo la necessità di implementare direttamente quelle di supporto.

5.2 Moduli principali

In questa sezione viene presentata l'implementazione dei tre moduli fondamentali del sistema: Classifier, SRE Agent e Code Agent. Le componenti specifiche impiegate da ciascun modulo sono approfondite nelle sezioni successive, a cui si rimanda di volta in volta nel corso della trattazione.

5.2.1 Classifier

Inizialmente, il modulo calcola gli hash del Binary File, incluso il digest SHA-256. L'hashing è uno strumento importante nell'analisi del malware [8], perché consente di ottenere una *fingerprint* del file, utilizzata dai reverse engineer per identificarlo e per condividere informazioni relative ad esso. Il digest ottenuto può, ad esempio, essere utilizzato per verificare se il Binary File sia già presente in database di malware o in servizi online di *threat intelligence*.

Virus Total

A questo scopo viene utilizzato il servizio VirusTotal [32], che consente di sottoporre un file all'analisi di numerosi *antivirus engine*, fornendo un report che elenca i rilevamenti di attività malevola e le informazioni tecniche sul campione, oltre a suggerire un nome e una categoria di *malware threat*.

Nell'implementazione è stata utilizzata l'API pubblica di VirusTotal, disponibile gratuitamente entro specifici limiti di utilizzo e destinata a un impiego non commerciale e non professionale. Per ottimizzare

il processo, viene inizialmente verificata la presenza del campione nel database tramite il suo hash; in caso positivo, si recupera direttamente il report associato, altrimenti, il file viene caricato e analizzato.

Le informazioni ottenute vengono utilizzate per determinare la natura del campione mediante un'euristica basata sul numero assoluto di rilevamenti e sulla percentuale di antivirus engine che lo classificano come malevolo. Quando il campione non supera la soglia prefissata, viene scartato e l'analisi termina.

YARA

Per l'individuazione della famiglia della botnet si ricorre all'applicazione delle *YARA rules*. YARA [8, 33] è uno strumento open source utilizzato per identificare e classificare campioni di malware attraverso regole che definiscono specifiche caratteristiche ricercabili all'interno dei file analizzati. Tali regole combinano pattern, come stringhe e sequenze di byte, con condizioni logiche che determinano quando una regola viene soddisfatta dal campione esaminato.

Per questa implementazione viene utilizzato YARA-X [33], una nuova versione del linguaggio di YARA, progettata per mantenere la compatibilità con le regole esistenti, introducendo correzioni e nuove funzionalità. Il Classifier carica e compila l'insieme delle YARA rules definite, con regole specifiche per ciascuna famiglia di botnet supportata. Successivamente, il campione viene confrontato con tutte le regole, alla ricerca di una corrispondenza e, nel caso venga trovata, è possibile fornire al SRE Agent l'informazione riguardante la famiglia a cui appartiene il campione.

5.2.2 SRE Agent

La prima operazione eseguita dal modulo consiste nell'importazione del Binary File all'interno del progetto attivo nel SRE Framework. Contestualmente al caricamento, il framework esegue un'analisi automatica del file, al termine della quale è necessario determinare se il binario sia stripped o not-stripped. Una funzione dedicata, in base al *magic number* dell'eseguibile, determina innanzitutto se il file sia in formato *Executable and Linkable Format* (ELF) oppure *Portable Executable* (PE). Successivamente, la funzione stabilisce se il campione sia stripped o not-stripped sulla base della presenza nel binario della tabella dei simboli e delle informazioni di debug.

Qualora venga rilevato lo stripping, si prosegue con la macro-attività denominata Renaming Task, finalizzata ad assegnare dei nomi significativi alle funzioni del binario. Questa viene eseguita attraverso due *agentic loop* consecutivi, nei quali viene richiesto di ricostruire ricorsivamente la logica del programma, assegnando alle funzioni nuovi nomi coerenti con il comportamento osservato. A questo scopo sono disponibili due modalità di esecuzione, Start Mode e Continue Mode. La prima consente di avviare l'analisi da zero, mentre la seconda permette di operare su un binario nel quale sono già presenti risultati di rinomina, con l'obiettivo di estenderli e, al contempo, verificarne la correttezza.

Segue quindi la macro-attività Reversing Task, avviata direttamente nel caso di campioni not-stripped. Il suo obiettivo è quello di eseguire il SRE del binario, per estrarre le informazioni rilevanti e descrivere il funzionamento del protocollo C&C nei SRE Reports. Anche la Reversing Task è articolata in due *agentic loop* che, seppur siano svolti in sequenza, sono indipendenti tra loro e, in linea teorica, parallelizzabili. Il primo, soprannominato Transport Analysis, si concentra sull'analisi del *Transport Layer* del protocollo di C&C, mentre il secondo, soprannominato Communication Analysis, analizza il *Communication Layer*. Sebbene lo svolgimento di più analisi possa comportare una parziale

sovrapposizione delle informazioni raccolte, tale approccio offre al LLM due occasioni per analizzare il campione a partire da prospettive diverse, concentrandosi comunque su aspetti differenti.

Sia per la Renaming Task che per la Reversing Task all'avvio di ogni ciclo agentico vengono caricate le rispettive istruzioni dal Prompt Archive e la porzione della Knowledge Base rilevante per l'analisi, sulla base delle informazioni ricevute dal Classifier. Il funzionamento generale dell'agentic loop alla base delle due marco-attività del SRE Agent è descritto nella Sezione 5.6, mentre le istruzioni specifiche fornite vengono approfondite nella Sezione 5.9.

Selezione dei tool di analisi

Durante la costruzione del SRE Agent, una scelta rilevante riguarda la selezione dei tool degli ambienti di analisi da mettere a disposizione del LLM. Per la Renaming Task sono sufficienti i tool del SRE Framework, mentre per la Reversing Task si aggiungono anche quelli del Sandbox Environment.

Per quanto riguarda il Sandbox Environment, trattandosi di un ambiente realizzato specificamente per il sistema, tutti i tool presenti sono esposti all'agente. Il loro scopo e le modalità di implementazione sono trattate nella Sezione 5.4. Una selezione ad hoc è invece necessaria per i tool del SRE Framework, che mette a disposizione un insieme più ampio di funzionalità, come descritto nella Sezione 5.3. Nell'implementazione proposta è stato quindi fissato un sottoinsieme di circa cinquanta tool, circoscritto alle operazioni necessarie per l'analisi statica del codice. Tale scelta non vincola in alcun modo l'architettura a questa configurazione, poiché la lista può essere successivamente estesa o modificata in funzione delle esigenze e del feedback raccolto.

La scelta di limitare i tool del SRE Framework è motivata da diverse ragioni. Innanzitutto, consente di evitare di esporre direttamente l'agente a funzionalità di analisi dinamica che potrebbero comportare l'esecuzione di codice malevolo, mantenendo al contempo un maggiore controllo sulle azioni che può intraprendere. Inoltre, si intende favorire la separazione e la complementarità tra i due ambienti, dedicando il SRE Framework all'estrazione di informazioni dal binario e utilizzando, quando necessario, gli strumenti della Sandbox Environment per la loro interpretazione e verifica.

Infine, la scelta adottata è coerente con il principio *Less-is-More* nell'utilizzo del tool calling da parte dei LLM. Come discusso da Paramanayakam et al. [34], limitare i tool disponibili a quelli pertinenti all'attività in corso può migliorare l'accuratezza delle chiamate, ridurre i tempi di esecuzione e contenere il consumo di risorse. Al contrario, la presenza di molti tool aumenta la complessità del contesto fornito al modello e il numero di alternative tra cui scegliere, con effetti rilevanti nel caso di LLM di dimensioni ridotte. Un approccio alternativo, proposto nello stesso lavoro, consiste nella selezione dinamica dei tool, caricando di volta in volta solo quelli più rilevanti. Tale soluzione introduce tuttavia un ulteriore livello di complessità e, per questo motivo, non è stata adottata nell'implementazione iniziale del sistema.

Knowledge Base

Per le attività di analisi del SRE Agent, come individuato nelle fasi di identificazione del problema e progettazione, è previsto il coinvolgimento della Knowledge Base. Nell'implementazione proposta, la Knowledge Base è creata e gestita da un esperto di cybersecurity ed è suddivisa per famiglie di botnet. Per ciascuna famiglia che si intende supportare è prevista una guida specifica, liberamente strutturata, da affiancare alle istruzioni di base delle attività. Ad esempio, ogni guida può essere articolata in sezioni

dedicate ai diversi aspetti caratteristici della famiglia, come l'identificazione iniziale dei campioni, la topologia dei processi, il flusso di esecuzione, le comunicazioni C&C, i metodi di attacco e altri elementi peculiari.

L'obiettivo di una Knowledge Base organizzata in questo modo è fornire al SRE Agent la conoscenza specifica per estendere l'approccio di *zero-shot prompting*, assunto come base per le attività Renaming Task e Reversing Task e utilizzato in assenza della Knowledge Base, verso un approccio di *few-shot prompting*. In questo modo, le informazioni aggiuntive fornite all'agente supportano la sua analisi, potenzialmente migliorandone l'accuratezza. Dal punto di vista del sistema, la Knowledge Base è una risorsa statica, resa opzionalmente disponibile come conoscenza aggiuntiva e non modificabile dall'agente. Il suo contenuto può però essere aggiornato esternamente dall'esperto di cybersecurity, permettendone così l'arricchimento sulla base delle informazioni emerse dall'utilizzo del sistema.

5.2.3 Code Agent

Al termine dell'analisi condotta dal SRE Agent, il principale output prodotto è costituito dai SRE Reports. Teoricamente, sulla base delle informazioni in essi contenute, sarebbe possibile determinare se sussistano condizioni sufficienti per procedere alla generazione del Capture Module o se convenga interrompere anticipatamente l'analisi, ad esempio in presenza di numerose informazioni mancanti oppure di un campione di scarsa rilevanza. Nonostante ciò, in questa implementazione si è scelto di procedere sempre con la sua generazione, principalmente per il suo costo marginale rispetto a quello complessivo della pipeline, determinato in larga parte dall'esecuzione del SRE Agent.

Come descritto successivamente nella Sezione 5.8, il Capture Module finale è composto da due moduli distinti, Botnet e Message Parser, ai quali sono assegnate responsabilità differenti. La generazione effettiva del codice viene affidata al LLM e viene coordinata dal Code Agent attraverso un'unica conversazione basata sul *multi-turn prompting*, all'interno della quale viene prima generato il modulo Botnet e successivamente il modulo Message Parser, fornendo di volta in volta gli artefatti di input rilevanti. La Sezione 5.7 presenta il Message Schema di riferimento per il parsing dei messaggi; i Code Templates impiegati nella generazione sono invece descritti nella Sezione 5.8, mentre le istruzioni fornite al Code Agent vengono riassunte nella Sezione 5.9.

5.3 SRE Framework

Il framework di SRE adottato è Ghidra [35], sviluppato e mantenuto dal Research Directorate della National Security Agency (NSA) e distribuito come progetto open source secondo la Apache License 2.0. L'ambiente Ghidra, offre una suite ricca di strumenti per l'analisi binaria del codice compilato, tra cui disassembly, decompiling, graphing, debugging e scripting. Il framework supporta inoltre un'ampia varietà di *instruction set* relativi a differenti architetture di processore e numerosi formati eseguibili.

Le attività di analisi possono essere svolte sia in modalità interattiva, attraverso una *Graphical User Interface* (GUI), sia in modalità automatizzata, tramite l'interfaccia *headless*, rendendo Ghidra adatto all'impiego in processi di analisi complessi [11]. Nell'implementazione proposta viene adottata quest'ultima modalità, mentre, come possibile evoluzione, Ghidra potrebbe essere eseguito all'interno di un *container* Docker, mantenendo invariata la modalità di interazione con la pipeline di analisi.

5.3.1 Ghidra MCP Plugin

Per integrare le funzionalità di analisi offerte da Ghidra con il SRE AGENT in conformità con il MCP è stato utilizzato il software open source *ghidra-mcp* [36], disponibile sull'omonimo repository GitHub e distribuito secondo la Apache License 2.0.

Il software fornisce un'estensione per Ghidra e un insieme di componenti di supporto che permettono di rendere accessibili le funzionalità dell'ambiente di analisi attraverso il MCP, esponendo complessivamente più di duecento tool. Si tratta di una soluzione in continua evoluzione che supporta l'integrazione con Ghidra Server, la gestione di più progetti e file, l'esecuzione in modalità GUI, in modalità headless e il deployment mediante Docker.

L'insieme delle funzionalità messe a disposizione è particolarmente ampio e comprende, innanzitutto, le principali operazioni di analisi binaria, relative a funzioni, data flow, strutture dati, stringhe e accesso alla memoria. A queste si aggiungono funzionalità più avanzate, tra cui, nell'ambito dell'analisi dinamica, l'emulazione del P-Code e l'integrazione con il debugger, oltre a strumenti per la gestione dei progetti e degli script di Ghidra.

5.3.2 SRE MCP Server

Il server MCP del software *ghidra-mcp* è implementato mediante il modulo `bridge_mcp_ghidra` [36] e costituisce il livello intermedio tra il SRE MCP CLIENT e l'estensione di Ghidra. I tool sono organizzati in gruppi semantici, così da permettere di gestire in maniera selettiva l'insieme delle funzionalità esposte al LLM. In modalità standard, il server rende subito disponibili tutti i gruppi di tool; in alternativa, il software può essere eseguito in modalità *lazy*, caricando inizialmente solo quelli più importanti. Gli altri gruppi possono essere successivamente individuati, caricati o rimossi dinamicamente dal LLM durante l'esecuzione, secondo le esigenze di analisi.

5.4 Sandbox Environment

Il SANDBOX ENVIRONMENT viene realizzato mediante un ambiente di esecuzione containerizzato, utilizzando *Docker* [37] per la creazione, la configurazione e la gestione del container.

La *containerization* è una tecnica di isolamento a livello di sistema operativo che consente di eseguire un'applicazione e le relative dipendenze all'interno di un ambiente isolato. Rispetto alle macchine virtuali, che richiedono l'esecuzione di un intero sistema operativo guest, i container offrono un approccio più leggero per ottenere ambienti di esecuzione separati.

Nel contesto del SANDBOX ENVIRONMENT, Docker fornisce i meccanismi necessari alla creazione e alla configurazione del container, consentendo di controllarne il filesystem, l'accesso alla rete e le risorse computazionali disponibili. Queste caratteristiche permettono di predisporre l'ambiente in modo coerente con i requisiti di isolamento e sicurezza individuati durante la fase di progettazione.

5.4.1 Configurazione

La configurazione del SANDBOX ENVIRONMENT è definita mediante un *Dockerfile*, che specifica l'immagine di base, le dipendenze e le impostazioni necessarie al funzionamento del container. L'immagine

utilizzata è basata su Python 3.13 e Debian Slim, così da fornire un ambiente leggero e minimale, ma sufficientemente completo da permettere al LLM di svolgere le operazioni richieste.

A questo scopo, durante la fase di costruzione vengono installati gli strumenti di sistema e le librerie Python necessarie alle attività previste nell'ambiente, tra cui quelle dedicate alle funzionalità crittografiche. Viene inoltre configurata una cartella dedicata al Sandbox Environment e viene creato un utente senza privilegi di root, utilizzato per l'esecuzione dei processi all'interno del container.

L'immagine così definita viene utilizzata per creare il container dell'ambiente, introducendo ulteriori misure di isolamento. In particolare, l'accesso alla rete viene completamente disabilitato, limitando l'interazione con risorse esterne e confinando le operazioni alle sole funzionalità e dipendenze disponibili all'interno dell'ambiente. Per consentire lo scambio controllato dei Sandbox Files, una cartella dell'host viene montata all'interno del container con permessi di lettura e scrittura. Infine, vengono imposti limiti alle risorse computazionali disponibili, in termini di memoria e capacità di elaborazione.

5.4.2 Sandbox MCP Server

Il server MCP del Sandbox Environment ha il compito di interagire con l'ambiente ed esporne le funzionalità attraverso tool MCP. Per la sua implementazione è stata utilizzata la libreria Python *FastMCP* [38]. Il server offre quindi un insieme di tool organizzati in due principali categorie:

File System: i tool `list_files`, `read_file` e `write_file` permettono di creare, consultare e modificare i file presenti nel Sandbox Environment. Ogni operazione prevede inoltre la gestione degli errori, così da fornire un'informazione esplicita in caso di esecuzione non riuscita.

Execution: il tool `execute_python_file` consente di eseguire file Python all'interno del container, imponendo un limite temporale all'esecuzione. Il tool `execute_command` permette invece di eseguire comandi della *shell*. Prima dell'esecuzione viene effettuato un controllo di sicurezza sui comandi richiesti per prevenire operazioni potenzialmente pericolose, come tentativi di *privilege escalation*, o aperture di nuove shell. In entrambi i casi, il risultato dell'esecuzione viene restituito in forma strutturata, includendo `exit code`, `stdout` e `stderr`.

5.5 MCP Client

Per i due client MCP, Sandbox MCP Client e SRE MCP Client, è stata definita una classe di base comune, `Base_MCP_Client`, responsabile dell'instaurazione e del mantenimento della connessione con il rispettivo server MCP mediante il metodo di trasporto *stdio*. La classe implementa i due metodi fondamentali per l'interazione con il server, `list_tools` per la consultazione dei tool disponibili e `call_tool` per la loro invocazione. La classe `Base_MCP_Client` viene ulteriormente estesa da `LLM_MCP_Client`, che introduce il livello di integrazione con il LLM. In particolare, i tool esposti dai server MCP vengono convertiti nel formato utilizzato dal meccanismo di *function calling* delle OpenAI API [39], includendo le informazioni relative al nome, alla descrizione e ai parametri richiesti.

Entrambi i client specifici estendono quindi `LLM_MCP_Client`, ereditandone le funzionalità comuni. Poiché i server sono stati sviluppati secondo modalità di interazione uniformi e già predisposte all'uso da parte di LLM, non è stato necessario introdurre funzionalità ulteriori nelle classi derivate.

A completamento di questa struttura, una classe dedicata fornisce un meccanismo per la registrazione dei client MCP e dei relativi tool. Essa consente di assegnare opzionalmente ai nomi dei tool un *namespace*, identificando il client MCP con un prefisso, e di selezionare solo un sottoinsieme dei tool disponibili. I tool non selezionati vengono così esclusi dall'insieme di quelli esposti al LLM.

5.6 LLM

Il LLM utilizzato dall'applicazione viene eseguito tramite un'istanza esterna di *vLLM* [40], un framework open source progettato per l'inferenza e il *serving* di LLM. Il server vLLM gestisce sia l'esecuzione del modello, sia le richieste di inferenza provenienti dal sistema, esponendo un'interfaccia standardizzata compatibile con le OpenAI API [39].

5.6.1 LLM Client

Il LLM Client è il componente utilizzato dai moduli SRE Agent e Code Agent per interagire con il LLM. Durante la sua inizializzazione vengono specificati i client MCP accessibili al LLM; i tool da essi esposti vengono quindi raccolti e memorizzati in una struttura di risoluzione. Questa associazione consente di individuare successivamente, a partire dal nome del tool richiesto dal LLM, il client MCP responsabile della sua gestione, così da poterlo invocare.

Il componente permette inoltre di specificare il livello di ragionamento per il LLM e di configurare i *sampling parameters*. Questi ultimi sono i parametri di campionamento, ovvero le variabili utilizzate per controllare le caratteristiche della generazione del testo: `temperature`, `top_p`, `top_k`, `min_p`, `presence_penalty`, `frequency_penalty` e `repetition_penalty`.

Sono quindi messi a disposizione metodi per effettuare richieste singole al LLM, con o senza tool, mediante *chat completion*, secondo l'interfaccia prevista dalle OpenAI API [39].

Infine, il componente fornisce la struttura portante alla base degli agentic loop del SRE Agent. Tale struttura prevede un ciclo iterativo che gestisce autonomamente lo stato della conversazione, invocando il LLM, eseguendo le tool call e inserendo nel contesto i risultati ottenuti, fino al raggiungimento dell'obiettivo. L'implementazione segue il paradigma ReAct [17], alternando le fasi di ragionamento e di azione e preservando la Chain of Thought sviluppata dal LLM.

All'interno del ciclo di esecuzione non sono stati introdotti meccanismi di compattazione del contesto, così da preservare integralmente la cronologia dell'esecuzione ed evitare la perdita di informazioni. Sebbene questa scelta possa comportare il raggiungimento del limite della finestra di contesto, nei test effettuati con un LLM dotato di una finestra massima di $262k$ token tale eventualità non si è verificata.

Logging

A supporto dell'esecuzione dell'agente, sono state implementate anche le funzionalità di logging richieste dalla progettazione, secondo due modalità complementari. La prima è orientata alla conservazione completa dell'esecuzione e raccoglie l'intera cronologia di conversazione, producendo un output strutturato conforme al formato utilizzato dalle OpenAI API [39]. La seconda modalità è focalizzata alla consultazione dell'utente e presenta una sintesi leggibile dell'esecuzione. In particolare, vengono riportati il

prompt iniziale e, per ogni passo, il reasoning, le tool call e le risposte prodotte dal LLM, escludendo i risultati restituiti dai tool, così da mantenere l'attenzione sulla catena di ragionamento del LLM.

5.7 Message Schema

Come individuato già durante l'analisi dei requisiti nel Capitolo 3, è necessario definire uno schema comune per i messaggi restituiti dal Capture Module. Per questo scopo è stato utilizzato *JSON Schema* [41], un linguaggio dichiarativo progettato per descrivere la struttura e i vincoli dei dati rappresentati in formato JSON, specificando proprietà, tipi, campi obbligatori, formati e altre restrizioni.

JSON Schema risulta particolarmente indicato per questa finalità per due motivi. Il primo è l'immediatezza con cui è possibile verificare, mediante un *validator*, la conformità di un documento JSON, denominato *instance*, rispetto a uno schema, confrontando automaticamente la struttura e i contenuti dell'istanza con i vincoli dello schema. Il secondo riguarda il suo valore come forma di documentazione, poiché fornisce una descrizione esplicita della struttura e dei vincoli dei dati, comprensibile sia da un essere umano sia da un LLM.

5.7.1 Classificazione dei messaggi

Il costrutto *oneOf* di JSON Schema consente di definire, all'interno di uno schema, diverse alternative di validazione, ciascuna associata a un sotto-schema specifico. Questa espressività viene sfruttata per distinguere semanticamente i messaggi C&C catturati in diverse tipologie, definendo per ciascuna campi specifici e facilitandone così l'interpretazione da parte dell'utente.

Per l'implementazione considerata sono state pertanto individuate le seguenti quattro categorie:

ACK: identifica i messaggi di *acknowledgment* a un precedente messaggio.

Attack: identifica i messaggi contenenti istruzioni relative all'esecuzione di un attacco.

Heartbeat: identifica i messaggi utilizzati per mantenere attiva la connessione.

Command: identifica i messaggi di servizio o generici che non rientrano nelle categorie precedenti, oppure i messaggi per i quali non si è in grado di effettuare una classificazione più specifica.

5.7.2 JSON Schema

Lo schema realizzato è conforme allo standard JSON Schema Draft 2020-12 [41]. Al primo livello vengono memorizzati il *timestamp* di ricezione del messaggio e l'indirizzo IP sorgente. Quindi è presente un campo dedicato alla classificazione del messaggio, secondo le tipologie introdotte nella sezione precedente, a cui segue uno schema annidato contenente le informazioni specifiche relative alla tipologia individuata.

Per ciascuna categoria sono quindi stati definiti vincoli e proprietà specifici. I messaggi `ACK` contengono l'informazione associata alla richiesta di acknowledgment, mentre i messaggi `Heartbeat` riportano un numero di sequenza e, opzionalmente, il tempo di connessione del bot. I messaggi `Command` includono invece il payload originale e, quando è possibile determinarla, una stringa che identifica la natura del comando. Infine, tutte le tipologie di messaggio prevedono un campo opzionale per annotazioni o informazioni aggiuntive, espresse come stringa.

La tipologia `Attack` è quella caratterizzata dalla struttura più articolata. Per ciascun messaggio vengono infatti memorizzati il codice identificativo e il nome dell'attacco, il protocollo di rete coinvolto e la durata dell'esecuzione. Lo schema permette inoltre di rappresentare gli argomenti forniti all'attacco attraverso il numero di elementi (`argc`) e i relativi valori (`argv`). Infine, vengono memorizzati il numero dei bersagli coinvolti e il relativo elenco di indirizzi IP di destinazione.

IP Schema e IP Lookup

Lo schema include anche un array dedicato alle informazioni relative agli indirizzi IP coinvolti negli attacchi, che può rimanere vuoto qualora non siano disponibili ulteriori informazioni sui bersagli. Per ciascun indirizzo è possibile memorizzare dati relativi alla sua localizzazione e alla rete di appartenenza.

In particolare, lo schema consente di rappresentare informazioni geografiche a diversi livelli, oltre a dati di geolocalizzazione quali latitudine, longitudine e *timezone*. Sono quindi previste informazioni sulla rete di appartenenza dell'indirizzo IP, con la relativa notazione CIDR e i principali attributi dell'*Autonomous System* (AS), quali numero identificativo (ASN), nome e dominio. Lo schema include infine informazioni relative alla risoluzione DNS e altri eventuali metadati associati all'indirizzo.

L'arricchimento dei dati relativi agli indirizzi IP deve essere gestito dai singoli CAPTURE MODULE; tuttavia, per evitare la duplicazione del codice e garantire uniformità nelle modalità di recupero delle informazioni, questa funzionalità è stata centralizzata in un componente dedicato, `IP_Lookup`, condiviso tra i diversi moduli e richiamabile quando necessario. Il componente implementa una classe progettata per estrarre le informazioni associate agli indirizzi IP da un insieme di database locali in formato *MaxMind DB*, quali i database *IPinfo Lite* [42] e *GeoLite2* [43], nelle varianti *City*, *Country* e *ASN*, distribuiti gratuitamente, secondo le relative condizioni di licenza [42, 43].

5.8 Code Templates

I CODE TEMPLATES definiscono la struttura che sarà adottata dai CAPTURE MODULE, separando le responsabilità relative alla cattura e all'elaborazione dei messaggi in due moduli distinti, come suggerito durante la descrizione del CODE AGENT. Il punto di ingresso principale è il modulo BOTNET, che implementa la BOTNET INTERFACE, eseguendo le attività di infiltrazione e cattura, mentre il parsing dei messaggi viene demandato al modulo MESSAGE PARSER.

Nei CODE TEMPLATES la classificazione dei messaggi viene rappresentata mediante l'enumerazione `Message_Type`, che definisce le quattro tipologie di messaggio individuate nel MESSAGE SCHEMA.

Infine, dal momento che il primo modulo utilizza funzionalità di logging e che il secondo dipende dalla classe centralizzata `IP_Lookup`, alla BOTNET INTERFACE è stato aggiunto un costruttore che consente di fornire le istanze del `logger` e di `IP_Lookup` già durante l'inizializzazione.

5.8.1 Modulo Botnet

Il template del modulo BOTNET fornisce un'esempio di implementazione dei metodi definiti dalla BOTNET INTERFACE, rispettandone i contratti, e predispone lo scheletro del modulo principale che dovrà essere completato in funzione dello specifico protocollo analizzato. La struttura dell'implementazione proposta può essere descritta a partire dai seguenti quattro elementi:

Attributi di classe: comprendono i parametri statici specifici del protocollo, quali l'*hostname* o l'indirizzo IP del C&C endpoint, le sequenze utilizzate durante l'handshake, i messaggi relativi agli acknowledgment e agli heartbeat e, più in generale, i valori costanti necessari alla comunicazione.

Costruttore `__init__`: inizializza le risorse e i componenti necessari all'esecuzione del modulo, tra cui il `socket`, il `buffer` utilizzato per la ricezione dei dati, il `logger` e il modulo Message Parser.

Metodo `connect`: stabilisce la connessione con il C&C endpoint e con il metodo `authentication` avvia la sequenza di inizializzazione prevista dal protocollo. Questa fase permette al modulo di registrarsi presso il C&C endpoint, simulando il comportamento del dispositivo di una potenziale vittima.

Metodo `sniff`: rimane in ascolto dei dati provenienti dal C&C endpoint e gestisce, quando previsto, l'invio periodico dei messaggi di keepalive tramite il metodo `heartbeat`. I dati ricevuti vengono eventualmente sottoposti a un pre-processing tramite `preprocess_stream` e sono successivamente aggiunti al buffer. Da quest'ultimo, tramite `extract_packets`, sono estratti i pacchetti, che possono essere decodificati o decifrati tramite `decode_packet`. Dai pacchetti elaborati vengono estratti i singoli messaggi mediante `extract_messages`, per poi essere classificati secondo il `Message_Type` tramite `get_message_type`. Inoltre, quando previsto dal protocollo, `handle_message` permette di eseguire l'azione attesa dal server, in risposta al messaggio ricevuto, come ad esempio l'invio di un acknowledgment. Il parsing del contenuto del messaggio viene demandato al modulo Message Parser tramite il metodo `parse_message`, che produce un dizionario Python, successivamente restituito tramite `yield`. L'uso dei dizionari garantisce una compatibilità nativa e immediata con il formato JSON, poiché entrambi condividono la stessa struttura basata su coppie chiave-valore.

Per rendere il template del modulo direttamente utilizzabile dal Code Agent, tutti i metodi sono accompagnati da *docstring* che ne descrivono il contratto, il comportamento atteso e le operazioni che devono essere implementate. In questo modo, nei prompt forniti al LLM è sufficiente riportare una descrizione generale del modulo, senza dover specificare nuovamente tutti i dettagli tecnici.

Una caratteristica particolare del template è la definizione dei metodi ausiliari con un'implementazione di base minimale. Questa scelta permette di esplicitare al LLM quali funzionalità sono opzionali e dipendono dallo specifico protocollo in esame: quando una determinata funzionalità è prevista, il relativo metodo dovrà essere sovrascritto con la logica specifica; in caso contrario, l'implementazione di base del template può essere mantenuta invariata.

5.8.2 Modulo Message Parser

Il template del modulo Message Parser definisce la sua struttura di base e fornisce un esempio di implementazione del metodo `parse_message`. Quest'ultimo sarà richiamato dal modulo Botnet e dovrà eseguire il parsing dei messaggi, restituendo un dizionario Python conforme alla struttura definita dal Message Schema e pronto per essere sottoposto alla relativa validazione.

L'implementazione di `parse_message` proposta estrae inizialmente le informazioni comuni alle diverse tipologie di messaggio e poi invoca il metodo dedicato all'elaborazione della specifica categoria: `parse_ack`, `parse_command`, `parse_heartbeat` o `parse_attack`. Nel caso dei messaggi di tipo `Attack`,

il modulo recupera le informazioni associate agli indirizzi IP dei bersagli dell'attacco attraverso la classe `IP_Lookup`. Il template implementa inoltre una logica di *fallback* nel caso in cui il parsing specifico non abbia esito positivo. Se uno dei metodi dedicati non riesce a interpretare il messaggio o incorre in un errore, il messaggio viene classificato come un generico `Command`, garantendo comunque la produzione di un risultato conforme allo schema previsto.

Anche il template del modulo MESSAGE PARSER segue gli stessi principi adottati per quello precedente, sia nella descrizione dei metodi tramite docstring, sia nelle modalità di implementazione.

5.9 Prompt Archive

In questa sezione vengono presentati i prompt utilizzati nei due componenti principali del sistema, ossia il SRE AGENT, con le macro-attività RENAMING TASK e REVERSING TASK, e il CODE AGENT.

Per aggiungere dinamicamente ai prompt tutte le informazioni necessarie viene utilizzato un meccanismo di *context injection*, basato su *placeholder* che possono essere valorizzati durante la costruzione del prompt. Questo meccanismo viene utilizzato sia per i materiali di riferimento forniti al CODE AGENT, sia, nel caso del SRE AGENT, per includere, quando disponibile, il contenuto della KNOWLEDGE BASE.

5.9.1 Renaming Task

Il system prompt assegna al LLM il ruolo di analista di malware specializzato in SRE e analisi binaria, con particolare attenzione alla ricostruzione della logica di binari stripped e alla rinomina delle funzioni. Richiede di identificare sistematicamente lo scopo delle funzioni, ricostruirne le relazioni e proporre nomi coerenti sulla base di evidenze verificabili. Il prompt stabilisce inoltre l'utilizzo delle funzionalità di analisi statica del SRE FRAMEWORK, sfruttando soprattutto la decompilazione e lo studio delle cross-reference per supportare le decisioni di rinomina.

L'user prompt della START MODE definisce il compito di ridenominazione delle funzioni all'interno di un binario stripped, partendo dall'identificazione della funzione *main*, per poi procedere ricorsivamente. L'attività principale consiste appunto nel rinominare le funzioni e, quando possibile, anche i parametri delle funzioni e le variabili locali, così da facilitare le successive analisi. Il prompt continua menzionando i tool MCP da utilizzare per effettuare le ridenominazioni. Infine, richiede ancora una volta di basare le sue azioni sulle evidenze raccolte durante l'analisi, riportando il ragionamento seguito e facendo un riepilogo delle attività svolte all'interno di un breve report.

L'user prompt della CONTINUE MODE definisce il compito di verificare e migliorare le ridenominazioni già effettuate su funzioni, parametri e variabili, individuando quelle inesatte e correggendole quando necessario. Il prompt richiede inoltre di identificare ulteriori opportunità di ridenominazione, cercando sempre di privilegiare gli elementi che hanno un impatto maggiore sulla comprensione del malware ed evitando di introdurre nomi potenzialmente fuorvianti. Rimangono invariate rispetto al prompt precedente le indicazioni relative ai tool, ai principi di analisi e all'output da produrre.

5.9.2 Reversing Task

Il system prompt assegna al LLM il ruolo di analista di malware specializzato in SRE e analisi binaria. Richiede di produrre risultati tecnici rigorosi, precisi e verificabili, basando ogni conclusione sulle eviden-

ze osservabili nel binario e indicando esplicitamente il livello di confidenza associato, distinguendo tra fatti verificati, ipotesi e informazioni sconosciute. Il prompt stabilisce l'uso combinato delle funzionalità di analisi statica del SRE Framework e degli strumenti disponibili nel Sandbox Environment per validare ipotesi, estrarre informazioni, analizzare dati offuscati e automatizzare attività ripetitive.

L'user prompt della Transport Analysis definisce il compito di analizzare e documentare il *Transport Layer* del malware, con particolare attenzione all'individuazione dei meccanismi e dei protocolli utilizzati per raggiungere l'infrastruttura C&C e alla creazione del canale di comunicazione. Il prompt richiede di determinare le modalità con cui vengono contattati gli endpoint, inclusi indirizzi, porte e domini, considerando eventuali tecniche di offuscamento, cifratura e recupero delle stringhe. Infine, richiede di produrre una specifica tecnica completa, sufficientemente dettagliata per la comprensione umana e direttamente utilizzabile per la successiva implementazione del Capture Module.

L'user prompt della Communication Analysis definisce il compito di analizzare e documentare il *Communication Layer* del malware, con particolare attenzione alle fasi di autenticazione, instaurazione della sessione C&C e al successivo ciclo di elaborazione dei comandi. Il prompt richiede di individuare e ricostruire i meccanismi di autenticazione e crittografia utilizzati, il formato e le tecniche di parsing dei messaggi, la gestione dei comandi e le modalità di mantenimento della sessione. Le indicazioni relative agli standard di analisi, alla struttura e al contenuto dell'output rimangono invariate.

5.9.3 Code Agent

Il system prompt assegna al LLM il ruolo di agente specializzato nella generazione di codice Python per moduli di rete. Richiede la produzione di codice completo ed eseguibile, conforme alle specifiche, alle informazioni e ai template forniti, definendo inoltre le modalità di estrazione delle informazioni dalla documentazione e i criteri di implementazione del codice. In particolare, stabilisce l'utilizzo di Python 3.11 o versioni successive, l'adozione di buone pratiche di sviluppo, un'adeguata gestione delle eccezioni e del logging.

L'user prompt per il modulo Botnet definisce il compito di generazione del componente di cattura dei messaggi, specificando che l'implementazione deve essere contenuta in un singolo file e deve ricostruire i layer di trasporto e comunicazione della botnet. Il prompt fornisce quindi i materiali di riferimento: la Botnet Interface, i SRE Reports, il template del modulo e l'enumerazione `Message_Type`, ciascuno affiancato da una breve descrizione. Vengono successivamente definiti i requisiti per l'implementazione dei metodi `connect` e `sniff`, specificando in generale il comportamento atteso. Infine, si richiede di rispettare il template esistente e di implementare esclusivamente le funzionalità necessarie, escludendo qualsiasi comportamento offensivo e restituendo in output il file completo.

L'user prompt per la generazione del modulo Message Parser, che segue quella del modulo Botnet, si limita a estendere i materiali di riferimento con il Message Schema e il relativo template. Il compito definito è la generazione del componente di parsing, il cui punto di ingresso è rappresentato dal metodo `parse_message`, responsabile di indirizzare i messaggi verso i metodi specifici, in base alla loro classificazione. Qualora si verifichino errori durante il parsing, viene indicato di utilizzare il tipo di messaggio generico `Command` come fallback. Anche in questo caso, il prompt richiede il rispetto del template e del formato di output previsto.

6

BotScope: valutazione

6.1 Obiettivi e criteri

Lo scopo generale della valutazione del sistema è quello di determinare il grado di maturità raggiunto, così da considerarne l'effettiva applicabilità nel contesto reale per cui è stato progettato e stabilire se il suo impiego possa risultare concretamente vantaggioso. La valutazione del sistema BotScope condotta in questa tesi si concentrerà su due obiettivi principali.

Il primo obiettivo, di carattere preliminare e meno rilevante, consiste nel verificare la correttezza funzionale del sistema, accertandosi che i tre moduli principali producano il comportamento atteso nello scenario operativo. In particolare, saranno verificati il corretto funzionamento della pipeline, il suo grado di autonomia e la produzione degli output previsti. Il secondo obiettivo, di maggiore rilevanza e su cui ci si concentrerà maggiormente durante la sperimentazione, consiste nel valutare l'accuratezza dei due output più importanti, i SRE Reports e il Capture Module, rispetto ai requisiti definiti.

A tal fine, saranno analizzati i risultati ottenuti su un insieme di campioni di malware sufficientemente vario e rappresentativo dello scenario operativo di riferimento. Per poter interpretare efficacemente e confrontare i risultati ottenuti sarà inoltre necessario definire una metodologia di classificazione che consenta di valutare oggettivamente il grado di complessità dei campioni.

In più, dovranno essere utilizzati più LLM, dimostrando l'indipendenza del sistema da uno specifico modello e, soprattutto, analizzando l'influenza delle loro differenti caratteristiche sui risultati ottenuti. Infine, le prestazioni in termini di risorse computazionali e tempi di esecuzione non costituiscono una dimensione di interesse per la seguente valutazione, poiché il sistema non è stato progettato per un utilizzo ad alta frequenza, né per operare sotto stringenti vincoli sul consumo di token del LLM.

La strategia di testing proposta prevede una valutazione *end-to-end* progressiva, condotta al termine di ciascun modulo principale della pipeline. Ad eccezione del Classifier, la cui valutazione risulta immediata una volta scelti i campioni, per gli altri due moduli è necessario effettuare un confronto con una *ground truth*. Quest'ultima è costituita dalla documentazione relativa al funzionamento e al comportamento dei malware analizzati, insieme ai moduli di cattura effettivamente funzionanti, ed è stata definita con il supporto di un esperto di cybersecurity e malware analysis.

La valutazione del Classifier considera due aspetti distinti: il riconoscimento della natura malevola del campione e l'individuazione della famiglia di botnet a cui appartiene.

La valutazione del SRE Agent si concentra sulla correttezza e sulla completezza delle informazioni riportate nei SRE Reports, al fine di determinare l'accuratezza della Reversing Task. In particolare, l'analisi tiene conto sia della presenza delle informazioni attese, sia della loro esattezza, penalizzando eventuali omissioni. Inoltre, per i binari stripped, è necessario valutare quantitativamente e qualitativamente le attività di rinomina svolte, al fine di misurare l'efficacia della Renaming Task.

Infine, la valutazione del Code Agent riguarda l'implementazione del Capture Module, verificandone la correttezza funzionale e la coerenza con le informazioni raccolte nei SRE Reports e con le specifiche fornite attraverso i prompt.

Come già evidenziato per la difficoltà dei campioni, anche le valutazioni delle informazioni estratte, della rinomina delle funzioni e del codice prodotto richiedono l'individuazione di criteri il più possibile oggettivi e riproducibili.

6.2 Configurazione sperimentale

6.2.1 Ambiente di esecuzione e LLM

Tutti gli esperimenti sono stati condotti utilizzando un computer personale dotato di hardware convenzionale, eseguendo il software sul sistema locale. L'inferenza dei LLM è stata invece eseguita sul *GPU cluster* dell'Università degli Studi di Udine, mediante il software vLLM [40].

Per la sperimentazione sono stati selezionati due LLM diversi, *Qwen 3.6-27B* [44] e *DeepSeek-V4-Flash Preview* [45]. Al fine di rendere più concisi i riferimenti ai LLM, si adotterà l'abbreviazione *Q3627* per il primo e *DS4FP* per il secondo. Nell'Appendice A sono descritte in dettaglio le caratteristiche dei LLM utilizzati, le risorse hardware impiegate, le configurazioni di deployment dei modelli e i parametri di campionamento usati per la presente sperimentazione.

La scelta dei LLM è stata effettuata soprattutto sulla base delle risorse computazionali disponibili, tenendo conto delle diverse dimensioni e caratteristiche architetturali dei modelli, nonché delle prestazioni riportate nelle principali valutazioni pubbliche, tra cui quelle raccolte da Artificial Analysis [46].

6.2.2 Protocollo sperimentale

Ciascun campione di malware è stato sottoposto a una singola esecuzione secondo un protocollo sperimentale uniforme che, grazie all'autonomia del sistema, è stato applicato in modo coerente a tutti i campioni. L'analisi è stata avviata manualmente fornendo in input il Binary File e la Knowledge Base, predisposta dall'esperto e mantenuta invariata per tutti gli esperimenti. Il sistema ha quindi eseguito le operazioni previste fino al completamento del processo, producendo come output i SRE Reports e il Capture Module, successivamente utilizzati per la valutazione dei risultati.

6.3 Definizione dei criteri di valutazione

6.3.1 Complessità del campione

Per caratterizzare la complessità di ciascun campione è necessario individuare gli elementi che contribuiscono a rendere più difficoltosa l'estrazione delle informazioni, attribuendo a ciascuno di essi un

punteggio proporzionale al grado di complessità introdotta.

A tal fine sono stati individuati i criteri riportati nella Tabella 6.1, insieme a una descrizione del loro ruolo e del relativo significato. Ad eccezione del primo criterio, che tiene conto del risultato della decompilazione prodotto dal framework di SRE adottato, tutti gli altri descrivono direttamente caratteristiche e proprietà del malware analizzato.

I criteri possono essere di tipo binario, con l'assegnazione del solo valore minimo o massimo, secondo la notazione $\{min, max\}$, oppure possono assumere valori variabili nell'intervallo $[min, max]$, in funzione della complessità osservata. In entrambi i casi, il punteggio minimo è pari a 0, mentre quello massimo varia in base alla caratteristica considerata. La complessità del campione, indicata come *Complexity Score*, è definita dalla somma dei singoli punteggi, su una scala da 0 a 165.

6.3.2 Efficacia della Renaming Task

Per la valutazione dell'efficacia della RENAMING TASK è necessario considerare due parametri principali. Il primo, indicato come $RC_{\%}$ e definito *Renaming Coverage*, rappresenta la percentuale di funzioni rinominate rispetto al totale delle funzioni presenti nel binario.

$$RC_{\%} = \frac{N_{renamed\ functions}}{N_{total\ functions}} \cdot 100$$

Il secondo parametro considera invece la qualità dei nomi assegnati e corrisponde alla percentuale di funzioni rinominate alle quali è stato associato un nome semanticamente corretto sulla base del comportamento osservato. Tale valore, indicato nel seguito con $RA_{\%}$, viene definito *Renaming Accuracy*.

$$RA_{\%} = \frac{N_{correctly\ named\ functions}}{N_{renamed\ functions}} \cdot 100$$

Considerando che la strategia adottata per la REVERSING TASK tende a coinvolgere principalmente le funzioni più rilevanti per l'analisi, escludendo quelle di minore interesse, e che il numero di funzioni presenti nei campioni è generalmente elevato, è difficile aspettarsi valori di $RC_{\%}$ prossimi al 100 %. Inoltre, all'aumentare del numero di funzioni rinominate, si può ipotizzare un effetto di *diminishing returns*, per cui il beneficio derivante da ulteriori ridenominazioni tende progressivamente a ridursi.

Risulta quindi logico proporre un punteggio composito che unisce i due parametri. Per tenere conto dei rendimenti decrescenti associati a $RC_{\%}$, viene applicata una trasformazione logaritmica attraverso la funzione g, normalizzata nell'intervallo $[0, 1]$ e dove $k > 0$ controlla il grado di attenuazione del contributo di $RC_{\%}$. Il punteggio finale, indicato con $RS_{\%}$ e chiamato *Renaming Score*, è volto a premiare una combinazione elevata di copertura e correttezza semantica.

$$RS_{\%} = g(RC_{\%}) \cdot RA_{\%}$$

$$g(x) = \frac{\log(1 + k \cdot \frac{x}{100})}{\log(1 + k)}$$

Criterio	Score	Descrizione
ghidra_dbf	$[0, 20]$	Quantifica la difficoltà per un reverse engineer nel comprendere e interpretare il risultato della decompilazione prodotto dal framework di SRE utilizzato, in questo caso Ghidra.
statically_linked	$\{0, 10\}$	Indica che il binario è stato collegato staticamente, incorporando il codice delle librerie esterne utilizzate e aumentando così la quantità di codice da analizzare.
stripped	$\{0, 20\}$	Indica che il binario è stato sottoposto al processo di stripping, con la conseguente rimozione di informazioni simboliche quali nomi di funzioni, parametri e variabili.
obfuscated_cc	$[0, 10]$	Quantifica il livello di offuscamento del C&C endpoint, considerando le modalità di memorizzazione e recupero, che possono variare da valori presenti in chiaro a valori cifrati, offuscati o recuperati dinamicamente a runtime.
dns_other_cc	$\{0, 5\}$	Indica che la risoluzione del C&C endpoint avviene tramite DNS o coinvolge altri meccanismi di risoluzione complessi.
enc_str	$[0, 10]$	Quantifica il livello di offuscamento della tabella delle stringhe, quando questa è presente, passando dalla memorizzazione in chiaro all'utilizzo di meccanismi di cifratura o offuscamento più complessi.
protocol	$[0, 20]$	Quantifica la complessità del protocollo di comunicazione utilizzato dal malware, considerando la struttura e le modalità di scambio dei dati, che possono variare da protocolli testuali e semplici a protocolli binari strutturati e più articolati.
passwd	$\{0, 5\}$	Indica che il malware richiede una password o una chiave per completare con successo il processo di autenticazione.
fingerprint	$\{0, 5\}$	Indica che il malware raccoglie informazioni relative al sistema compromesso.
enc_conn	$\{0, 10\}$	Indica che il malware utilizza un algoritmo di cifratura per cifrare e decifrare le comunicazioni.
dyn_att	$\{0, 20\}$	Indica che il malware registra dinamicamente, a runtime, i metodi di attacco.
non_std_parsing	$\{0, 10\}$	Indica che per il parsing dei comandi ricevuti il malware utilizza dei metodi che non sono tipici della famiglia di appartenenza.
has_response	$\{0, 10\}$	Indica che il malware risponde ai comandi ricevuti.
heartbeat	$[0, 10]$	Quantifica la complessità del meccanismo di heartbeat utilizzato dal malware, che può variare da una semplice comunicazione periodica a meccanismi più complessi basati su challenge-response.

Tabella 6.1: Criteri di valutazione della complessità dei campioni malevoli e relativi intervalli di punteggio. La notazione $\{min, max\}$ indica che il punteggio può assumere esclusivamente i valori min o max, mentre $[min, max]$ indica che può assumere qualsiasi valore compreso nell'intervallo.

6.3.3 Accuratezza della Reversing Task

In modo analogo alla valutazione della complessità del campione, è necessario definire i criteri che consentano di misurare l'efficacia e l'accuratezza della REVERSING TASK, il cui risultato è rappresentato dalla redazione dei SRE REPORTS. La valutazione considera quindi la capacità dell'agente di recuperare gli artefatti rilevanti, oltre che di ricostruire il comportamento e la logica di funzionamento del malware. Vengono penalizzate l'introduzione di ipotesi non supportate, la presenza di informazioni incoerenti o discordanti tra loro e la mancata identificazione di artefatti o elementi rilevanti.

I criteri scelti per la valutazione sono riportati nella Tabella 6.2, insieme a una descrizione della loro

Criterio	Score	Descrizione
host_info	$[0, 10]$	Misura il grado di successo nel recupero delle informazioni relative al C&C endpoint contattato dal malware, quali indirizzo IP, hostname e porta.
crypto_obfuscation	$[0, 15]$	Misura il grado di recupero dei meccanismi di cifratura e offuscamento utilizzati dal malware, includendo sia il funzionamento della cifratura del traffico sia l'algoritmo di decifratura e il contenuto originale della tabella delle stringhe; in caso di loro assenza, valuta la corretta conferma della stessa.
protocol_parsing	$[0, 20]$	Misura il grado di identificazione e ricostruzione del protocollo di comunicazione utilizzato dal malware, considerando il formato dei pacchetti e la logica di parsing e framing.
login_heartbeat	$[0, 15]$	Misura il grado di successo nell'individuazione della procedura di autenticazione, inclusi l'ordine e il contenuto dei messaggi scambiati, e nella ricostruzione della logica di heartbeat, inclusi la frequenza e i payload utilizzati.
payload_capabilities	$[0, 20]$	Misura il grado di individuazione e corretta associazione delle procedure di attacco alle rispettive funzionalità e ai payload utilizzati dal malware.
cc_lifecycle	$[0, 10]$	Misura il grado di ricostruzione del ciclo di vita della comunicazione C&C, con riferimento alla gestione degli acknowledgment dei messaggi ricevuti, alla trasmissione di feedback al C&C, alla gestione dei fallimenti di connessione, ai meccanismi di riconnessione e alle procedure di arresto del malware.
consistency_reliability	$[0, 10]$	Misura il grado di coerenza delle informazioni riportate nei SRE Reports, penalizzando la presenza di informazioni contraddittorie o incoerenti, anche quando le stesse risultano corrette in altre parti della documentazione.

Tabella 6.2: Criteri di valutazione della Reversing Task.

interpretazione. A ogni criterio può essere assegnato un punteggio compreso tra 0 e un valore massimo proporzionato alla sua importanza. I punteggi parziali consentono di valorizzare anche ricostruzioni incomplete ma corrette, evitando che un errore finale annulli le informazioni recuperate, a condizione che queste siano sufficientemente corrette. L'accuratezza della Reversing Task, indicata come *Reversing Score*, è definita dalla somma dei singoli punteggi, su una scala da 0 a 100.

6.3.4 Qualità del Capture Module

La valutazione del Capture Module prende in esame l'implementazione prodotta dal Code Agent in termini di funzionalità, correttezza e conformità alle specifiche, senza estendersi agli aspetti stilistici del codice. Inizialmente vengono considerati la correttezza sintattica del codice, l'aderenza ai Code Templates forniti e al Message Schema. A questi si aggiungono due ulteriori criteri, a cui è assegnato un peso maggiore in quanto più rappresentativi dell'effettiva funzionalità del modulo. Il primo misura la conformità al protocollo di comunicazione descritto nei SRE Reports, mentre il secondo valuta la *human independence*, ovvero il grado di autonomia del modulo rispetto alla revisione umana, con punteggi maggiori associati a un minor numero di modifiche necessarie per renderlo operativo.

I criteri considerati sono riportati nella Tabella 6.3, insieme alla relativa descrizione. I primi tre assumono esclusivamente il valore minimo o massimo previsto, mentre gli ultimi due possono assumere

Criterio	Score	Descrizione
syntax_correctness	$\{0, 10\}$	Verifica che il codice sia sintatticamente valido e che non presenti errori di compilazione o di importazione tali da impedirne il caricamento nell'ambiente operativo previsto.
architectural_adherence	$\{0, 15\}$	Verifica che il modulo Botnet sia conforme al contratto definito dalla Botnet Interface, accertando che implementi i metodi richiesti (`connect` e `sniff`), e che deleghi il parsing dei messaggi al modulo Message Parser.
schema_validation	$\{0, 15\}$	Verifica che i dati prodotti dal modulo siano conformi al JSON Schema Message Schema, rispettandone la struttura, i campi previsti, i relativi tipi e gli ulteriori vincoli specificati.
protocol_conformance	$[0, 20]$	Misura il grado con cui i moduli Botnet e Message Parser riproducono correttamente il comportamento del protocollo di C&C descritto nei SRE Reports, verificando che le informazioni estratte dai report siano state interpretate e tradotte adeguatamente nella logica dei moduli.
human_independence	$[0, 40]$	Misura la capacità del modulo generato di raggiungere autonomamente il comportamento atteso, senza richiedere modifiche o correzioni manuali da parte dell'umano.

Tabella 6.3: Criteri di valutazione del Capture Module.

anche valori intermedi. La qualità del Capture Module, indicata come *Module Score*, è definita dalla somma dei singoli punteggi, su una scala da 0 a 100.

6.4 Campagna sperimentale

Per la campagna sperimentale condotta nell'ambito di questa tesi, si è deciso di concentrare l'analisi su una singola famiglia di IoT botnet, *Mirai*, selezionando cinque campioni significativi. La scelta non è casuale, visto che, come descritto nella Sezione 2.1.3, anche a distanza di dieci anni dalla sua comparsa la famiglia Mirai continua a essere uno dei principali riferimenti nel panorama delle IoT botnet.

Tale rilevanza è confermata da due fonti indipendenti. Il *Threat Report 2025* di Zscaler ThreatLabz [47] identifica Mirai come la minaccia prevalente, con una quota del 40 %, seguita da Mozi e Gafgyt, entrambe con una quota inferiore al 20 %. Anche il report *IT Threat Evolution in Q2 2026* [48], basato in parte sulle osservazioni raccolte tramite gli IoT honeypot proprietari di Kaspersky, evidenzia la rilevanza di Mirai e segnala al contempo la crescente attività di un'altra botnet nota, Prometei.

6.4.1 Dataset dei campioni

Nella tabella Tabella 6.4 sono riportate le principali informazioni relative ai campioni selezionati, tra cui i nomi univoci utilizzati per identificarli e gli hash SHA-256 dei file binari. Come anticipato, tutti appartengono alla famiglia Mirai, sono in formato ELF e risultano *statically linked*; i primi due sono not-stripped, mentre gli ultimi tre sono stripped.

Per quanto riguarda la provenienza dei campioni, i primi tre sono stati catturati tramite l'honeypot proprietario del gruppo di ricerca in cybersecurity dell'Università degli Studi di Udine, mentre gli ultimi

Id	Nome	SHA-256	Dimensione	Architettura	Formato	Stripped
01	GHOST	0446b258525e6b3528dcf0d9da15387f32f8de9e544926979e9095462415e84e	197.64 KB	MIPS (32-bit)	ELF	×
02	MARVISXOXO	c5e55543cfd8f4534986b4fde98d96d7a8b38dc3afdd80da5b622d955cebfb32	163.12 KB	ARM (32-bit)	ELF	×
03	DEEPSORROW	f6d0e5142fd0d1a0f07e916c9c7432ffcbacc2a6d4eb0594f1524cb7661477ce	127.10 KB	x86 (32-bit)	ELF	✓
04	HOPTO	021e1c202a1d8defd3752ab749e50dac53645e40fa09d50aea9fa8004f48ba27	114.98 KB	x86 (64-bit)	ELF	✓
05	ONBOARDBOAT	9299d85a2b47b198d6381f4cc3afd296f271c1f0c303c8ae144a667bcf58e4c1	65.35 KB	ARM (32-bit)	ELF	✓

Tabella 6.4: Caratteristiche principali dei campioni di malware selezionati per la campagna sperimentale.

due sono stati reperiti dal database *MalwareBazaar* [49]. Attualmente, tutti i campioni sono disponibili su MalwareBazaar e possono essere recuperati direttamente mediante il relativo hash.

6.4.2 *Complexity Score* dei campioni

Sulla base dei criteri riportati nella tabella Tabella 6.1, la tabella Tabella 6.5 riporta il *Complexity Score* associato a ciascun campione scelto. I punteggi totali, espressi su un massimo teorico di 165, vengono anche normalizzati in termini percentuali.

La complessità dei campioni è piuttosto eterogenea, HOPTO è il più difficile da analizzare (62 %), seguito da DEEPSORROW (56 %) e MARVISXOXO (50 %); ONBOARDBOAT presenta una difficoltà intermedia (43 %), mentre GHOST è nettamente il più semplice (23 %).

Risulta particolarmente interessante osservare il contributo dei singoli criteri alla costruzione del punteggio complessivo. Alcuni criteri, come la cifratura della tabella delle stringhe e il parsing non standard dei comandi, sono presenti nella maggioranza dei campioni, mentre altri, ad esempio la gestione delle risposte e la cifratura della connessione, compaiono in uno soltanto. Il campione MARVISXOXO combina la maggior parte delle difficoltà considerate, risultando privo solo della registrazione dinamica degli attacchi e dei meccanismi di risposta, oltre a essere not-stripped. Escludendo i criteri non legati alle funzionalità e al comportamento del malware, ossia *ghidra_dbf*, *statically_linked* e *stripped*, MARVISXOXO presenta un punteggio maggiore a quello di DEEPSORROW e HOPTO, che hanno una difficoltà superiore principalmente a causa del criterio *dyn_att*.

6.5 Risultati sperimentali

Nella seguente sezione vengono riportati i principali risultati della campagna sperimentale, condotta secondo il protocollo definito e basata sulle valutazioni effettuate attraverso i criteri selezionati.

Ad eccezione della Tabella 6.7, i risultati sono organizzati in tabelle caratterizzate da una struttura comune. Le colonne seguono un indice gerarchico a due livelli, in cui il primo identifica il campione considerato, mentre il secondo indica il modello LLM utilizzato; le righe riportano invece i criteri di valutazione e i relativi punteggi.

Criterio	GHOST	MARVISXOXO	DEEPSORROW	HOPTO	ONBOARDBOAT
ghidra_dbf	2	6	13	14	13
statically_linked	10	10	10	10	10
stripped	0	0	20	20	20
obfuscated_cc	0	8	3	7	2
dns_other_cc	0	5	0	5	0
enc_str	0	8	2	2	2
protocol	4	13	12	12	12
passwd	5	5	0	0	0
fingerprint	5	5	0	0	0
enc_conn	0	10	0	0	0
dyn_att	0	0	20	20	0
non_std_parsing	0	10	10	10	10
has_response	10	0	0	0	0
heartbeat	2	2	2	2	2
Complexity Score	38	82	92	102	71
	23 %	50 %	56 %	62 %	43 %

Tabella 6.5: *Complexity Score* dei campioni di malware selezionati per la campagna sperimentale.

6.5.1 Statistiche di esecuzione

La Tabella 6.6 presenta alcune statistiche relative alle esecuzioni effettuate sui campioni, riportate a scopo puramente informativo e non considerate nella valutazione del sistema.

La metà superiore della tabella elenca il numero complessivo di token utilizzati dalle attività previste dalle Renaming Task e Reversing Task del SRE Agent e da quelle del Code Agent, oltre al totale. Il conteggio è riferito all'intera esecuzione e non distingue tra token di input, che comprendono prompt iniziali e risposte dei tool, e token di output, che includono reasoning, risposte e tool call.

La metà inferiore mostra invece i tempi di esecuzione corrispondenti alle medesime attività. Rispetto al numero di token, la misura dei tempi è maggiormente soggetta a variabilità e quindi meno precisa, pur fornendo una buona indicazione dell'impegno richiesto dalle diverse attività.

6.5.2 Raccolta dei risultati

La Tabella 6.7 riporta i risultati della valutazione del Classifier, nella quale è stato verificato che il modulo riconosca la natura malevola dei campioni e ne identifichi correttamente la famiglia di botnet di appartenenza.

La Tabella 6.8 riporta i risultati della valutazione della Renaming Task del SRE Agent. La valutazione riguarda esclusivamente i campioni stripped, ossia DEEPSORROW, HOPTO e ONBOARDBOAT, mentre GHOST e MARVISXOXO, essendo not-stripped, sono esclusi. Per completezza, questi ultimi contengono rispettivamente 329 e 368 funzioni. Le prime righe della tabella riportano il numero totale di funzioni, il numero di funzioni rinominate e quello delle funzioni a cui è stato assegnato un nome corretto.

Statistica		GHOST		MARVISXOXO		DEEPSORROW		HOPTO		ONBOARDBOAT	
		Q3627	DS4FP	Q3627	DS4FP	Q3627	DS4FP	Q3627	DS4FP	Q3627	DS4FP
Token [*tk*]	*Ren.*	N/A	N/A	N/A	N/A	251*k*	322*k*	270*k*	221*k*	242*k*	359*k*
	Rev.	250*k*	174*k*	296*k*	289*k*	207*k*	210*k*	241*k*	234*k*	292*k*	231*k*
	Code	27*k*	22*k*	39*k*	27*k*	52*k*	27*k*	46*k*	29*k*	34*k*	24*k*
	Totale	277*k*	196*k*	335*k*	315*k*	510*k*	559*k*	557*k*	485*k*	568*k*	613*k*
Tempo [*min* : *s*]	*Ren.*	N/A	N/A	N/A	N/A	18 : 46	09 : 06	14 : 38	09 : 30	19 : 05	09 : 53
	Rev.	13 : 10	04 : 55	30 : 23	16 : 00	14 : 37	11 : 58	15 : 46	14 : 03	26 : 22	14 : 55
	Code	01 : 06	00 : 51	04 : 27	01 : 14	06 : 35	01 : 04	05 : 05	01 : 14	03 : 19	00 : 52
	Totale	14 : 16	05 : 46	34 : 50	17 : 14	39 : 58	22 : 08	35 : 29	24 : 47	48 : 46	25 : 40

Tabella 6.6: Statistiche relative ai token utilizzati e ai tempi di esecuzione delle macro-attività del sistema, con distinzione tra quelle previste dalla Renaming Task (*Ren.*), dalla Reversing Task (*Rev.*) e dal Code Agent (*Code*).

Controllo	GHOST	MARVISXOXO	DEEPSORROW	HOPTO	ONBOARDBOAT
Riconoscimento Natura Malevola	✓	✓	✓	✓	✓
Riconoscimento Famiglia Mirai	✓	✓	✓	✓	✓

Tabella 6.7: Risultati della valutazione del Classifier.

Da questi valori sono stati quindi calcolati la *Renaming Coverage*, la *Renaming Accuracy* e il *Renaming Score*, secondo quanto definito nella Sezione 6.3.2. Nel calcolo del *Renaming Score*, la funzione g è stata parametrizzata con $k = 10$.

La Tabella 6.9 riporta i risultati della valutazione della Reversing Task del SRE Agent. La valutazione è stata effettuata sulla base dei criteri definiti nella Tabella 6.2, da cui è stato calcolato il *Reversing Score*.

La Tabella 6.10 riporta i risultati della valutazione dei Capture Module prodotti dal Code Agent. La valutazione è stata effettuata sulla base dei criteri definiti nella Tabella 6.3, calcolando infine il *Module Score*.

6.6 Discussione dei risultati

Per tutte le esecuzioni effettuate, il sistema ha operato in maniera autonoma secondo il flusso previsto, completando le diverse fasi della pipeline senza richiedere interventi manuali. Questo conferma la corretta integrazione dei diversi componenti e consente quindi di passare alla discussione dei risultati ottenuti per i singoli moduli.

La valutazione del Classifier, riportata in Tabella 6.7, è stata necessariamente limitata ai campioni scelti. In tali casi, il Classifier si è dimostrato sufficientemente affidabile, visto che non ci sono stati errori di classificazione. Una valutazione più completa richiederebbe, tuttavia, di considerare anche casi

Criterio	DEEPSORROW		HOPTO		ONBOARDBOAT	
	Q3627	DS4FP	Q3627	DS4FP	Q3627	DS4FP
Funzioni Totali	450	450	240	240	247	247
Funzioni Rinominate	169	172	133	129	169	141
Funzioni Non Rinominate	281	278	107	111	78	106
Nomi Corretti	147	137	98	102	152	129
Nomi Errati	22	35	35	27	17	12
Renaming Coverage ($RC_{\%}$)	37.6	38.2	55.4	53.8	68.4	57.1
Renaming Accuracy ($RA_{\%}$)	87.0	79.7	73.7	79.1	89.9	91.5
Renaming Score ($RS_{\%}$)	56.6	52.3	57.7	61.1	77.2	72.6

Tabella 6.8: Risultati della valutazione della Renaming Task.

Criterio	GHOST		MARVISXOXO		DEEPSORROW		HOPTO		ONBOARDBOAT	
	Q3627	DS4FP	Q3627	DS4FP	Q3627	DS4FP	Q3627	DS4FP	Q3627	DS4FP
host_info	10.0	10.0	7.0	8.0	10.0	10.0	8.0	10.0	10.0	10.0
crypto_obfuscation	14.5	13.5	13.0	13.0	15.0	14.0	15.0	15.0	11.0	15.0
protocol_parsing	19.0	19.0	18.0	14.0	20.0	20.0	20.0	20.0	12.5	20.0
login_heartbeat	12.0	13.0	11.0	11.0	15.0	15.0	15.0	15.0	9.0	12.0
payload_capabilities	17.0	19.0	18.0	16.0	6.0	6.0	14.0	9.0	6.0	20.0
cc_lifecycle	7.0	7.0	8.0	4.0	9.0	9.0	10.0	10.0	7.0	10.0
consistency_reliability	5.0	5.0	5.0	5.0	5.0	5.0	5.0	5.0	5.0	5.0
Reversing Score	84.5	86.5	80.0	71.0	80.0	79.0	87.0	84.0	60.5	92.0

Tabella 6.9: Risultati della valutazione della Reversing Task.

limite, utili a verificare il rilevamento di eventuali falsi positivi e falsi negativi e la capacità di distinguere correttamente tra più famiglie di botnet, in particolare con campioni che presentano caratteristiche riconducibili a più di una famiglia.

A questo punto, per fornire una visione sintetica dei risultati e facilitare il confronto tra i diversi LLM e i campioni, sono stati realizzati i grafici a barre mostrati nella Figura 6.1, che riportano i punteggi complessivi ottenuti nelle tre valutazioni principali.

A partire dalle statistiche raccolte nella Tabella 6.6 e dai risultati riassunti nella Figura 6.1, è possibile confrontare ad alto livello i due LLM nelle attività considerate. Q3627 e DS4FP presentano prestazioni generalmente comparabili, con differenze più evidenti nella produzione del codice, in cui DS4FP mostra un vantaggio più marcato. Anche il consumo di token risulta nel complesso simile, sebbene DS4FP ne utilizzi di più per la Renaming Task e di meno nel Code Agent. Per quanto riguarda i tempi di esecuzione, DS4FP risulta costantemente più rapido, con tempi circa dimezzati rispetto a Q3627. Tale differenza è compatibile con le caratteristiche architetturali dei due modelli, in particolare con l'architettura sparsa di DS4FP, che attiva circa la metà dei parametri rispetto a Q3627 [44, 45]. La vicinanza dei punteggi ottenuti risulta inoltre coerente con alcuni benchmark pubblici. Nell'Artificial

Criterio	GHOST		MARVISXOXO		DEEPSORROW		HOPTO		ONBOARDBOAT	
	Q3627	DS4FP	Q3627	DS4FP	Q3627	DS4FP	Q3627	DS4FP	Q3627	DS4FP
syntax_correctness	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0
architectural_adherence	15.0	15.0	15.0	15.0	15.0	15.0	15.0	15.0	15.0	15.0
schema_validation	15.0	15.0	15.0	15.0	15.0	15.0	15.0	15.0	15.0	15.0
protocol_conformance	20.0	20.0	18.0	14.0	16.0	18.0	14.0	16.0	14.0	16.0
human_independence	38.0	38.0	22.0	20.0	18.0	32.0	22.0	28.0	18.0	28.0
Module Score	98.0	98.0	80.0	74.0	74.0	90.0	76.0	84.0	72.0	84.0

Tabella 6.10: Risultati della valutazione dei Capture Module.

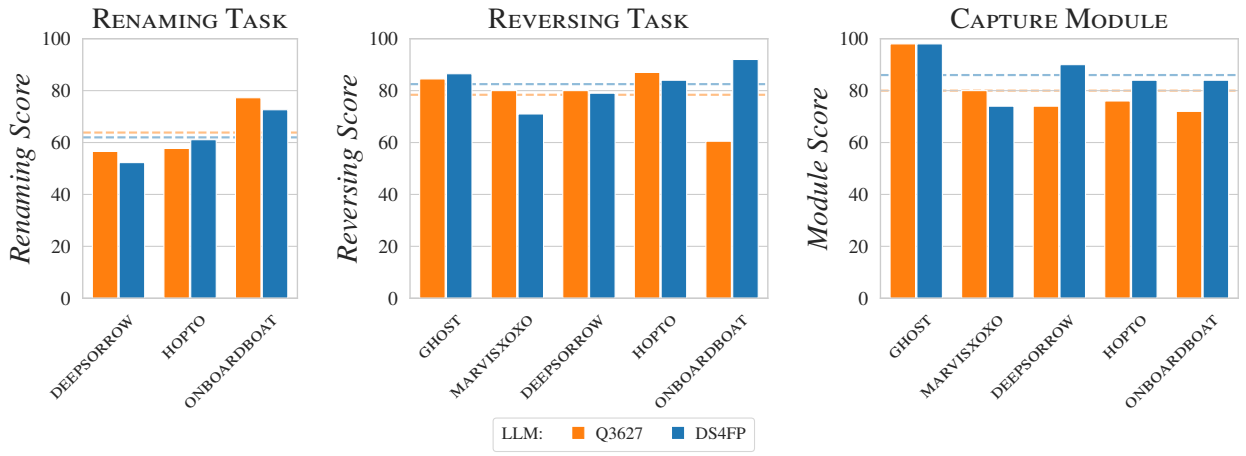


Figura 6.1: Confronto tra i punteggi *Renaming Score*, *Reversing Score* e *Module Score* ottenuti dai due LLM considerati nelle valutazioni di Renaming Task, Reversing Task e Capture Module su tutti i campioni selezionati. Le linee orizzontali rappresentano le medie delle rispettive serie.

Analysis Intelligence Index v4.3.2 [46], che aggrega dieci valutazioni delle capacità dei LLM in vari ambiti, DS4FP ottiene 24 punti su 100, contro i 21 di Q3627. Nel Terminal-Bench v2.1 [46, 50], che comprende anche attività simili a quelle di SRE e cybersecurity implementate in questo sistema, DS4FP raggiunge un punteggio del 61.8 %, contro il 60.7 % di Q3627. I risultati esterni considerati sono quindi coerenti con gli andamenti osservati nel sistema, pur appartenendo a contesti di valutazione differenti.

Tornando all'analisi delle singole attività, per la Renaming Task la Tabella 6.8 evidenzia una buona *Renaming Accuracy*, con valori medi compresi tra l'80 % e l'85 %. Più contenuta risulta invece la *Renaming Coverage*, che si attesta mediamente tra il 50 % e il 55 % e costituisce il principale fattore di riduzione del *Renaming Score*, i cui valori medi si collocano tra il 60 % e il 65 %. I risultati suggeriscono che, nella maggior parte dei casi in cui il modello decide di rinominare una funzione, assegna un nome corretto, ma al tempo stesso una parte significativa delle funzioni non viene rinominata. La combinazione di elevata accuratezza e copertura più limitata evidenzia quindi un comportamento della Reversing Task tendenzialmente conservativo, anche a fronte di due cicli agentici di rinomina. Tuttavia, la valutazione presenta due principali limitazioni: da un lato, la *Renaming Coverage* non tiene conto della diversa importanza delle funzioni all'interno del programma, attribuendo a tutte lo stesso peso; dall'altro, la *Renaming Accuracy* non riflette i diversi gradi di correttezza dei nomi assegnati, senza distinguere tra nomi pienamente corretti, parzialmente corretti o fuorvianti.

La Reversing Task ha ottenuto punteggi medi intorno all'80 %, evidenziando l'utilità dei SRE Reports come sintesi dell'attività di SRE e come output finale del modulo. Dalla revisione dei report emerge infatti un livello di completezza e correttezza sufficiente a fornire al reverse engineer una valida base di partenza per proseguire autonomamente con l'analisi e la successiva revisione del Capture Module. Tuttavia, il principale fattore penalizzante, comune alle diverse esecuzioni, è la scarsa consistenza dei SRE Reports, caratterizzati da sovrapposizioni e contraddizioni riconducibili in gran parte alla natura parallela delle modalità di analisi Transport Mode e Communication Mode nella Reversing Task. Ciò mette in luce una variabilità tra esecuzioni agentiche indipendenti non trascurabile, che può tradursi in una diversa presentazione degli stessi risultati o, nei casi più critici, in risultati discordanti. Le altre riduzioni dei punteggi che emergono dalla Tabella 6.9 risultano invece abbastanza uniformemente distribuite tra i diversi criteri; tra questi, il riconoscimento delle capacità del payload è l'attività di SRE con i risultati meno favorevoli. Inoltre, è interessante notare l'esito della combinazione onboardboat–Q3627: a fronte del buon risultato ottenuto da DS4FP sullo stesso campione, Q3627 mostra una prestazione sensibilmente peggiore, nonostante un alto *Renaming Score* e un livello di complessità del campione intermedio. Questo comportamento verrà ripreso durante la successiva analisi del *Reversing Score* in relazione al *Complexity Score*.

Per quanto riguarda la valutazione del Capture Module, i *Module Score* risultano complessivamente elevati, grazie al contributo dei punteggi massimi ottenuti per i criteri relativi agli aspetti sintattici, architetturali e di integrazione. Tale risultato è coerente con l'obiettivo del sistema di agevolarne l'utilizzo all'interno di altri workflow e con il requisito di limitare le implementazioni dei moduli alle sole funzionalità di infiltrazione e cattura, escludendo quelle di natura offensiva. Rispetto ai *Reversing Score*, non si osservano differenze particolarmente marcate, anche considerando i diversi gradi di successo nell'implementazione delle funzionalità del modulo. Come evidenziato nella Tabella 6.10, le differenze osservate tra i moduli dipendono principalmente dal criterio *human_independence*, che risente maggiormente degli errori accumulati nelle diverse fasi della pipeline.

Come anticipato, i dati raccolti consentono di analizzare la relazione tra la complessità del campione, misurata mediante il *Complexity Score*, e il punteggio ottenuto nella valutazione della Reversing Task, espresso dal *Reversing Score*. Si ipotizza che le due variabili siano negativamente associate, ovvero che all'aumentare del *Complexity Score* corrisponda una diminuzione del *Reversing Score*. Tale relazione potrebbe essere lineare oppure, più in generale, monotona, senza assumere una specifica forma funzionale.

Per la valutazione dell'ipotesi, si procede all'analisi dei dati relativi a ciascun LLM. In primo luogo, viene realizzato un grafico a dispersione, riportato in Figura 6.2, al fine di rappresentare graficamente la relazione tra il *Complexity Score* e il *Reversing Score*. Successivamente, per verificare la presenza di una relazione lineare, viene calcolato il coefficiente di correlazione di Pearson, mentre per verificare l'esistenza di una relazione monotona, si utilizza il coefficiente di correlazione di Kendall. Poiché l'ipotesi formulata è direzionale e prevede un'associazione negativa, per entrambi i coefficienti viene testata un'ipotesi alternativa unilaterale, assumendo un livello di significatività pari a $\alpha = 0.05$. I valori dei coefficienti e dei relativi *p-value* sono riportati in Tabella 6.11.

L'osservazione della Figura 6.2 e, soprattutto, dei risultati presentati nella Tabella 6.11 non fornisce evidenza statistica sufficiente a supportare l'ipotesi dell'esistenza di un'associazione negativa tra *Complexity Score* e *Reversing Score*. Sebbene per DS4FP entrambi i coefficienti assumano valori negativi, i

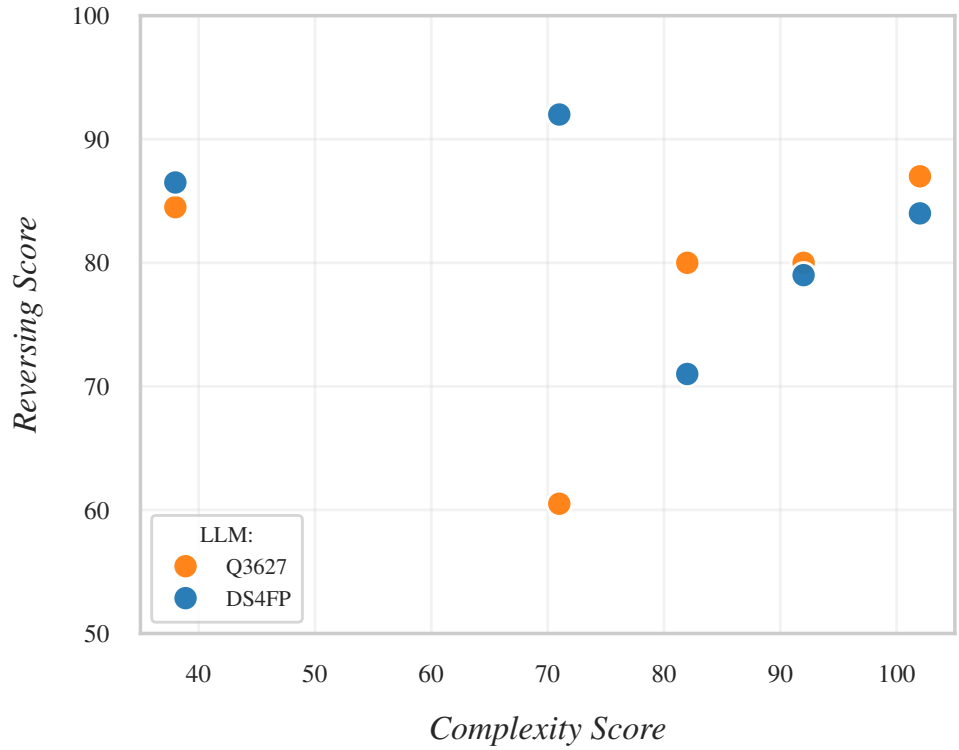


Figura 6.2: Relazione tra i punteggi *Complexity Score* e *Reversing Score* per i diversi LLM.

Test		**Q3627**	**DS4FP**
Pearson	r	0.113	−0.364
	p	0.572	0.274
Kendall	τ	0.316	−0.200
	p	0.776	0.408

Tabella 6.11: Risultati delle correlazioni di Pearson e Kendall tra *Complexity Score* e *Reversing Score*.

relativi *p-value* non raggiungono la soglia di significatività α. Per Q3627, invece, entrambi i coefficienti assumono valori positivi, in contrasto con la direzione prevista dall'ipotesi. Tali risultati devono tuttavia essere interpretati con cautela, considerando la dimensione estremamente ridotta del campione ($n = 5$), che limita fortemente la potenza statistica dei test e la possibilità di generalizzare le osservazioni.

Nel complesso, i risultati suggeriscono che il *Complexity Score*, considerato isolatamente, non sia sufficiente a spiegare la performance nella Reversing Task. In linea con quanto già emerso dalle precedenti analisi, che hanno evidenziato una certa variabilità tra le diverse esecuzioni del LLM e la presenza di un outlier, il successo nella Reversing Task può essere interpretato come il prodotto dell'interazione tra più fattori, riconducibili al campione considerato, al LLM impiegato e alla singola esecuzione. In questa prospettiva, il *Reversing Score* può essere concettualmente rappresentato come una funzione di questi tre fattori, senza che sia possibile quantificarne separatamente il contributo o valutarne le eventuali interazioni.

$$\textit{Reversing Score} = f(\text{Campione}, \text{LLM}, \text{Esecuzione})$$

Infine, è opportuno considerare le implicazioni dei risultati ottenuti rispetto al rapporto tra il sistema e l'intervento umano. Come già discusso, il sistema è in grado di operare autonomamente fino alla costruzione di Capture Module compatibili con le interfacce previste e potenzialmente integrabili all'interno di workflow più ampi, senza richiedere interventi durante l'esecuzione. Questo comportamento soddisfa il requisito di indipendenza del sistema dall'umano, pur lasciando spazio a una supervisione utile per monitorare le azioni intraprese e le strategie di analisi adottate.

D'altra parte, l'utilizzo operativo dei Capture Module senza alcuna revisione umana risulta più difficile da ipotizzare, poiché nella maggior parte dei casi è necessario apportare alcune modifiche e correzioni prima del loro impiego. Il sistema appare pertanto maggiormente maturo per scenari in cui gli output prodotti vengano sottoposti a una verifica finale da parte dell'umano, considerando che il contributo richiesto risulta contenuto. Il raggiungimento di una piena autonomia richiederebbe invece ulteriori miglioramenti, soprattutto nella Reversing Task, in termini di affidabilità nell'estrazione delle informazioni e di consistenza tra le diverse esecuzioni.

7

Conclusioni

7.1 Sintesi del lavoro

L'evoluzione dei malware, con riferimento alla progressiva diffusione delle botnet nell'ecosistema IoT, e il rapido sviluppo dei Large Language Model, unitamente alla loro crescente adozione in un'ampia varietà di applicazioni, costituiscono due fenomeni rilevanti che, seppur afferenti a contesti diversi, rivestono un ruolo significativo nel panorama dell'informatica contemporanea. Proprio dall'incontro di queste due tendenze prende forma il presente lavoro, che esplora l'impiego dei LLM nell'analisi del malware e nella generazione di moduli per la cattura e lo studio delle comunicazioni C&C delle IoT botnet.

In particolare, il lavoro è partito dalla caratterizzazione del processo seguito da un esperto di cybersecurity per la realizzazione di tali moduli. L'obiettivo primario era automatizzare, per quanto possibile, le principali fasi di analisi attraverso un sistema facilmente integrabile in altri workflow. Da questa esigenza è nato BotScope, di cui sono stati inizialmente definiti i confini, le responsabilità e i principali artefatti di input e output. La progettazione del sistema ha portato all'organizzazione delle sue funzionalità in tre moduli principali, denominati Classifier, SRE Agent e Code Agent, con questi ultimi due basati sulle capacità offerte dai LLM, in particolare quelle di natura agentica.

Successivamente, si è proceduto all'implementazione di BotScope, durante la quale le principali sfide affrontate sono state individuare e fornire, attraverso il MCP, i tool rilevanti per le analisi agli agenti e, soprattutto, garantire l'*alignment* del LLM con i compiti da svolgere attraverso il prompt engineering. Per la validazione complessiva del lavoro è stata fondamentale anche l'attività sperimentale, definendo degli obiettivi e un protocollo che hanno permesso di valutare il sistema sia nelle sue singole componenti, sia nel suo insieme. I risultati ottenuti sono già stati ampiamente discussi al termine del Capitolo 6, dal quale è emerso che BotScope è in grado di portare a termine in modo autonomo le attività previste, raggiungendo un livello complessivo di accuratezza soddisfacente. Il sistema rappresenta senza dubbio un passo avanti rispetto al processo interamente manuale preesistente, ma, come sottolineato nel capitolo precedente, le piccole imprecisioni introdotte progressivamente dal LLM tendono ad accumularsi nel corso dell'esecuzione, rendendo opportuna una revisione finale da parte dell'esperto di cybersecurity. Risulta pertanto più realistica un'applicazione di BotScope secondo un modello di autonomia supervisionata, nel quale l'intervento umano si limita alla verifica dei risultati prodotti al termine dell'esecuzione.

7.2 Limiti e attività rimanenti

Oltre alle limitazioni in termini di autonomia complessiva e di accuratezza dei risultati, allo stato attuale il lavoro realizzato presenta anche altri punti deboli, che è opportuno evidenziare.

Una parte di questi riguarda alcune scelte implementative adottate. In particolare, il sistema risente della natura non deterministica e probabilistica dei LLM, rendendo l'efficacia dell'analisi fortemente dipendente dal percorso seguito dall'agente. Questo incide soprattutto sulla riproducibilità dei risultati e, in misura minore, anche sulla loro accuratezza. Inoltre, la KNOWLEDGE BASE, per come è stata definita, è statica e non prevede un'evoluzione autonoma, rendendo necessario l'intervento umano per aggiornarne i contenuti.

Le principali limitazioni del lavoro presentato sono, tuttavia, legate ai test svolti. In primo luogo, alcuni dei criteri di valutazione individuati presentano una granularità medio-alta, risultando in certi casi potenzialmente poco indicativi delle carenze riscontrate e non catturando direttamente l'impatto che gli errori e le allucinazioni del LLM introducono sull'analisi. In secondo luogo, le sperimentazioni sono state condotte su un dataset di dimensioni ridotte, con campioni appartenenti a una sola famiglia di IoT botnet. Questi aspetti pregiudicano fortemente la generalizzabilità dei risultati, anche in virtù dei possibili margini di soggettività presenti nella valutazione.

Alla luce delle limitazioni emerse, la valutazione sperimentale dovrebbe essere ulteriormente ampliata e consolidata. In particolare, sarebbe opportuno estendere i test a un numero significativamente maggiore di campioni e a diverse famiglie di botnet, anche non necessariamente appartenenti all'ambito IoT. Sarebbe inoltre interessante sperimentare il sistema con LLM caratterizzati da capacità differenti rispetto a quelle dei due utilizzati, includendo modelli con prestazioni sia inferiori che superiori, fino ad arrivare ai *frontier LLM*, dotati di capacità significativamente maggiori. Inoltre, resta da caratterizzare in maniera approfondita l'impatto della presenza della KNOWLEDGE BASE sulle analisi, quantificando in che misura le prestazioni possano risentire della sua assenza. In modo simile, si può esplorare maggiormente la selezione di diverse combinazioni di tool del SRE FRAMEWORK, valutandone l'impatto e gli eventuali benefici sull'analisi.

In generale, queste attività permetterebbero di ampliare e rendere più preciso il feedback raccolto sul sistema, fornendo una base più solida da cui delineare i suoi possibili sviluppi futuri.

7.3 Sviluppi futuri

I risultati ottenuti e le osservazioni formulate fino a questo punto consentono di individuare tre principali direzioni lungo le quali, qualora si decidesse di proseguire o ampliare il lavoro, il sistema BOTSCOPE potrà essere ulteriormente sviluppato.

Le tre possibilità considerate presentano una natura abbastanza diversa tra loro, ma non sono mutuamente esclusive. La prima riguarda l'aggiunta all'interno del SRE AGENT di ulteriori funzionalità e strumenti basati sul MCP, sfruttando la modularità offerta dal protocollo. Questo potrebbe tradursi nell'estensione delle capacità del SANDBOX ENVIRONMENT, nell'inclusione di funzionalità per l'analisi dinamica del SRE FRAMEWORK, ma anche nell'integrazione di altri server MCP non originariamente contemplati. Questi potrebbero fornire funzionalità di carattere generale, come la ricerca web, oppure essere specificamente orientati alla cybersecurity, ad esempio per attività di threat intelligence.

Una seconda direzione di sviluppo punta a superare le attuali limitazioni della KNOWLEDGE BASE. Un'ipotesi interessante consiste nel comporre la KNOWLEDGE BASE di una conoscenza di riferimento e di una conoscenza specifica per ciascun campione già analizzato, rappresentata da un artefatto che ne sintetizza le peculiarità e gli aspetti rilevanti rispetto al contenuto di riferimento. In questo modo, la KNOWLEDGE BASE non assumerebbe più il ruolo di semplice guida, ma potrebbe essere aggiornata dall'agente al termine di ogni analisi, evolvendosi progressivamente e adeguandosi ai campioni effettivamente osservati. Una soluzione simile introdurrebbe tuttavia una maggiore complessità, ponendo sfide implementative legate al recupero delle informazioni pertinenti al campione corrente e al mantenimento della qualità della conoscenza, mitigando il rischio di degradazione delle informazioni nel tempo.

Infine, per come è stato concepito BOTSCOPE, appare difficile raggiungere un livello di autonomia totale e incrementare ulteriormente l'accuratezza senza intervenire in modo sostanziale sulla sua architettura complessiva e su quella dei singoli componenti. L'ultima strada di sviluppo riguarda proprio questo aspetto e consiste in una revisione globale della pipeline di analisi, ridefinendo in tutto o in parte la suddivisione e le responsabilità dei componenti. Interventi meno invasivi possono riguardare, ad esempio, l'introduzione di un'analisi preliminare volta a caratterizzare ad alto livello il malware prima della RENAMING TASK e della REVERSING TASK, decidendo di procedere con le analisi successive solo nei casi ritenuti rilevanti. Un'altra opportunità significativa di miglioramento potrebbe interessare il SRE AGENT, con l'introduzione di un *verification loop*, incaricato di verificare l'output prodotto e fornire un feedback all'agente. Quest'ultimo potrebbe così ripetere l'analisi sulla base delle informazioni aggiuntive ricevute, indipendentemente dall'esito della verifica oppure solamente in caso di esito negativo.

Una modifica più sostanziale consisterebbe invece nel ripensare il sistema BOTSCOPE come un agente orchestratore, rendendolo responsabile della gestione autonoma della pipeline di analisi e trasformando gli attuali CLASSIFIER, SRE AGENT e CODE AGENT in suoi *sub-agent*. Questi verrebbero coordinati e supervisionati dall'orchestratore secondo il pattern *Supervisor–Subagents*, derivato dall'adattamento del pattern architetturale *Orchestrator–Worker* al contesto dei sistemi agentici. Tale evoluzione risulterebbe particolarmente impegnativa, poiché introdurrebbe una maggiore complessità nella gestione dello stato e del flusso di esecuzione, esponendo il sistema al rischio di *state drift* tra l'orchestratore e i diversi sub-agent, nonché alla possibilità di cicli o esecuzioni non terminanti, con una conseguente perdita del determinismo che caratterizza l'attuale pipeline di analisi.

A

LLM e inferenza

Come anticipato nel Capitolo 6, i LLM utilizzati per le valutazioni sperimentali del sistema sono stati eseguiti sul GPU cluster dell'Università degli Studi di Udine, dotato di diverse GPU NVIDIA H100 Tensor Core basate sull'architettura NVIDIA Hopper, ciascuna con 80 GB di memoria HBM3 e una larghezza di banda superiore a 3 TB/s [51].

A.1 LLM e configurazione dell'inferenza

Le principali caratteristiche dei due LLM utilizzati, Q3627 e DS4FP, vengono descritte di seguito.

Qwen 3.6-27B (Q3627, [44]): sviluppato dal Qwen Team di Alibaba e appartenente alla famiglia Qwen 3.6, è stato rilasciato il 22 aprile 2026 con pesi aperti e licenza Apache 2.0. Il modello integra nativamente funzionalità multimodali, supportando in input anche immagini e video, ed è caratterizzato da un'architettura densa con 27 miliardi di parametri. Presenta una finestra di contesto nativa di circa $262k$ token, estendibile fino a un milione di token. Rispetto alle generazioni precedenti, introduce significativi miglioramenti nelle attività agentiche, con particolare riferimento allo sviluppo software, e nelle capacità di ragionamento.

DeepSeek-V4-Flash Preview (DS4FP, [45]): sviluppato da DeepSeek e appartenente alla famiglia DeepSeek-V4, è stato rilasciato il 24 aprile 2026 in versione *preview* come modello con pesi aperti e licenza MIT. Adotta un'architettura di tipo *Mixture-of-Experts* (MoE), con circa 284 miliardi di parametri complessivi, dei quali 13 miliardi vengono attivati per ciascun token. Il modello supporta una finestra di contesto fino a un milione di token ed è progettato per offrire elevate capacità di ragionamento e un efficace utilizzo in scenari agentici, con prestazioni simili alla variante V4-Pro nei compiti meno complessi.

Per Q3627 è stata utilizzata la versione ufficiale quantizzata in FP8, sufficientemente compatta da consentirne l'esecuzione su una singola GPU NVIDIA H100. Per DS4FP è stato invece impiegato il modello ufficiale in *mixed precision* FP4/FP8. In quest'ultimo caso, considerando congiuntamente l'occupazione dei pesi del LLM e della *KV cache*, il serving richiede quattro GPU in parallelo, permettendo al contempo di sfruttare l'intera finestra di contesto. La Figura A.1 riporta le configurazioni di vLLM [40], denominate *vLLM recipe*, adottate per il deployment dei due LLM.

```yaml
model: Qwen/Qwen3.6-27B-FP8
tensor-parallel-size: 1
attention-backend: FLASHINFER
performance-mode: interactivity
max-model-len: auto
gpu-memory-utilization: 0.90
compilation-config:
  mode: VLLM_COMPILE
optimization-level: 3
async-scheduling: true
tool-call-parser: qwen3_coder
reasoning-parser: qwen3
enable-auto-tool-choice: true
speculative-config:
  method: mtp
  num_speculative_tokens: 2
default_chat_template_kwargs:
  preserve_thinking: true
mamba-cache-mode: all
mamba-block-size: 8
enable-prefix-caching: true
enable-chunked-prefill: true
```

(a) Configurazione di Qwen 3.6-27B

```yaml
model: deepseek-ai/DeepSeek-V4-Flash
tensor-parallel-size: 4
performance-mode: interactivity
max-model-len: auto
gpu-memory-utilization: 0.90
trust-remote-code: true
kv-cache-dtype: fp8
block-size: 256
enable-expert-parallel: true
tokenizer-mode: deepseek_v4
tool-call-parser: deepseek_v4
enable-auto-tool-choice: true
reasoning-parser: deepseek_v4
speculative-config:
  method: mtp
  num_speculative_tokens: 1
default-chat-template-kwargs:
  thinking: true,
  reasoning_effort: high
enable-prefix-caching: true
enable-chunked-prefill: true

```

(b) Configurazione di DeepSeek-V4-Flash Preview.

Figura A.1: Configurazioni vLLM utilizzate per i LLM

Agente	`temperature`	`top_p`	`top_k`	`min_p`	`presence penalty`	`frequency penalty`	`repetition penalty`
SRE Agent	0.80	0.95	40	0.00	0.00	0.00	1.00
Code Agent	0.60	0.90	20	0.00	0.00	0.00	1.00

Tabella A.1: Parametri di campionamento definiti per il SRE Agent e il Code Agent.

A.2 Parametri di campionamento

In funzione delle caratteristiche delle attività svolte dal SRE Agent e dal Code Agent, sono state definite a priori due configurazioni dei parametri di campionamento, riportate nella Tabella A.1. Per il primo agente è stata proposta una configurazione contraddistinta da maggiore variabilità, coerente con la natura delle attività di analisi e SRE, mentre per il secondo sono stati utilizzati valori più conservativi, considerata la specificità della generazione del codice. Queste scelte non costituiscono una valutazione sperimentale dei parametri, dal momento che non è stato condotto uno studio comparativo volto alla loro ottimizzazione. Inoltre, i valori impostati non corrispondono necessariamente a quelli effettivamente utilizzati durante l'inferenza, in quanto la loro gestione varia a seconda del LLM considerato.

Per Q3627, i cui parametri raccomandati [52] risultano simili a quelli proposti, sono stati mantenuti i valori della Tabella A.1, attivando la *Thinking Mode*, ma senza specificare il *reasoning effort*, in quanto non previsto dal LLM. Per DS4FP è stato dapprima impostato il livello più alto di *reasoning effort*, pari a `max`. Con l'utilizzo della *Thinking Mode* [53], i parametri `temperature`, `presence_penalty` e `frequency_penalty` non hanno effetto, mentre `top_p` è vincolato all'intervallo $[0.95, 1.00]$. Non è stato quindi necessario introdurre una configurazione specifica per DS4FP e sono stati usati i valori già definiti.

B

Risorse, artefatti e risultati

A completamento delle informazioni presentate nella tesi, è stato predisposto un *repository GitHub* che raccoglie le principali risorse utilizzate e prodotte nel corso del lavoro, scelte in quanto significative per documentarne lo sviluppo e i risultati ottenuti.

B.1 Contenuto e struttura del repository

All'interno del repository è presente una copia digitale della tesi in formato PDF, corrispondente al documento presentato, oltre a tre cartelle il cui contenuto è descritto di seguito.

La cartella `project_artifacts` mette a disposizione, nella loro forma completa, gli artefatti prodotti nell'ambito dell'implementazione, che nel Capitolo 5 sono stati descritti nei passaggi più importanti o solamente in sintesi. Tra di essi rientrano la definizione finale della Botnet Interface e i Code Templates utilizzati dal Code Agent. Questi includono le implementazioni di riferimento del modulo Botnet e del modulo Message Parser, che comprende anche l'enumerazione `Message_Type`. Viene quindi riportato il JSON Schema completo che definisce il Message Schema, accompagnato dal *sub-schema* IP Schema, utilizzato per rappresentare le informazioni relative agli indirizzi IP dei bersagli degli attacchi. È inoltre presente l'implementazione della classe `IP_Lookup`, attraverso la quale i Capture Module possono effettuare la popolazione dei campi dell'IP Schema. Viene poi presentato il Prompt Archive, al cui interno sono riuniti tutti i system prompt e gli user prompt utilizzati, organizzati secondo la stessa struttura seguita nella trattazione della Sezione 5.9. Infine, è stato incluso un file che elenca il sottoinsieme dei tool MCP di analisi statica del codice esposti dal SRE Framework e utilizzati nell'implementazione attuale, oltre al *Dockerfile* e al codice Python necessario alla creazione del container destinato al Sandbox Environment.

La cartella `experimental_setup` raccoglie le risorse utilizzate come riferimento per la preparazione e l'impostazione della fase sperimentale descritta nel Capitolo 6. Essa contiene innanzitutto il file `malware_samples.md`, che riporta l'elenco dei campioni considerati e le relative informazioni principali. Il contenuto riprende quello presentato nella Tabella 6.4, integrandolo con i collegamenti URL ai report dei campioni disponibili su VirusTotal [32] e MalwareBazaar [49]. Le due piattaforme forniscono ulteriori informazioni di threat intelligence relative ai campioni; MalwareBazaar offre anche la possibilità di scaricarli. Nella medesima cartella è presente la *YARA rule* utilizzata dal Classifier per il rico-

noscimento della famiglia *Mirai*, insieme al file della KNOWLEDGE BASE relativo alla stessa famiglia, realizzato dall'esperto di cybersecurity e utilizzato come conoscenza aggiuntiva dal SRE AGENT. Sono infine incluse le configurazioni di vLLM e i parametri di campionamento utilizzati per i LLM, già documentati nell'Appendice A.

La cartella `experimental_results` raccoglie tutti i risultati delle esecuzioni effettuate nel corso della fase sperimentale e utilizzati per le valutazioni presentate nel Capitolo 6. Per ciascun campione è presente una cartella dedicata, al cui interno i risultati sono organizzati in base al LLM impiegato, Q3627 o DS4FP. Per ogni combinazione di campione e LLM sono riportati i SRE REPORTS, i LOGGING FILES e il CAPTURE MODULE prodotto dal sistema. I SRE REPORTS comprendono gli output delle attività di TRANSPORT ANALYSIS e COMMUNICATION ANALYSIS della REVERSING TASK; per i campioni stripped sono inoltre presenti i report sintetici prodotti dai due agentic loop dedicati alla RENAMING TASK. A questi si aggiungono i LOGGING FILES corrispondenti alle attività svolte dal LLM nel SRE AGENT, sia sotto forma di conversazioni complete in formato JSON, sia in un formato semistrutturato, più facilmente consultabile. Infine, per ciascuna esecuzione è riportato il CAPTURE MODULE prodotto dal CODE AGENT, con il modulo principale BOTNET contenuto in un file denominato secondo la convenzione `sample_name.py` e con il modulo MESSAGE PARSER nel file `parser.py`.

B.2 Accesso al repository

Il repository *GitHub* è accessibile al seguente URL:

`https://github.com/xomarpvt/bachelor-thesis`

Per un accesso più immediato, è possibile utilizzare anche il codice QR riportato nella Figura B.1.



Figura B.1: QR code per l'accesso al repository *GitHub*.

Bibliografia

[1] Sérgio S. C. Silva, Rodrigo M. P. Silva, Raquel C. G. Pinto e Ronaldo M. Salles. "Botnets: A Survey". In: *Computer Networks* 57.2 (2013), pp. 378–403. DOI: 10.1016/j.comnet.2012.07.021.

[2] Moemedi Lefoane, Ibrahim Ghafir, Sohag Kabir e Irfan-Ullah Awan. "Internet of Things Botnets: A Survey on Artificial Intelligence Based Detection Techniques". In: *Journal of Network and Computer Applications* 236.C (2025), p. 104110. DOI: 10.1016/j.jnca.2025.104110.

[3] Alex Pavlovic. *DDoS in 2025: The Year Automation Took the Wheel*. Nokia Blog. 18 Dic. 2025. URL: https://www.nokia.com/blog/ddos-in-2025-the-year-automation-took-the-wheel/.

[4] Jiawei Li, Yizhe Yang, Yu Bai, Xiaofeng Zhou, Yinghao Li et al. "Fundamental Capabilities of Large Language Models and their Applications in Domain Scenarios: A Survey". In: *62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 11116–11141. DOI: 10.18653/v1/2024.acl-long.599.

[5] Asif Shahriar, Md Nafiu Rahman, Sadif Ahmed, Farig Sadeque e Md. Rizwan Parvez. *A Survey on Agentic Security: Applications, Threats and Defenses*. Preprint. 2025. arXiv: 2510.06445 [cs.CL].

[6] Juan Caballero, Pongsin Poosankam, Christian Kreibich e Dawn Song. "Dispatcher: Enabling Active Botnet Infiltration Using Automatic Protocol Reverse-Engineering". In: *16th ACM Conference on Computer and Communications Security*. Association for Computing Machinery, 2009, pp. 621–634. DOI: 10.1145/1653662.1653737.

[7] Hojun Jin, GyuHyun Jeon, Hee Won Aneka Choi, Seungho Jeon e Jung Taek Seo. "A Threat Modeling Framework for IoT-Based Botnet Attacks". In: *Heliyon* 10.20, e39192 (2024). DOI: 10.1016/j.heliyon.2024.e39192.

[8] Michael Sikorski e Andrew Honig. *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press, 2012. ISBN: 9781593272906.

[9] Nabil Hachem, Yosra Ben Mustapha, Gustavo Gonzalez Granadillo e Herve Debar. "Botnets: Lifecycle and Taxonomy". In: *2011 Conference on Network and Information Systems Security*. 2011, pp. 1–8. DOI: 10.1109/SAR-SSI.2011.5931395.

[10] Manos Antonakakis, Tim April, Michael Bailey, Matthew Bernhard, Elie Bursztein et al. "Understanding the Mirai Botnet". In: *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, 2017, pp. 1093–1110. URL: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis.

[11] Kara Nance e Chris Eagle. *The Ghidra Book: The Definitive Guide*. 2ª ed. No Starch Press, 2026. ISBN: 9781718504684.

[12] Andreas Moser, Christopher Kruegel e Engin Kirda. “Limits of Static Analysis for Malware Detection”. In: *Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007)*. IEEE, 2007, pp. 421–430. DOI: 10.1109/ACSAC.2007.21.

[13] Manuel Egele, Theodoor Scholte, Engin Kirda e Christopher Kruegel. “A survey on automated dynamic malware-analysis techniques and tools”. In: *ACM Computing Surveys* 44.2, 6 (2012), pp. 1–42. DOI: 10.1145/2089125.2089126.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser e Illia Polosukhin. “Attention Is All You Need”. In: *31st International Conference on Neural Information Processing Systems*. Vol. 30. Curran Associates, 2017, pp. 6000–6010. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

[15] Numa Dhamani e Maggie Engler. *Introduction to Generative AI. Reliable, Responsible, and Real-world Applications*. 2ª ed. Manning, 2026. ISBN: 9781633434882.

[16] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni e Percy Liang. “Lost in the Middle: How Language Models Use Long Contexts”. In: *Transactions of the Association for Computational Linguistics* 12 (2024), pp. 157–173. DOI: 10.1162/tacl_a_00638.

[17] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan e Yuan Cao. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *The Eleventh International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023. URL: https://openreview.net/forum?id=WE_vluYUL-X.

[18] Model Context Protocol. *What is the Model Context Protocol (MCP)?* URL: https://modelcontextprotocol.io/docs (visitato il 28/09/2026).

[19] Anna Gutowska. *What Is Model Context Protocol (MCP)?* IBM Think. URL: https://www.ibm.com/think/topics/model-context-protocol (visitato il 28/09/2026).

[20] Hanzhuo Tan, Qi Luo, Jing Li e Yuqun Zhang. “LLM4Decompile: Decompiling Binary Code with Large Language Models”. In: *2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 3473–3487. DOI: 10.18653/v1/2024.emnlp-main.203.

[21] Guoqiang Chen, Huiqi Sun, Daguang Liu, Zhiqi Wang, Qiang Wang, Bin Yin, Lu Liu e Lingyun Ying. *ReCopilot: Reverse Engineering Copilot in Binary Analysis*. Preprint. 2025. arXiv: 2505.16366 [cs.CR].

[22] Brian Caswell, Dustin Fraze, Sarah Smith, Rodrigo Racanicci, Tim Middleton-Sally, Shelby Hayes, Stanley He, Katy Smith, Bhakta Pradhan e Mike Walker. *Project Ire Autonomously Identifies Malware at Scale*. Microsoft Research Blog. 5 Ago. 2025. URL: https://www.microsoft.com/en-us/research/blog/project-ire-autonomously-identifies-malware-at-scale/.

[23] Xingyuan Wei, Ziyang Yu, Lihao Liu, Degang Sun, Wentao Li e Yan Wang. "MASDroid: A Multi-Agent System for Enhancing the Analysis of Android Malware". In: *2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2026, pp. 14147–14151. DOI: 10.1109/ICASSP55912.2026.11463206.

[24] Zion Leonahenahe Basque, Samuele Doria, Ananta Soneji, Wil Gibbs, Adam Doupé, Yan Shoshitaishvili, Eleonora Losiouk, Ruoyu Wang e Simone Aonzo. "Decompiling the Synergy: An Empirical Study of Human–LLM Teaming in Software Reverse Engineering". In: *Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2026. DOI: 10.14722/ndss.2026.240380.

[25] Salem Radey, Jack West e Kassem Fawaz. *Challenges and Future Directions in Agentic Reverse Engineering Systems*. Preprint. 2026. arXiv: 2604.14317 [cs.CR].

[26] Christopher S. Johnson, Mark Lee Badger, David A. Waltermire, Julie Snyder e Clem Skorupka. *Guide to Cyber Threat Information Sharing*. Pubblicazione speciale NIST SP 800-150. National Institute of Standards e Technology, ott. 2016. DOI: 10.6028/NIST.SP.800-150.

[27] TJ O'Connor. *Violent Python: A Cookbook for Hackers, Forensic Analysts, Penetration Testers and Security Engineers*. Syngress, 2012. ISBN: 9781597499576.

[28] Luciano Ramalho. *Fluent Python: Clear, Concise, and Effective Programming*. 2ª ed. O'Reilly Media, 2022. ISBN: 9781492056355.

[29] Object Management Group. *OMG Unified Modeling Language (OMG UML). Version 2.5.1*. Specifica formal/2017-12-05. OMG, dic. 2017. URL: https://www.omg.org/spec/UML/2.5.1/PDF.

[30] Python Software Foundation. *Python 3.13 Documentation*. URL: https://docs.python.org/3.13/ (visitato il 28/09/2026).

[31] Conda. *Conda Documentation*. URL: https://docs.conda.io/projects/conda/en/stable/ (visitato il 28/09/2026).

[32] VirusTotal. *VirusTotal*. URL: https://www.virustotal.com/ (visitato il 28/09/2026).

[33] YARA-X. *YARA-X*. VirusTotal. URL: https://virustotal.github.io/yara-x/ (visitato il 28/09/2026).

[34] Varatheepan Paramanayakam, Andreas Karatzas, Iraklis Anagnostopoulos e Dimitrios Stamoulis. "Less is More: Optimizing Function Calling for LLM Execution on Edge Devices". In: *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–7. DOI: 10.23919/DATE64628.2025.10992798.

[35] National Security Agency. *Ghidra. Software reverse engineering framework*. Ver. 12.0.4. 4 Mar. 2026. URL: https://github.com/NationalSecurityAgency/ghidra (visitato il 28/09/2026). Repository GitHub.

[36] Benjamin Ethington. *ghidra-mcp. Ghidra MCP Server – 200+ MCP tools for AI-powered reverse engineering*. Ver. 5.10.0. 15 Mag. 2026. URL: https://github.com/bethington/ghidra-mcp (visitato il 28/09/2026). Repository GitHub.

[37] Docker. *What Is a Container?* URL: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/ (visitato il 28/09/2026).

[38] FastMCP. *FastMCP: The Framework for MCP*. URL: https://gofastmcp.com/getting-started/welcome (visitato il 28/09/2026).

[39] OpenAI. *OpenAI API Documentation*. URL: https://developers.openai.com/api/docs (visitato il 28/09/2026).

[40] vLLM. *vLLM Documentation*. URL: https://docs.vllm.ai/en/stable/ (visitato il 28/09/2026).

[41] JSON Schema. *What Is JSON Schema?* URL: https://json-schema.org/overview/what-is-jsonschema (visitato il 28/09/2026).

[42] IPinfo. *IPinfo Lite Database*. URL: https://ipinfo.io/developers/ipinfo-lite-database (visitato il 28/09/2026).

[43] MaxMind. *GeoIP and GeoLite Databases Documentation*. URL: https://dev.maxmind.com/geoip/docs/databases/ (visitato il 28/09/2026).

[44] Qwen. *Qwen3.6-27B: Release*. 22 Apr. 2026. URL: https://qwen.ai/blog?id=qwen3.6-27b.

[45] DeepSeek. *DeepSeek V4 Preview Release*. 24 Apr. 2026. URL: https://api-docs.deepseek.com/news/news260424.

[46] Artificial Analysis. *AI Model Evaluations*. URL: https://artificialanalysis.ai/evaluations (visitato il 28/09/2026).

[47] Zscaler ThreatLabz. *Zscaler ThreatLabz 2025 Mobile, IoT, & OT Threat Report*. 5 Nov. 2025. URL: https://www.zscaler.com/campaign/threatlabz-mobile-iot-ot-report.

[48] Kaspersky. *IT Threat Evolution in Q2 2026. Non-Mobile Statistics*. 10 Ago. 2026. URL: https://securelist.com/malware-report-q2-2026-pc-iot-statistics/120960/.

[49] abuse.ch. *MalwareBazaar – Malware sample exchange*. URL: https://bazaar.abuse.ch/ (visitato il 28/09/2026).

[50] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj et al. "Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces". In: *14th International Conference on Learning Representations*. ICLR, 2026, pp. 40903–40986. URL: https://proceedings.iclr.cc/paper_files/paper/2026/file/444a3737adaee10d86ad2ef5f74468e6-Paper-Conference.pdf.

[51] NVIDIA. *NVIDIA H100 GPU Datasheet*. Mag. 2022. URL: https://resources.nvidia.com/en-us-hopper-architecture/nvidia-tensor-core-gpu-datasheet.

[52] Qwen. *Qwen3.6-27B*. 2026. URL: https://huggingface.co/Qwen/Qwen3.6-27B#best-practices (visitato il 28/09/2026). Hugging Face model card.

[53] DeepSeek. *Thinking Mode*. DeepSeek API Docs. 2026. URL: https://api-docs.deepseek.com/guides/thinking_mode (visitato il 28/09/2026).